



M256 Unit 6

UNDERGRADUATE COMPUTING

Software development with Java



Core system designs

Unit **6**

This publication forms part of an Open University course M256 *Software development with Java*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA
First published 2007.

Copyright © 2007 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in Malta by Gutenberg Press.

ISBN 978 0 7492 1567 5

The background of the page is a teal color with a vertical gradient. On the left side, there are several large, overlapping, semi-transparent spheres that resemble bubbles or droplets. These spheres are in various shades of teal and have highlights and shadows, giving them a three-dimensional appearance. They are positioned on the left side of the page, partially overlapping the teal background.

CONTENTS

1	Introduction	5
1.1	Models	5
1.2	Design in Units 6, 7 and 8	7
1.3	The aims of this unit	7
1.4	Studying this unit	8
2	Communication between user interface and core system	9
2.1	Initiating a use case	9
2.2	Returning results	15
2.3	Core system integrity	17
3	Getting information	19
3.1	Getting simple information	19
3.2	Getting complex information	31
3.3	Splitting the coordination	39
4	Anthropomorphism	45
5	Changing state	48
5.1	Creating a one-way link	48
5.2	Creating an object and a two-way link	55
6	Summary	71
Appendix	Hospital System implementation model (the structure so far)	73
	Glossary	77
	References	78
	Index	79

M256 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Sarah Mattingly, Course Chair and Author

Lindsey Court, Author

Marion Edwards, Software Developer

Rob Griffiths, Critical Reader

Benedict Heal, Critical Reader

Simon Holland, Author

Barbara Poniatowska, Course Manager

Barbara Segal, Author

Rita Tingle, Author

Richard Walker, Author and Critical Reader

Robin Walker, Author and Critical Reader

Ray Weedon, Academic Editor

Ray Welland, External Assessor, University of Glasgow

Julia White, Course Manager

Ian Blackham, Editor

Phillip Howe, Composer

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1

Introduction

In this unit, we begin to design the way in which a system will be implemented; our examples will be from the Hospital System. *Units 7 and 8* will continue this consideration of design issues. Because the user interface and the core system are treated as separate concerns, in these units we consider how the core system will work, leaving the user interface until later. In a real project, another team might work in parallel on the initial stages of the user interface design.

To decide what is required to implement the system, each task the system has to perform is investigated, breaking down what is required into separate steps within the core system. Then responsibility for each step is assigned to appropriate objects, and appropriate methods are specified for the objects of each class. As you will discover, there are usually many choices to be made when allocating responsibilities; in other words, there are many possible designs for a system, some better than others. It is this element of choice that makes the design phase both interesting and challenging.

As the design progresses, the decisions are documented in the form of models of the system.

Where the context is clear, we refer to the system instead of the core system.

1.1 Models

In *Unit 3* we analysed the requirements document for the Hospital System, and we produced a conceptual model of the relevant categories of real-world entities and their relationships. As software classes often usefully correspond to categories of real-world entities, this model provides an initial model of the structure of the core system, forming a sensible starting point for the design of the core system. We therefore begin work on the basis that the Hospital System will have the structure set out in its *initial structural model*, the class diagram of which is reproduced in Figure 1.

The conceptual model forms part of the documentation that will be made available to those interacting with the system, to help them to understand it.

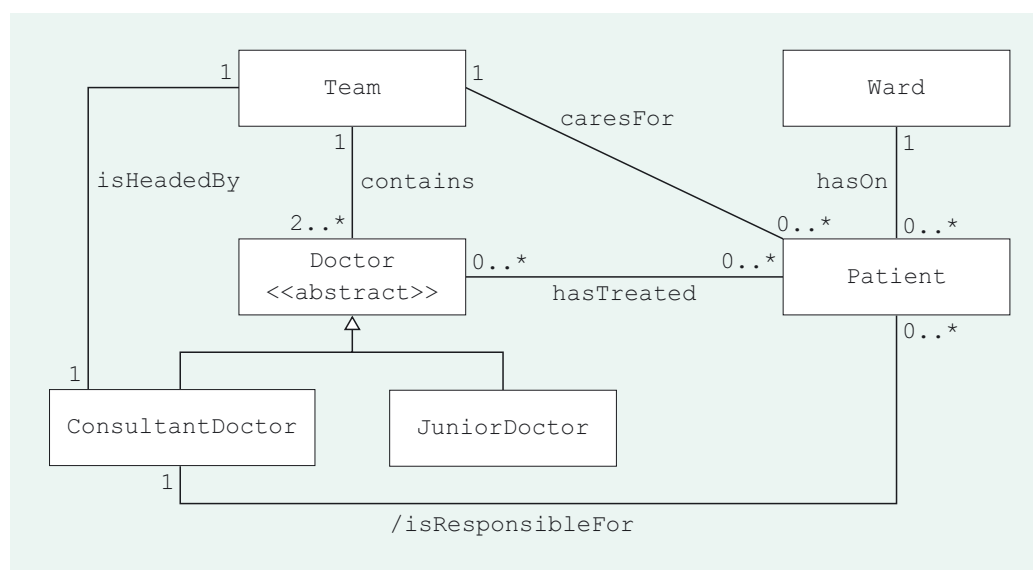


Figure 1 The class diagram from the initial structural model for the Hospital System

SAQ 1

A class diagram is part of an initial structural model. What other parts does an initial structural model have?

ANSWER.....

This was explained in
Unit 3, Section 8.

There is also a textual part. This contains a description of each class, and also sets out invariants – restrictions on the allowable states of objects and on the links between them that are not evident from the class diagram or class descriptions.

We initially view the Hospital System as having classes described in the initial structural model. That is, classes such as `Patient` and `Doctor`, whose instances and the links between them represent real-world entities – such as patients and doctors – and the connections between them.

Exercise 1

Suppose that the object diagram in Figure 2 below illustrates the state of part of the Hospital System at some point in time. What information is imparted by the diagram about:

- the software objects concerned;
- the real-world entities they represent?



Figure 2 The state of part of the Hospital System

Discussion.....

- The system contains a `Patient` object, referred to as `patient7`, and a `Ward` object, `ward9`. There is a link between these objects, but the diagram does not say anything about how this link is implemented in the software. Deciding how links between objects should be implemented will feature in our design deliberations.
- The diagram indicates that the patient represented by `patient7` has some connection to the ward represented by `ward9`. From your knowledge of the context of the system, you might expect the connection to be that the patient is on the ward.

The design process will generate a succession of structural models of increasing detail. The structure outlined in the initial structural model will be enhanced with the addition of classes, associations and methods, but we may also remove features if we decide that they are not required for our chosen designs. The initial structural model is just the starting point.

The design decisions will ultimately be presented as an **implementation model** for the Hospital System. An implementation model contains a structural model, which describes classes in terms of a class diagram and textual descriptions; it is the final result of the evolution of the initial structural model. The classes in an implementation model specify the actual software classes that will be used to implement the system; unlike the classes in the initial structural model, these implementation classes will have protocols containing method specifications.

An implementation model also contains **dynamic models** – sequence diagrams, for example – illustrating the actions and interactions within the system when it is in operation.

The diagram in Figure 3 below shows how the phase of development being explored in this and the next two units follows from the work you have already done, and how it leads to the subsequent phases.

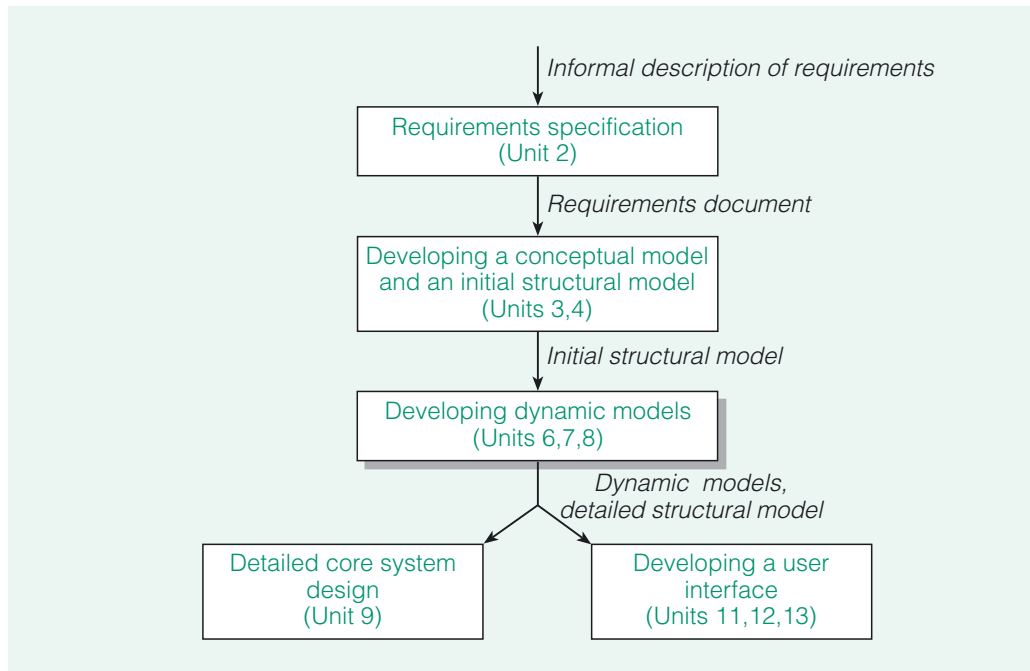


Figure 3 Software development in M256

1.2 Design in Units 6, 7 and 8

As you work through *Units 6, 7 and 8*, you will be introduced to some of the concepts and techniques that are involved in design. We do not aim to teach you how design is carried out in a *particular* software development method: the concepts and techniques we introduce can be employed in different ways within different development methods. To illustrate what is involved, we investigate examples from the Hospital System. In *Unit 6*, we produce a single design for each use case, before looking at alternative designs in *Unit 7*. In *Unit 8*, you will practice the skills you have learnt by applying them in the context of another small system.

1.3 The aims of this unit

This unit:

- ▶ introduces a simple way of dealing with communication between the user interface and the core system (Section 2);
- ▶ shows one way of dividing responsibility for a use case between the user interface and the core system (Section 2);
- ▶ analyses some use cases that do not involve changing the state of the core system (Section 3);
- ▶ analyses some use cases that do involve changing the state of the core system (Section 5);

- ▶ for each use case, shows what is involved in producing a dynamic model that illustrates a design (Sections 3 and 5);
- ▶ shows how, from a design, methods can be allocated to appropriate classes, and explains what is involved in navigating and creating links (Sections 3 and 5);
- ▶ briefly considers the use of anthropomorphic language in object-oriented software development (Section 4).

1.4 Studying this unit

This unit is a little longer than average, and may take you slightly more time to study than the *Study Calendar* indicates. However, *Unit 7* will be correspondingly shorter.

The unit contains a number of SAQs and paper-based exercises which are designed to reinforce your understanding of the concepts presented. These form an important part of the teaching strategy and you should work through them as they arise, and read the solutions provided before moving on.

There is no practical work on the computer in this unit. Sections 3 and 5 are the most substantial sections of this unit, requiring approximately equal study time. Section 2 might require approximately half the study time of Section 3. Section 4 is very short.

Throughout this unit, you will need to refer to previous information about the Hospital System – its requirements document and its initial structural model. These can be found in the *Handbook*. We suggest you keep it by your side as you work. Note that the implementation model in Section 5 of the *Handbook* incorporates all the design decisions taken in the development of the Hospital System. In the spirit of iterative development, some of the designs in this current unit are returned to later in the development process, and therefore do not exactly match the final versions recorded in the implementation model.

2

Communication between user interface and core system

In *Unit 2*, use cases – specific tasks – were identified for the Hospital System (see Section 3 of the *Handbook*), each being given a short name. For example, two of these were *Treat Patient* (labelled B) and *List Ward's Patients* (labelled E).

For each use case, the requirements document includes a description of what it involves. The description for *Treat Patient* is:

The administrator identifies the patient and the doctor; the doctor must be in the team that cares for the patient. The system records that the doctor has treated the patient.

In the operational system, the user interface and the core system will work together to enable each use case to be carried out. The core system will contain software objects – referred to as **core objects** – that represent the real-world objects of the use case descriptions – `Patient` objects and `Doctor` objects, for example, representing patients and doctors. The user interface will enable the user to interact with the core system in carrying out each use case, accessing and manipulating the core objects in clearly defined ways.

Using the two use cases mentioned above as illustrations, this section explores the division of labour between the user interface and the core system, and in particular how:

- ▶ the user interface tells the core system to begin its work;
- ▶ the core system returns results to the user interface.

There are various ways in which such communication can be accomplished. Recalling the ideas of *Unit 5*, the guiding principle will be that the user interface and the core system should be designed as separate components such that:

- ▶ the core system is protected from threats to its integrity through misuse by the user interface;
- ▶ the user interface is independent of how the core system is implemented;
- ▶ the user interface is replaceable without significant impact on the core system.

In M256, we assume that only one user interface is connected to the core system at any one time.

In M256, these components will be implemented as Java packages. Components may be implemented in other ways in other languages, so to keep our designs general at this stage, we refer to 'components' rather than 'packages'.

2.1 Initiating a use case

Each application of a use case has to be initiated in some way. In the M256 example systems, it is the user who does this, by causing a message to be sent from the user interface to the core system so that appropriate actions can be taken within the core system.

For example, for *Treat Patient*, the user (the administrator) will first identify a patient and a doctor in the user interface, and then instruct the user interface, perhaps by clicking a button or entering a command, to send the appropriate message (for example, `recordTreatment`) to the core system. The core system will then record the treatment of the patient by the doctor. Exactly what this involves is not the current concern; we will start investigating how the core system responds in Section 3.

The details of how the patient and doctor are identified are not of concern yet either. One possibility is that the identification might be by name. For example, the user could type in a patient's name, or select a name from a list. Later in the course, you will learn about

designing the user interface; until then there is no need to specify the means of identification.

Supplying information to the core system

What is of concern here is the *kind* of information the user interface should send the core system. For example, what arguments should be supplied within a message that requested the core system to record a treatment? You might reasonably suggest that, if the user identifies a patient by name, then this name could be supplied to the core system. But bear in mind that, as discussed in *Unit 5*, user interfaces change rapidly. Even if the initial user interface uses name identification, a future user interface may well use a different means of identification: perhaps the user might select from a set of displayed photographs of patients.

Future-proofing a system necessitates a means of interaction between the user interface and the core system that will allow for different kinds of user interface, involving different kinds of identification, to be used with the same core system in the future. There are different ways in which this can be done.

We will arrange for the user interface to pass appropriate core objects – that is, objects that are themselves part of the core system – into the core system. Here, the `Patient` object that represents the patient identified by the user would be a message argument. This might initially seem rather counter-intuitive, but it is actually rather a simple arrangement. The core system supplies the user interface with a collection of all the `Patient` objects. It is then up to that user interface, using whatever means of identification it offers its users, to select the appropriate `Patient` object and use it in sending a `recordTreatment` message, as illustrated in the sequence diagram in Figure 4.

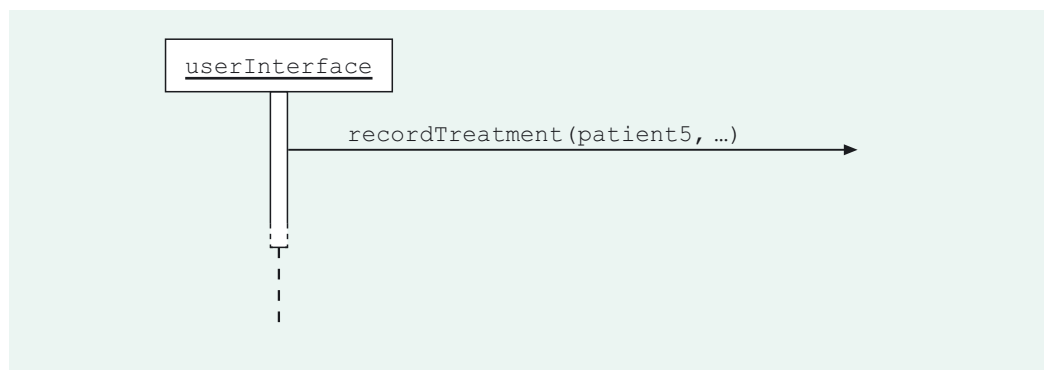


Figure 4 The user interface sends a `recordTreatment` message with a `Patient` object, `patient5`, as an argument. There may be additional arguments, as indicated by the '...' – this is not yet determined

Under this arrangement, how the user interface allows users to identify patients is no concern of the core system. As long as the core system is sent a suitable message including a `Patient` object as an argument, it will be able to do its job.

The coordinating object

The sequence diagram above models part of how the behaviour required for the *Treat Patient* use case might be initiated, but it does not show the object to which the `recordTreatment` message is sent. To which object or objects in the core system should the user interface send a message to initiate the actions within the core system? In *Unit 5* you were introduced to the idea that an object which we referred to as a *coordinating object* receives such initiating messages. We now need to explore this idea further, and explain how it fits into the current design phase. We start by considering

some alternative possibilities for how the core system might receive such an initiating message.

Consider *Treat Patient* again, and assume that the user has identified the appropriate patient and doctor in the user interface. We will refer to the corresponding objects as `patient5` and `doctor3`. There are a number of ways in which the core system could be told to record the treatment of `patient5` by `doctor3`.

The user interface could send a message directly to `doctor3` telling it to record the fact that it has treated `patient5`. This state of affairs is represented in the sequence diagram in Figure 5.

The names `patient5` and `doctor3` are simply labels we use in our discussions to refer to the objects involved. They are not identifiers in the Java sense.

The object `doctor3` could be an instance of either `ConsultantDoctor` or `JuniorDoctor`.

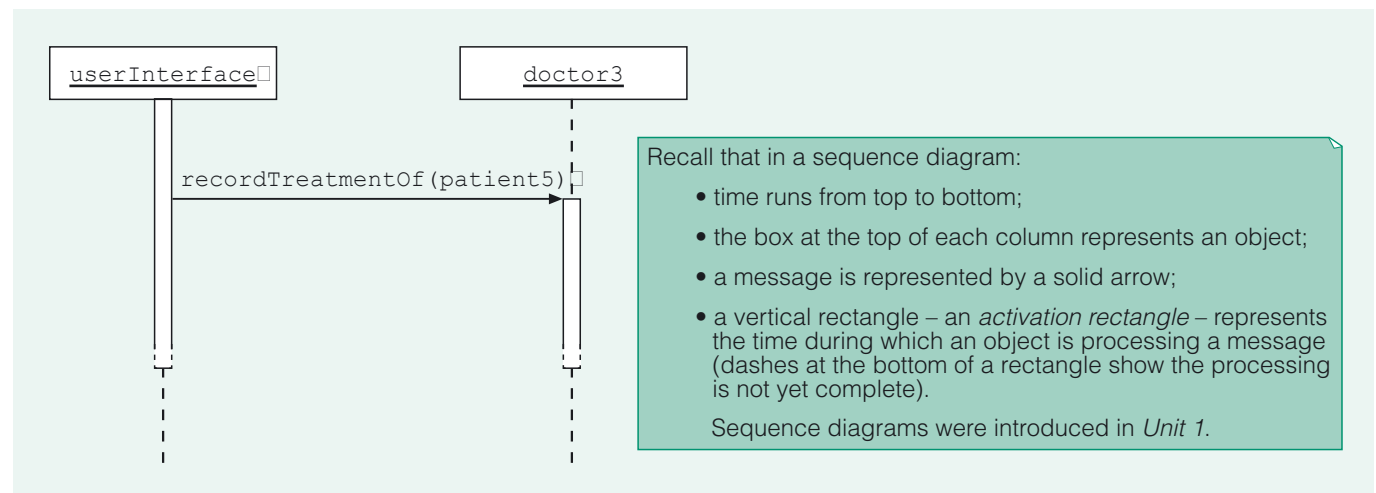


Figure 5 `doctor3` records the treatment

Figure 5 shows the message `recordTreatmentOf(patient5)` being sent from the user interface to `doctor3`. The object `patient5` is provided as the argument to this message in order to tell `doctor3` which `Patient` object is involved. At the arrowhead, an activation rectangle for `doctor3` starts. The method corresponding to the `recordTreatmentOf(patient5)` message is invoked, and `doctor3` starts processing the message.

Alternatively, the user interface could send a message to `patient5` telling it to record the fact that it has been treated by `doctor3`, as shown in Figure 6.



Figure 6 `patient5` records the treatment

Another possibility involves sharing the responsibility for recording the treatment between the `Doctor` and `Patient` objects, as shown in Figure 7.

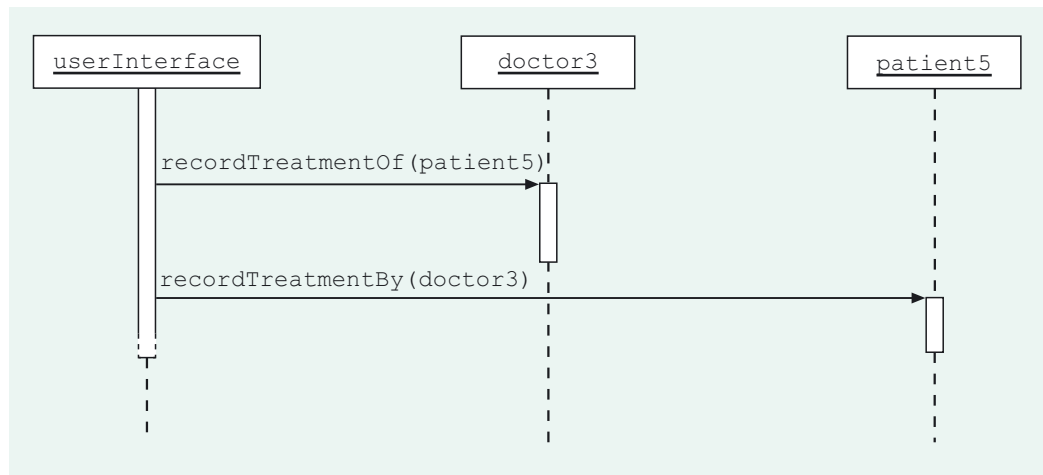


Figure 7 Both `doctor3` and `patient5` record the treatment

The trouble with all these possibilities is that they make the user interface dependent on how the core system works. If it is decided to make `Doctor` objects responsible for recording treatments, then the required messages for this use case will be initiated (as in Figure 5) by the user interface sending a message to a `Doctor` object. Any user interface that is used with the core system will send a `recordTreatmentOf(aPatient)` message to a `Doctor` object to initiate the behaviour required for this use case. However, later in the development process or when the system is extended, it may be decided that it would be better to make the `Patient` objects responsible for recording treatments (as in Figure 6). Then the user interface would have to be altered so that it sent a `recordTreatmentBy(aDoctor)` message to a `Patient` object to record a treatment. The change to the core system would therefore require changes to be made to the user interface. Since the user interface should remain independent of the core system, none of the possibilities considered so far is satisfactory.

A number of techniques have been devised to allow communication between the user interface and the core system while avoiding this kind of dependency problem. In M256 we use a simple technique which involves introducing an extra object, the *coordinating object*, to the core system. It is this object that is to be the receiver of all messages from the user interface that initiate the core system behaviour required for a use case. The coordinating object is the only instance of its class, the **coordinating class**. For the Hospital System, we will call the coordinating class and coordinating object `HospCoord` and `hospital` respectively. For the *Treat Patient* use case, the role of the coordinating object is shown in Figure 8.

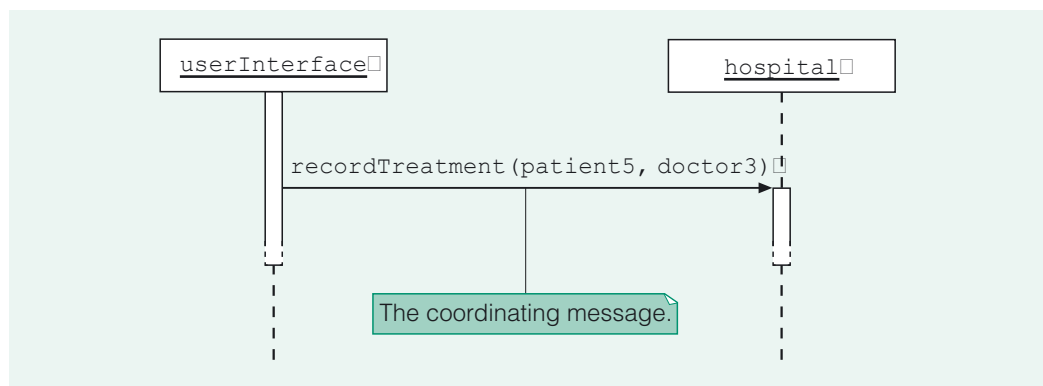


Figure 8 Using the coordinating object

The concept of a coordinating object was introduced in *Unit 5*, Subsection 5.2.

The coordinating object, `hospital`, receives a message such as `recordTreatment(patient5, doctor3)` from the user interface, telling it to record the treatment of `patient5` by `doctor3`. Such a message, from the user interface to the coordinating object to initiate the core system behaviour required for a use case, is called a **coordinating message**, and the corresponding method that is invoked on receipt of a coordinating message is called a **coordinating method**.

How the recording of the treatment is then accomplished is no concern of the user interface. It is not our concern at this point in the unit either: we devote the rest of this unit and the next to such design matters. But, to illustrate that different designs for the core system do not impact on the user interface, consider the two possibilities shown in Figure 9.

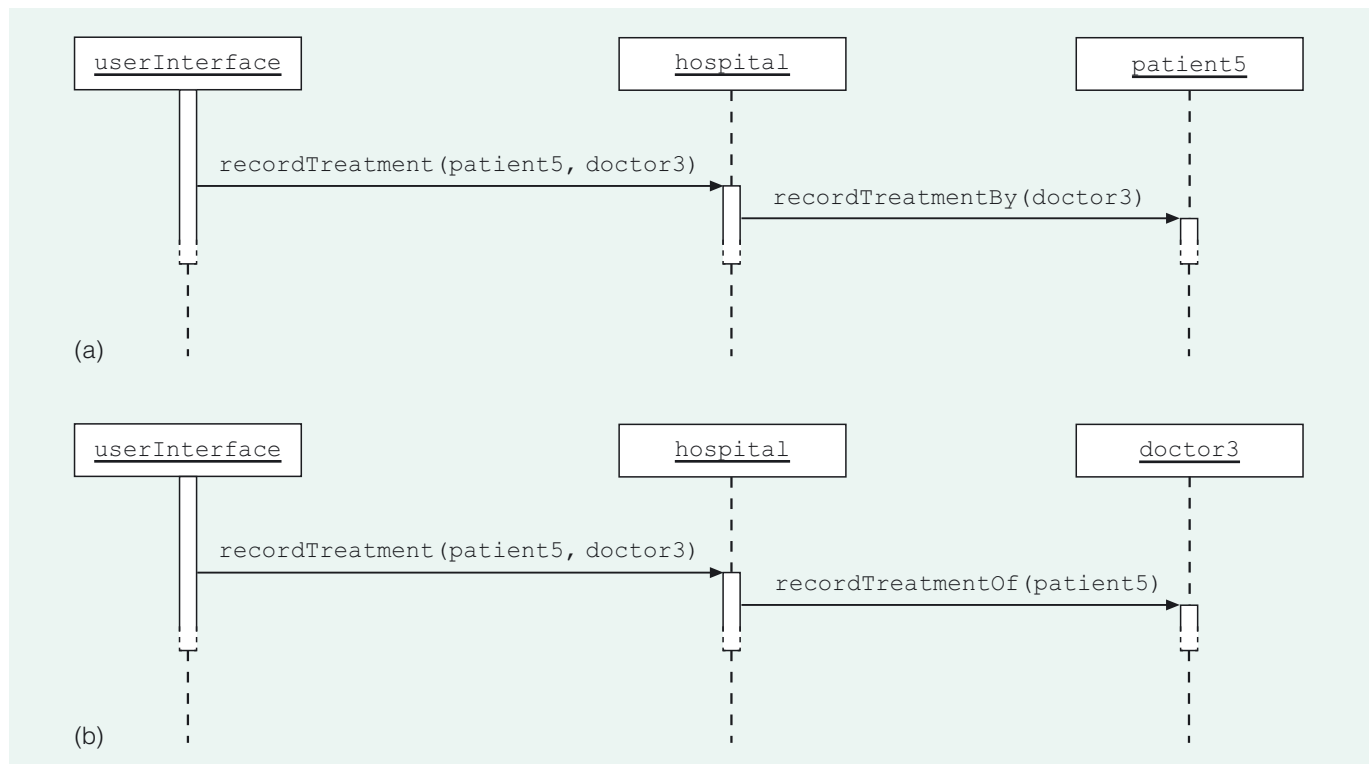


Figure 9 The user interface sends the same coordinating message whichever core design is adopted

SAQ 2

In Figure 9(a), what does the coordinating object, `hospital`, do when it receives the coordinating message?

ANSWER.....
`hospital` sends the message `recordTreatmentBy(doctor3)` to `patient5`, telling it to record the treatment.

SAQ 3

In Figure 9(a), how does `hospital` know which `Patient` object to send the message to?

ANSWER.....
 The relevant `Patient` object is provided as the first argument of the message sent from the user interface.

The important point illustrated by the diagrams in Figure 9 is that, regardless of how the core system carries out the behaviour required for the use case, the user interface always sends the same coordinating message to the coordinating object. It is the coordinating object that ensures the appropriate messages are sent to the other objects when a coordinating message is received: it *coordinates* the behaviour within the core system. In this way, the coordinating object acts as a kind of gatekeeper to the core system, providing, yet controlling, access to it. Figure 10 below illustrates this important principle.

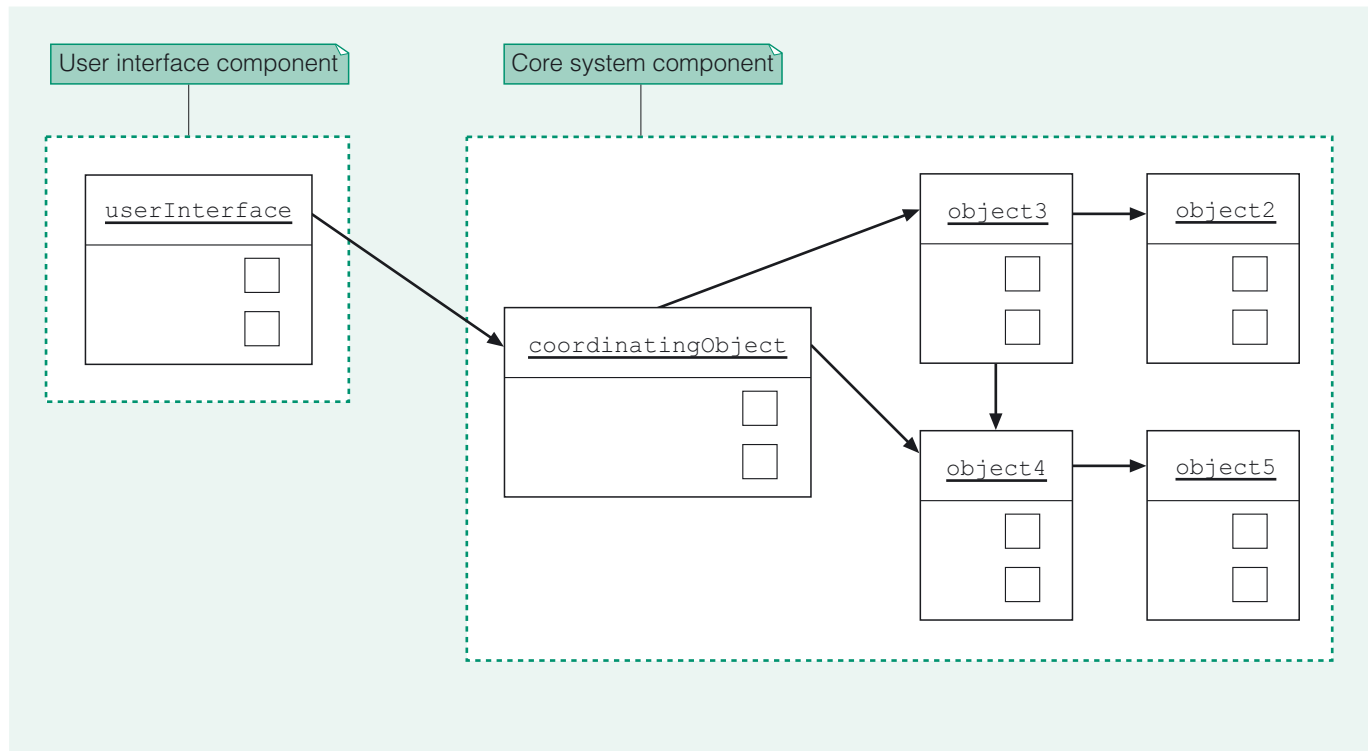


Figure 10 The coordinating object as gatekeeper to the core system

Subsequent development will follow the same pattern. The behaviour for each use case will be initiated by the user interface sending a coordinating message to the coordinating object. Core objects may be supplied as arguments to the message, providing information to the coordinating object.

Having decided to add the coordinating class `HospCoord`, we have made a start on the design process. The class diagram, depicting the structure of the system, evolves into the one below. There are, as yet, no associations between `HospCoord` and other classes; later design decisions will require some to be added.

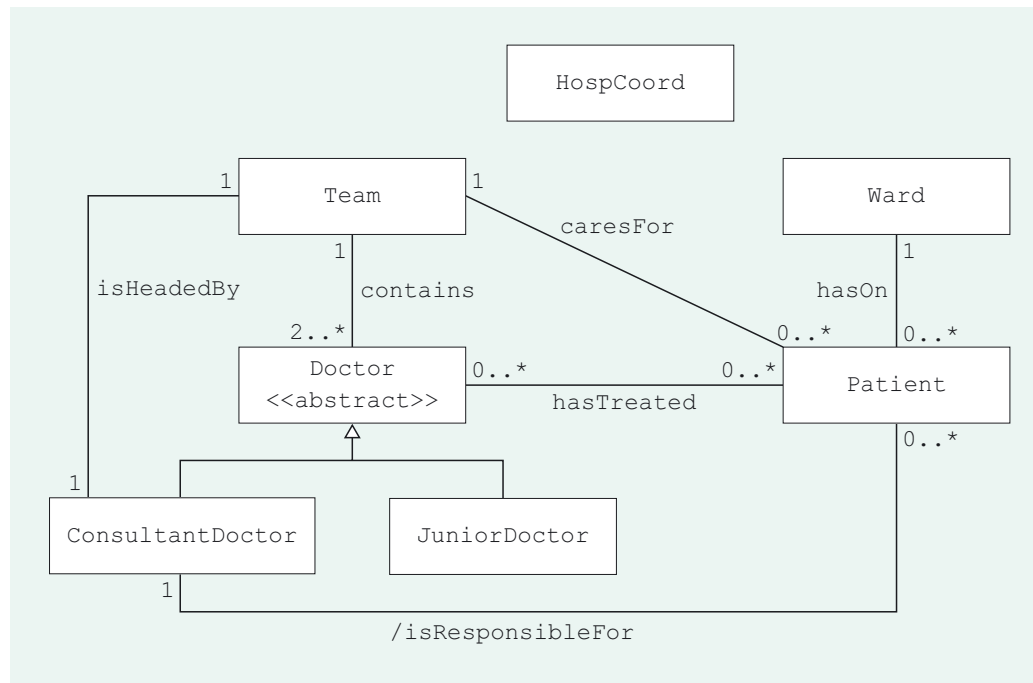


Figure 11 The class diagram for the Hospital System now includes the coordinating class

2.2 Returning results

It can be helpful to distinguish between use cases on the following basis.

- 1 Some use cases, such as *Treat Patient*, are about putting information into the system. They simply change the state of the system, but do not require any particular results to be displayed in the user interface.
- 2 Some, such as *List Ward's Patients* are about getting information from the system. They require results to be returned for display by the user interface, but do not change the state of the system.
- 3 Some involve both of these kinds of requirement.

As well as acting as the receiver of messages from the user interface, and coordinating the subsequent behaviour of the other objects in the core system, the coordinating object is responsible for returning any information required to be displayed to the user. An important initial decision is about the form in which any results should be returned from the core system to the user interface.

Consider the *List Ward's Patients* use case. Here is its description.

The administrator identifies the ward. For each patient on the ward the system displays the patient's name and age in years.

For the user interface to be able to display the patients' names and ages, we adopt the approach that the coordinating object simply returns a collection of the relevant **Patient** objects. This is illustrated in Figure 12 below. (The **Ward** object, **ward3**, represents the ward whose patients are wanted, and those patients are represented by **patient2**, **patient4** and **patient6**.) The user interface can then retrieve the names and ages from these **Patient** objects by sending appropriate getter messages (this part is not shown).

There are other design approaches to returning information, which we do not discuss here.

{...} simply denotes a collection of some kind.

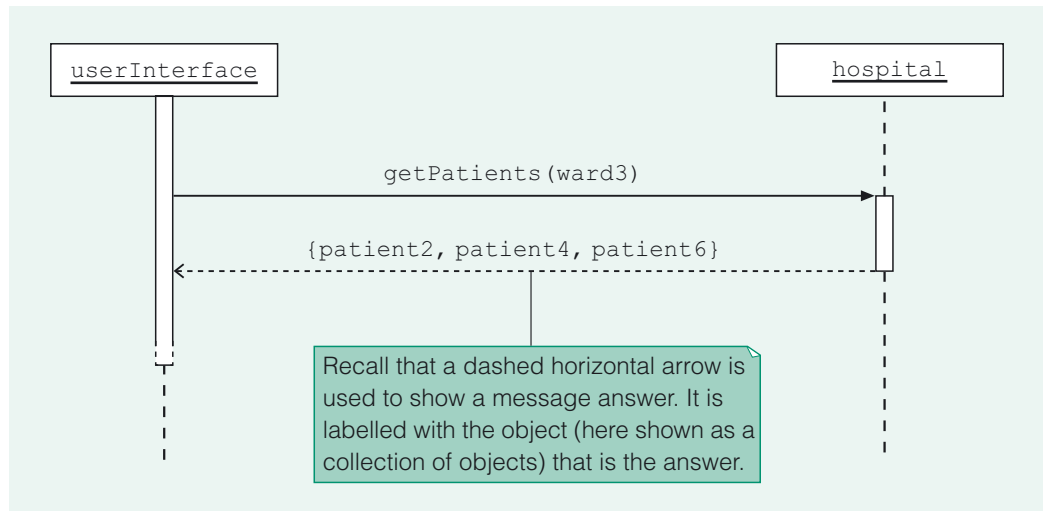


Figure 12 Returning objects to the user interface

The label on the message answer arrow shows that in this case a collection, `{patient2, patient4, patient6}`, consisting of the three `Patient` objects is returned. Since the current focus is on communication between the user interface and the core system, there are no details yet of how the collection is formed.

Returning objects as results

You may wonder why ‘whole’ `Patient` objects are returned and not just their names and ages. A system that simply returns the names and ages of the patients is certainly feasible. But returning the actual `Patient` objects rather than just their attribute values again contributes to making the core system independent of changes that may be made to the user interface in the future. For example, it may be decided that for this use case it would be better to display the dates of birth of the patients rather than their ages. This is essentially a change to the user interface rather than a fundamental change to the use case. By returning the `Patient` objects rather than particular attribute values, the core system does not need to be changed to accommodate this change to the user interface; a different user interface would simply retrieve the values of the `dateOfBirth` attributes rather than those of the `age` attributes from the `Patient` objects that are returned.

This same approach will be adopted for all use cases that require information from the core system to be displayed. The coordinating object will return the relevant core objects, and the user interface will be able to extract their attribute values for display. This splits the responsibility between the user interface and the core system. The core system is responsible for locating the objects that hold the required information, and returning them to the user interface; the user interface is responsible for retrieving the information from these objects.

The following exercise asks you to apply the ideas you have just learnt to the School System that you met in *Units 1* and *5*. There is no need for you to recall the details of the School System: the exercise is self-explanatory.

Exercise 2

The user interface for the School System displays the name of the oldest pupil in a selected form (that is, a class of pupils). Suppose that the following is a description of the corresponding use case.

The secretary identifies the form. The system displays the name of the oldest pupil in that form.

The core system contains, amongst others, the classes `Form` and `Pupil`.

- To initiate this use case, what object should the user interface send as an argument in its message to the coordinating object?
- What should the coordinating object return to the user interface?
- How will the user interface access the name of the pupil?
- What changes would be needed to the core system if it were later decided that the date of birth of the oldest pupil in the form should also be displayed?

Discussion.....

- The `Form` object corresponding to the form identified by the secretary.
- The `Pupil` object corresponding to the oldest pupil in the form.
- By sending the appropriate getter message to the `Pupil` object to retrieve the value of its `name` attribute.
- No changes to the core system would be needed. However, the user interface would need to send a different getter message to the `Pupil` object to retrieve the value of its `dateOfBirth` attribute.

2.3 Core system integrity

It is reasonable to question what the user interface will be able to do with the core objects it receives from the coordinating object. The answer, in short, is: not very much! The user interface will typically be able to query these objects about their attribute values – patient names and ages, and so on. In *Unit 5*, Section 6, you saw that allowing access to such information poses no threat to the integrity of the core system, so long as care is taken that this does not allow the attribute values to be changed.

SAQ 4

What term is used to describe a situation where it may be possible for a client to access and improperly manipulate the state of an object?

ANSWER.....

A privacy leak.

Privacy leaks were discussed in *Unit 5*, Subsection 6.1.

The user interface will *not* have access to messages that reveal the internal structure of the core system – such things as how one object is linked to another. Otherwise, the user interface may be dependent on a particular implementation of the core system and be forced to change when this implementation changes. It might also be able to manipulate the internal structure of the core system and invalidate the system's state. The core system is protected by allowing access only to messages that get information about attribute values so that all state-changing must be done via the coordinating object.

This section has shown how the design phase starts by specifying that the user interface communicates with the core system via a coordinating object, to maintain the integrity of the core system, and to keep the user interface and core system as independent of each other as possible.

In the next section, we turn to specifying what the core system must do for each use case, and setting out designs for achieving this.

However, before moving on, pause to think about the use of sequence diagrams. In *Unit 1* you used sequence diagrams to illustrate how an existing system works. You based your diagrams on knowledge of the system code: the code was the blueprint for the sequence diagram. Now, in the process of creating a system, we are using sequence diagrams in an explorative way. They are dynamic models, expressing outline ideas – designs – for how a system might work. Based on the designs we select, the code will then be written: the sequence diagrams will form a blueprint for the code.

3

Getting information

A coordinating object has been added to the core system and made responsible for communication with the user interface. It receives messages from the user interface which initiate the relevant behaviour in the core system, and it is responsible for returning core objects to the user interface.

We now begin to investigate what happens after the coordinating object receives a coordinating message. What must it do? That is, what must its coordinating method – the method that is invoked on receipt of the coordinating message – do? What actions must it initiate within the core system, and by what objects? And what sequence of messages will be needed to achieve these actions?

For each use case, the first task is to specify what it demands of the core system; that is, to specify the coordinating method for that use case. Then we can decide how this specification will be achieved by creating a dynamic model of the system in action. This dynamic model expresses a design, which forms the basis for assigning responsibilities to core objects.

In fact, there will typically be several possible designs corresponding to each use case. Consequently, an important activity at this stage of development will be to choose between them. Here we simply present a single design for each use case, to introduce you to what is involved. In *Unit 7* you will learn about alternative designs.

What is required of the core system varies depending on the purpose of the use case. In this section, we consider use cases whose purpose is to provide information to the user, so requiring the coordinating object to return objects to the user interface. Each example will illustrate important design concepts.

3.1 Getting simple information

The *List Ward's Patients* use case will be used to introduce you to the process of creating a design for a use case.

The description of this use case is as follows.

The administrator identifies the ward. For each patient on the ward the system displays the patient's name and age in years.

In Subsection 2.2 you saw that, for this use case, the core system's communication with the user interface could be modelled by looking at a typical example of the use case involving particular objects, namely, a `Ward` object, which we called `ward3`, and `Patient` objects, `patient2`, `patient4`, and `patient6`. An example of a use case such as this is called a **scenario**, and using such typical scenarios will be an important part of our design process. For example, what is involved in the design of *List Ward's Patients* will be examined in terms of the above scenario, whose objects we will shortly illustrate in an object diagram. However, although the scenario involves specific objects, the reasoning will be general, and from it we will derive a design that is applicable to whatever `Ward` and `Patient` objects are involved.

Like an object, which is an instance of a class, a scenario can be thought of as an instance of a use case.

Specifying the coordinating method

The scenario must be considered in detail to determine what the core system must do and how it can be done. The structure of the system, as it has been planned so far, provides the context. The relevant part of the class diagram is shown in Figure 13.

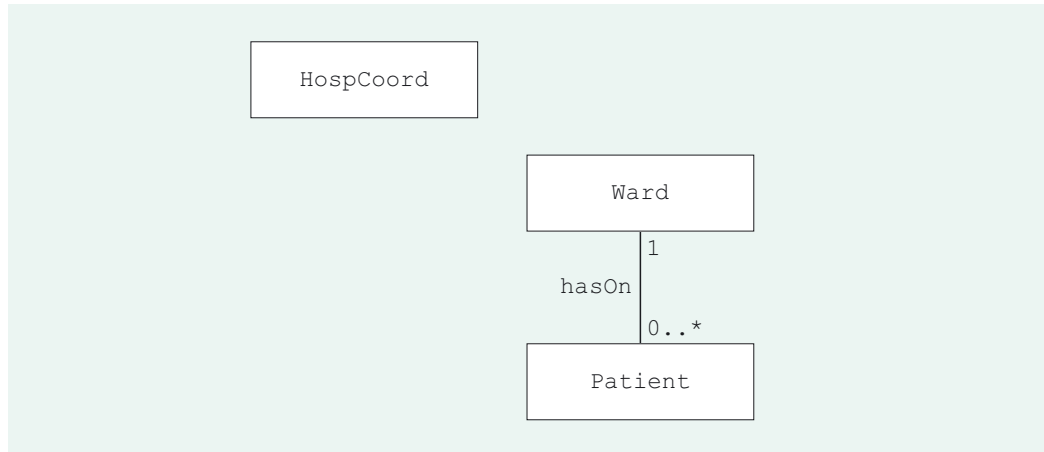


Figure 13 Classes and associations relevant to *List Ward's Patients*

The class diagram shows the classes `Ward` and `Patient` connected by the association `hasOn`. This illustrates the fact that a `Ward` object, which represents a ward, participates in `hasOn` links with the `Patient` objects that represent the ward's patients. Notice the presence in Figure 13 of the coordinating class, `HospCoord`.

On the basis of the class diagram, the object diagram in Figure 14 depicts the particular objects referred to in our scenario, by illustrating that the `Ward` object, `ward3`, is connected to each of the `Patient` objects `patient2`, `patient4` and `patient6` by a link of the `hasOn` association.

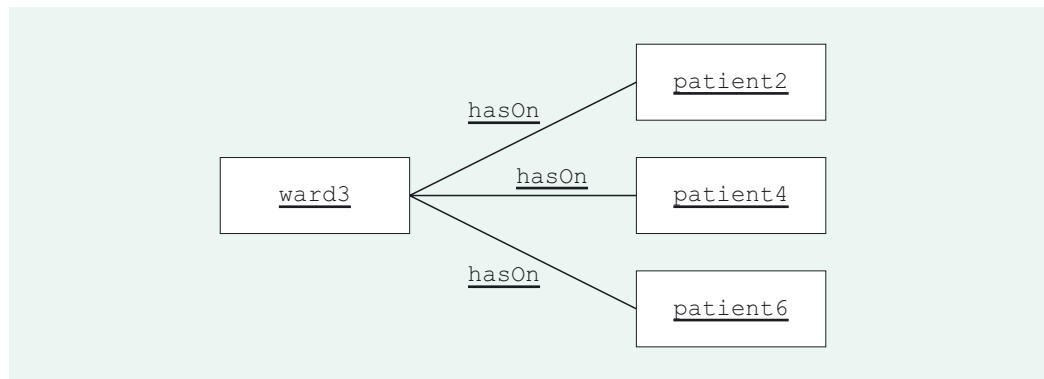


Figure 14 Links of the `hasOn` association connect `ward3` and its `Patient` objects

Earlier, in Subsection 2.2, we devised a sequence diagram to show communication between the user interface and the coordinating object (`hospital`) for this scenario. It is repeated below in Figure 15.

As object diagrams become more complex, links can be labelled for clarity. Here `hasOn` represents a link of the `hasOn` association. As for objects (being instances of classes), links (being instances of associations) have names which are underlined.

No attribute information is shown here, as none is relevant.

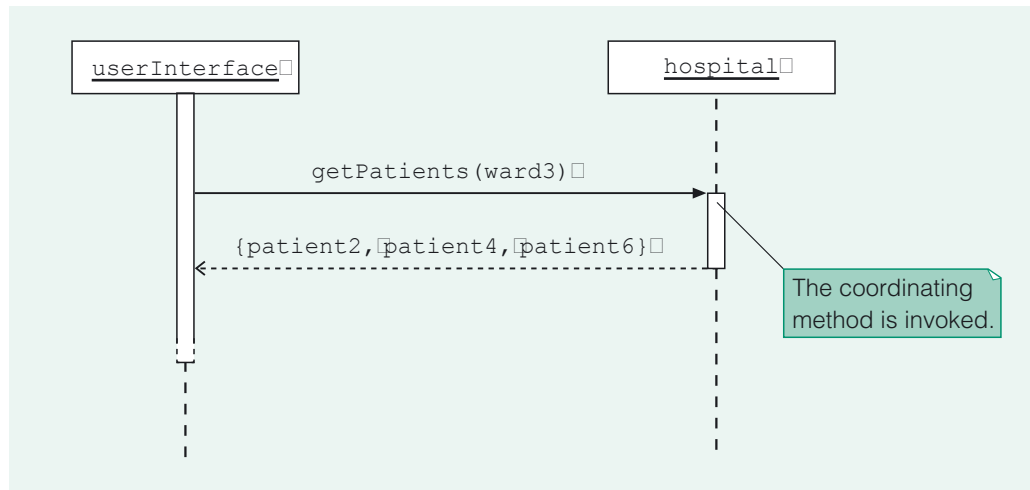


Figure 15 The *List Ward's Patients* coordinating method is invoked

The user interface sends a message to the coordinating object asking it to find the patients on a ward. It tells the coordinating object which ward is involved by supplying the corresponding `Ward` object as an argument.

On receipt of this coordinating message – `getPatients(ward3)` – the coordinating method is invoked; it must initiate the actions within the core system which result in the coordinating object returning the appropriate collection of `Patient` objects.

SAQ 5

A different design for *List Ward's Patients* is shown in Figure 16. Why is this inconsistent with the pattern adopted in Section 2 for communication between the user interface and the core system?

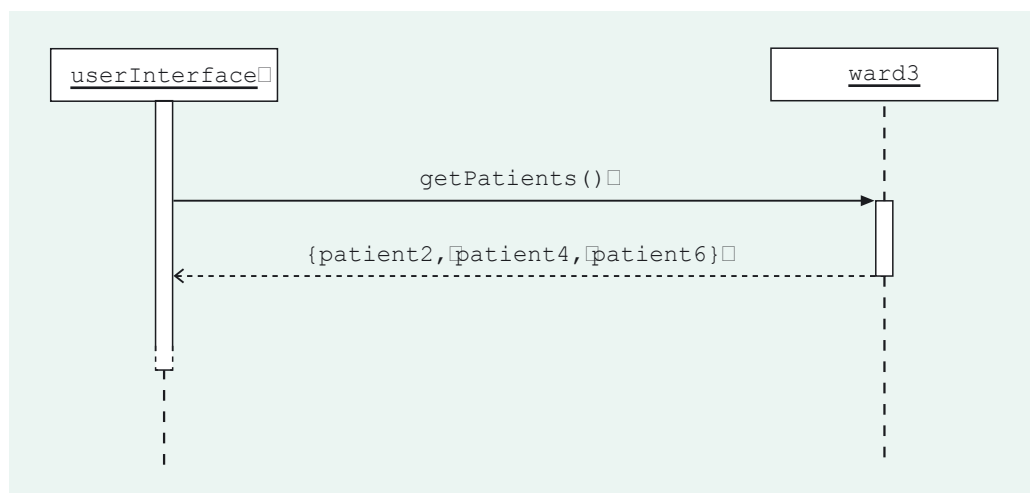


Figure 16 An alternative design for *List Ward's Patients*?

ANSWER.....

The sequence diagram shows the user interface sending a message directly to `ward3`, to get the `Patient` objects. You saw in Section 2 that to maintain the independence of the user interface from the core system the user interface should initiate the behaviour required for a use case by sending a message to the coordinating object.

We chose `ward3` and its particular `Patient` objects to illustrate what should typically happen for the production of a list of patients on a ward. This typical example can be used as the basis for stating what is generally required of the core system for this use case. This is done by specifying the coordinating method as follows.

Collection is of course a Java interface.

```
Collection <Patient> getPatients(Ward aWard)
```

Post-condition: returns a collection of all the `Patient` objects linked to `aWard`.

Note the following.

- 1 The coordinating method specification consists of the following.
 - (a) Information about invoking the method: the return type (the type of the method answer), the name of the method and the name and type of the method arguments. Here, we have:


```
Collection<Patient> getPatients(Ward aWard)
```

The return type is `Collection<Patient>`, the name of the method is `getPatients`, and the method takes a single argument, `aWard`, whose type is given as `Ward`.
 - (b) Information about the post-condition of the method – the consequences of validly invoking it. Here, we have ‘returns a collection of all the `Patient` objects linked to `aWard`’.
- 2 The coordinating method specification is in terms of general objects (`aWard`) rather than the specific objects of the scenario, and sets out, in the post-condition, what will be the effects of sending a valid coordinating message. This specification will form part of the system documentation, used by those working with the core system – user interface developers, for instance – as well as those maintaining the system in the future.
- 3 Some developers prefer to reserve the term ‘post-condition’ for the state changes effected by a method (there are no such changes here); our use of the term is wider, also encompassing a description of any results returned.
- 4 At this stage of development, to keep designs general, software developers often express them in a language-independent way. This requires use of some form of ‘design language’, which is beyond the scope of M256. Instead, we use Java-like notation for describing our designs, whilst keeping them sufficiently general that they could be implemented in a different language.
- 5 As development progresses, it is possible that Java implementation details may be chosen that do not correspond exactly with this current specification. For example, we may decide to hold `Patient` objects in a new kind of collection which does not implement the `Collection` interface. As with other aspects of the models being developed, the method specification may be revisited and revised later in the process. At this stage, it is a helpful representation of the current understanding of what is required.

Having set out *what* the coordinating method needs to achieve, the next stage is to decide *how* this will be done. We must decide what is required of core objects, and how each object will be told to carry out its tasks, so that the resulting interactions achieve the method’s post-conditions.

These interactions will be initiated by the coordinating method, to which a `Ward` object is supplied as argument. The question now is how the coordinating method is able to return the required collection of `Patient` objects. The object diagram shown in Figure 14, which illustrates the objects of our scenario for this use case, provides the key. The `Ward` object supplied is `ward3`, and it is linked to the `Patient` objects `patient2`, `patient4` and `patient6` by the `hasOn` association. It is these links of the `hasOn` association which, if suitably implemented, will allow a collection of the three `Patient` objects linked to `ward3` to be accessed. This is precisely what is needed here.

The argument `aWard` is a variable referencing a `Ward` object. Strictly speaking, we should write: ‘returns a collection of all the `Patient` objects linked to the `Ward` object referenced by `aWard`’. The briefer version will be used unless we need to emphasise issues of referencing.

How can links be implemented?

Recall that you first met associations, in *Unit 3*, as an element in modelling the real world. An association represents a relationship between two categories of real-world entity. The association `hasOn`, for example, represents a relationship between wards and patients. A link of an association represents a particular connection between individual real-world entities: the fact that a particular ward has on it a particular patient.

In the real world, it may not matter if a patient cannot tell us the ward they are on, or if a ward cannot reveal who its patients are. We still consider there to be a connection between the patient and their ward, regardless of what each knows about the other. When progressing towards implementing a system, however, more precision is required. We as humans might consider a `Patient` object and a `Ward` object to be linked if the real-world objects they represent are connected, as depicted by each `hasOn` link in the object diagram of Figure 14. But to the system, this means nothing unless each link is implemented, that is, unless there is something in the software to represent it.

There is more than one way of implementing (representing) links, but a common approach is for one or both of the linked objects to hold a reference to the other. This is the approach we take in M256.

The precise way in which this referencing is implemented depends upon the use the system needs to make of it. For the present use case, the coordinating object needs to access a collection of the `Patient` objects to which a `Ward` object is linked. This can be achieved by arranging for the `Ward` object to have an instance variable referencing a collection of references to the appropriate `Patient` objects. The object `ward3` thus references the `Patient` objects (indirectly, via a collection).

You looked briefly at this way of implementing links in the School System in *Unit 1*, Subsection 2.2.

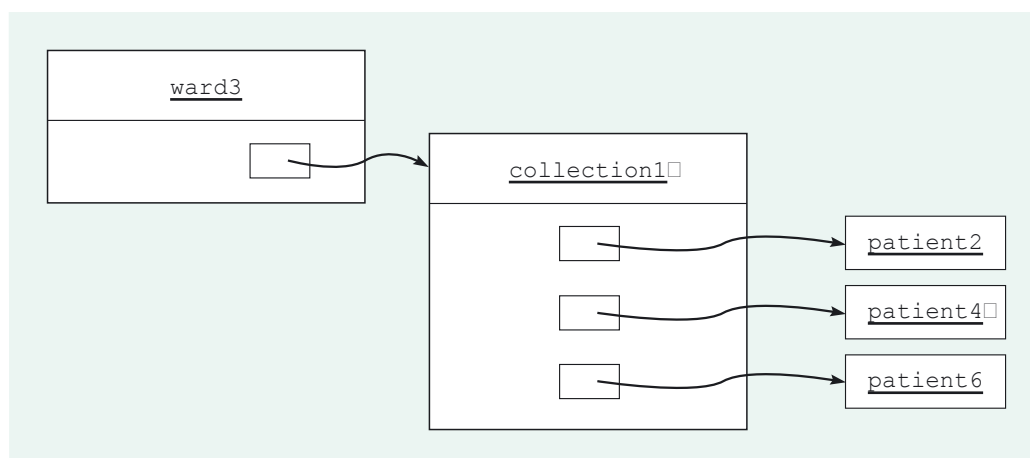


Figure 17 The object `ward3` holds a reference to a collection of references to `Patient` objects

Note that the implementation of referencing between objects corresponds to the multiplicities of the association between the classes of the objects. Looking at Figure 13, the association `hasOn` has multiplicity `0..*` at the `Patient` end: each `Ward` object may be linked with several `Patient` objects. Hence, as we have just discussed, a `Ward` object will reference a collection of the `Patient` objects to which it is linked. By contrast, `hasOn` has multiplicity `1` at the `Ward` end: each `Patient` object is linked with exactly one `Ward` object. So, if later designs require the `Ward` object linked to a `Patient` object to be accessed this will be achieved by giving each `Patient` object a reference to a single `Ward` object, as illustrated in Figure 18 overleaf.

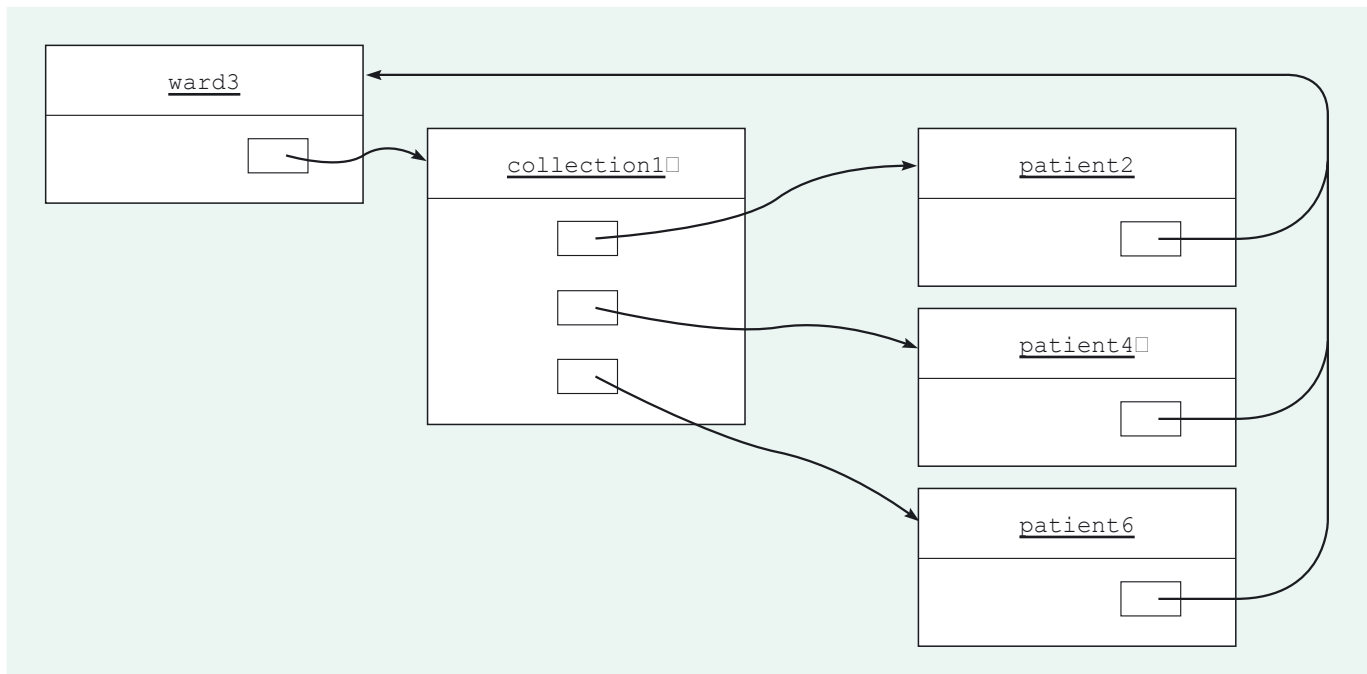


Figure 18 The Patient objects each hold a reference to ward3

The way in which links of an association are actually implemented depends upon the overall use that is made of the links in all the designs. All the use cases must be considered before making such implementation decisions. What is important here is that we have established that there will be some appropriate way for links to be implemented so that they can be used for accessing objects. Thus the design can safely require the collection of Patient objects linked to ward3 to be accessed.

A walk-through and a sequence diagram

We can now list in more detail what is involved for this particular scenario as follows.

Given: the Ward object, ward3.

- 1 Access the collection of Patient objects linked to ward3, which in this case is {patient2, patient4, patient6}.
- 2 Return {patient2, patient4, patient6}.

Such a list of the individual steps that are needed within the core system to carry out a particular scenario of a use case is called a **walk-through** of the use case. Here, the steps are simple and you may feel they are obvious; for other use cases much more will be involved.

The steps of a walk-through will be prompted by a sequence of messages passing through the core system, initiated within the coordinating method. We need to determine which object will carry out each step, and therefore what a suitable message sequence will be.

For this simple use case, it is clear which object should carry out each step. If ward3 references a collection of the Patient objects linked to it, then ward3 will be able to access the required collection. The coordinating method just has to send a message – let us call it `getPatients()` – to ward3 asking it for this collection of Patient objects.

Of course, for one object to send a message to another object, the first object must have a reference to the second. But that is just the situation we have here: the coordinating object, `hospital`, receives ward3 as an argument to the coordinating message, and so can reference ward3 and send it a message.

Note that the message sent by `hospital` to `ward3` has no argument. A common mistake would be to assume that `hospital` sends the message `getPatients(ward3)` to `ward3`, but the argument is not required (`ward3` does not need to be told who it is!).

The sequence diagram from Figure 15 can now be refined into the one in Figure 19 below.

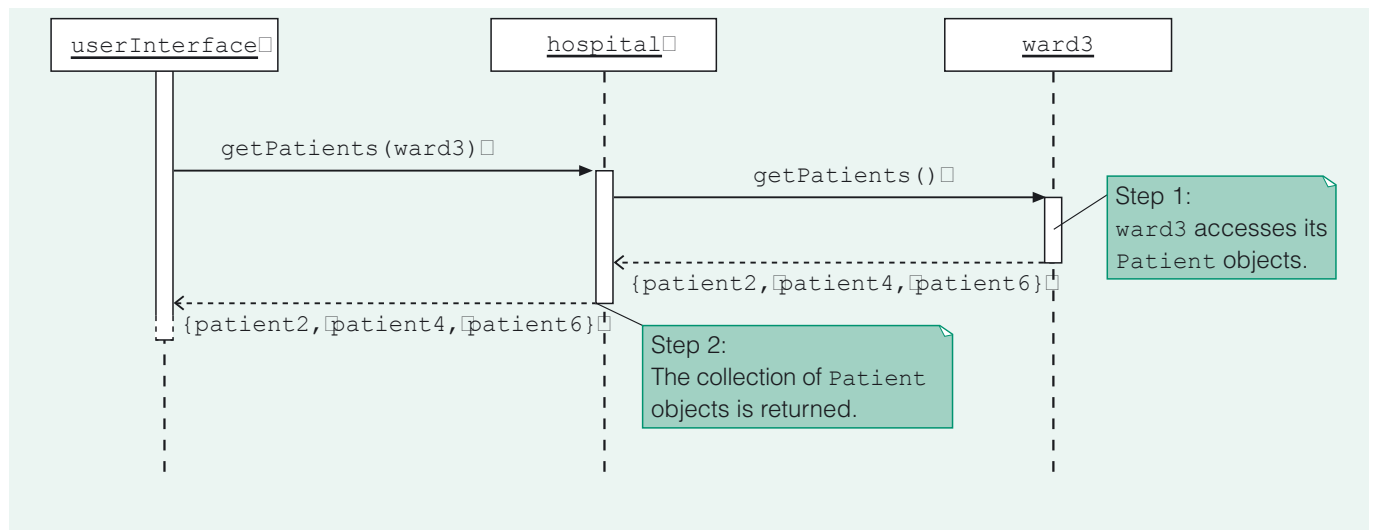


Figure 19 Sequence diagram for *List Ward's Patients*

Figure 19 shows how the steps of the walk-through described above are achieved. As part of its coordination role, `hospital` sends the `getPatients()` message to `ward3`, which, in response, accesses its `Patient` objects (step 1). This is a collaboration between `hospital` and `ward3`: they cooperate to complete the step. For this collaboration, `ward3` is the server and `hospital` is the client. On receiving the collection of `Patient` objects from `ward3`, `hospital` returns them to the user interface (step 2).

Recall that the client sends the message to the server to request a service; the server provides the service to the client.

Updating the structural model

The above walk-through and sequence diagram rely on no special features of the particular scenario: they apply for any `Ward` object, no matter what `Patient` objects it is linked to. They therefore form a dynamic model, providing the basis of a general design for how the core system will carry out its responsibility for listing a ward's patients.

The next task is to determine the consequences of the design for the structure of the system, and record them. The development of the core system began on the assumption that it would contain objects whose classes are described by the *initial structural* model – classes such as `Patient`, `Ward` and `Doctor`. As development progresses a succession of structural models is formed that iteratively refine the initial structural model. For example, we have already decided to have an additional class, `HospCoord`, which was added to the class diagram (see Figure 11).

Not only does the class diagram evolve throughout the design process; so too do the class descriptions. Each message shown in the dynamic model requires a corresponding method to be listed in the protocol of the appropriate class. And each time objects are accessed using links of a particular association, the responsibility for holding a suitable reference must be assigned to objects of the appropriate class. We describe the class as having a *responsibility for holding link references*.

The result of this process, when all use cases have been considered, will be the *implementation* model. This specifies the actual classes that will be used to implement the system, consisting of the final structural model (class diagram and associated text) and the dynamic models (walk-throughs and sequence diagrams) that represent the

designs and illustrate how the system will work. The implementation model will provide the basis for writing the code for the core system. It should ideally be sufficiently detailed that someone who has not been involved in the design stages can nevertheless write the code. It should also be sufficiently general to allow the designs it expresses to be implemented in different languages.

Section 5 of the *Handbook* shows a completed implementation model for the Hospital System. However, for ease of reference, at the end of this unit there is an appendix containing the structural part of an, as yet incomplete, implementation model that includes the consequences of all the designs developed in this unit.

For *List Ward's Patients*, the consequences of the design on the system structure must be recorded. Additional responsibilities are required for objects of the classes `Ward` and `HospCoord`. For `Ward` objects the situation is as follows.

- ▶ Each `Ward` object needs to hold a reference to a collection of all the `Patient` objects linked to it. To enable this, an instance variable `patients` which references such a collection should be specified for the `Ward` class.
- ▶ Each `Ward` object needs to respond to a `getPatients()` message, so the `Ward` protocol should include a specification for a method `getPatients()`. The method should return a collection of all the `Patient` objects linked to the receiver `Ward` object.

These responsibilities, added to the class description for `Ward`, result in the following.

Class	Ward	A hospital ward
Attributes		
	name	The unique name of the ward
	type	Whether the ward is for male or female (M or F) patients
	capacity	The maximum number of patients that can be on the ward at any one time
	/numberOfFreeBeds	The number of free beds on the ward
Links		
	Collection<Patient> patients References a collection of all the linked Patient objects.	
Protocol		
	Collection<Patient> getPatients() Post-condition: returns a collection of all the linked Patient objects.	

We use `patients` – plural – because it is a collection. For a reference to a ‘single’ object, we would use a singular name (e.g. `team`).

The **Attributes** part is unchanged here from the initial structural model.

The class details for `HospCoord` at this stage in the development simply include the coordinating method specification, as follows.

Class `HospCoord`

Attributes

None

Links

None

Protocol

`Collection<Patient> getPatients(Ward aWard)`

Post-condition: returns a collection of all the `Patient` objects linked to `aWard`.

Note the similarity, and the difference, between the methods `getPatients(aWard)` of `HospCoord` and `getPatients()` of `Ward`, reflecting the fact that in the design, `hospital` delegates responsibility for getting the `Patient` objects to the `Ward` object.

It is important to realise that the design so far generates no responsibilities for `Patient` objects. A common misconception is that accessing an object, or a collection of objects, involves sending a message to that object or collection of objects. This is not the case. The `Patient` objects here have a passive role. The `Ward` object holds a reference to them and returns them as a collection: no action is required of the `Patient` objects themselves.

Navigating an association

We have decided that each `Ward` object will hold a reference to a collection of all the `Patient` objects to which it is linked via `hasOn`, thereby enabling it to access each linked `Patient` object. Accessing a `Patient` object linked to a `Ward` object via `hasOn` is described as **navigating** the `hasOn` association from `Ward` to `Patient`.

You will see that when one use case requires an association to be navigated, this can have an impact on the design for another use case. It is therefore important to emphasise the direction of navigation of an association by adding an arrow to the association in the class diagram, as shown in Figure 20 overleaf, for the `hasOn` association. Note that this navigation arrow should not be confused with the arrows in a sequence diagram that represent messages. As noted above, the `Ward` object does not need to send any messages to `Patient` objects to access them. Rather, as you saw in Figure 19, it is the `Ward` object, `ward3`, that is sent a message to access the `Patient` objects which are linked to it.

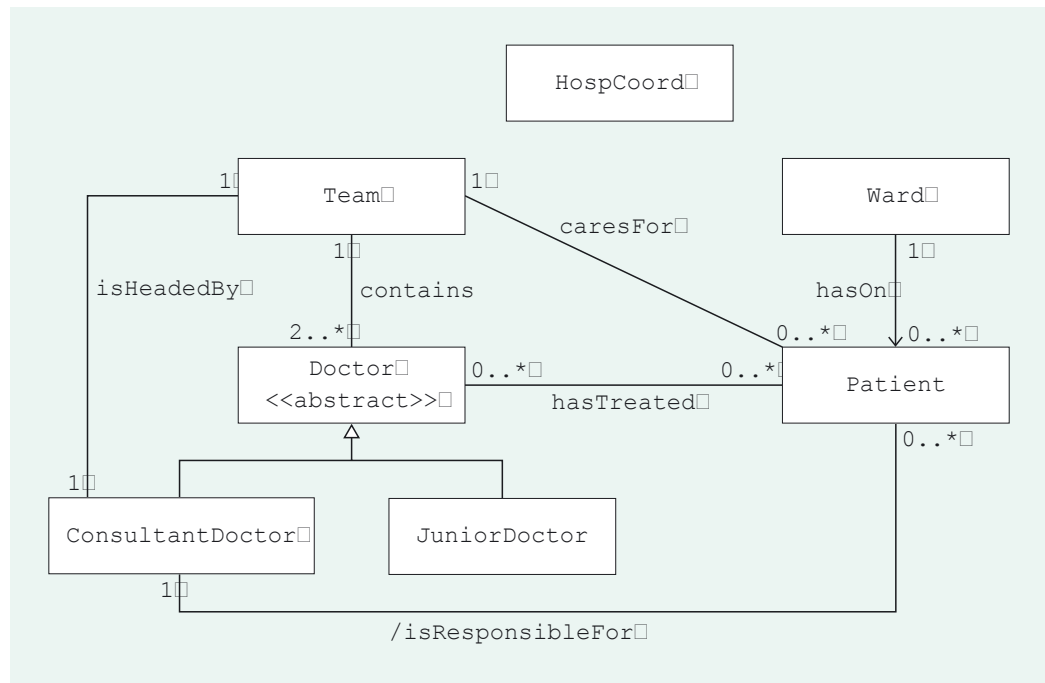


Figure 20 Showing the navigation of `hasOn` in the class diagram

We have mentioned the possibility that another use case may require the accessing of a `Ward` object linked to a particular `Patient` object. In that case, `hasOn` would also need to be navigated in the direction from `Patient` to `Ward`. Only after all use cases and designs have been examined can final decisions be made about what navigation is needed for each association. If an association is, at a certain point in the design process, navigated only in a single direction, it is said to be a **one-way association**, and its links are described as being **one-way links** at that point; if it is navigated in both directions, it is a **two-way association** with **two-way links**. At this point, `hasOn` is a one-way association.

Our approach to design

We can summarise our approach to this use case as follows. Remember that the design so far only covers what the core system must do. We have not yet considered how the user interface displays patient's names and ages.

- 1 We used the use case description from the requirements document and the class diagram from the initial structural model to invent a typical scenario, involving relevant example objects and links set out in the form of an object diagram. We supposed that we had `ward3` linked to the three `Patient` objects `patient2`, `patient4` and `patient6`.
- 2 We constructed (in Section 2) a sequence diagram illustrating communication between the user interface and the coordinating object for the scenario.
- 3 Generalising from our scenario led to our specification of the coordinating method for the use case as follows.

```
Collection<Patient> getPatients(Ward aWard)
```

Post-condition: returns a collection of all the `Patient` objects linked to `aWard`.

- 4 We determined the detailed steps required to carry out our scenario, that is, the steps that lead from what is given as an argument to the coordinating method (a `Ward` object) to what is required (the return of a collection of `Patient` objects).

This approach is summarised in Section 2 of the *Handbook*.

These steps were set out in the form of a walk-through for the scenario, as follows.

Given: the `Ward` object, `ward3`.

- 1 Access the collection of `Patient` objects linked to `ward3`, which in this case is `{patient2, patient4, patient6}`.
- 2 Return `{patient2, patient4, patient6}`.

We created a sequence diagram, extending the one created in step 2, to indicate how the steps of the walk-through can be prompted by sending messages to appropriate objects.

- 5 Because the particular scenario chosen was an example of what was generally required to fulfil the requirements of the use case, the walk-through and sequence diagram constitute a dynamic model, providing the basis for a design for the core system behaviour. The consequences of the design on the system's structure were recorded as follows:

- the requirements for classes to provide methods and to hold link references were identified and added to the class descriptions;
- corresponding to each responsibility of a class to hold a link reference, an association in the class diagram is navigated; this is shown with an arrow.

This resulted in an updated version of the developing implementation model.

This approach to design, which is summarised in Section 2 of the *Handbook*, is generally applicable, and we will repeatedly return to it.

The Appendix contains an updated version, for this unit, of the structural part of the implementation model.

SAQ 6

For this SAQ, you will need to look at the implementation model for the Hospital System in Section 5 of the *Handbook*.

- (a) What four components describe the dynamic model and its context for use case E, *List Ward's Patients*?
- (b) What elements identified from this dynamic model are incorporated into the structural model?

ANSWER.....

- (a) The four components given are:

- 1 the coordinating method that initiates the use case;
- 2 the object diagram illustrating the objects of a typical scenario;
- 3 the walk-through for the scenario;
- 4 the sequence diagram which shows how message passing within the system will achieve the walk-through.

The dynamic model consists of the walk-through and the sequence diagram, while the context is provided by the object diagram and coordinating method.

- (b) The elements incorporated into the structural model are as follows.

- 1 The specifications of the `Ward` method `getPatients()` and the `HospCoord` method `getPatients(aWard)` are included in the class descriptions for `Ward` and `HospCoord` respectively.
- 2 The responsibility of `Ward` for holding link references for the `hasOn` association is included in the **Links** section of the `Ward` class description.
- 3 The requirement for `hasOn` to be navigated from `Ward` to `Patient` is incorporated into the class diagram by an arrowhead at the `Patient` end of the `hasOn` association.

The implementation model referred to in SAQ 6 (Section 5 of the *Handbook*) reflects many more design decisions than have been covered at this stage. For example:

- ▶ some elements of the class descriptions are prefixed with the term 'public';
- ▶ types are specified for attribute values.

You will see such changes materialise as the design process continues.

Envisaging an implementation

At this stage, several use cases remain to be considered, with many decisions yet to be made before code can be written for the Hospital core system. It is possible that some future decisions could necessitate returning to and revising the designs for the *List Ward's Patients* use case we have just looked at.

Nevertheless, it should be helpful for you to momentarily move beyond the design process, and visualise how the design for *List Ward's Patients* could result in core system code.

SAQ 7

- (a) What message does the design for *List Ward's Patients* show hospital responding to? (See Figure 19.)
- (b) Describe what the corresponding method should involve.

ANSWER.....

- (a) The coordinating message `getPatients(ward3)`.
- (b) The method should send the message `getPatients()` to the `Ward` object provided as the argument, and return the answer from this message to the user interface.
-

Irrelevant code, and some comments, have been omitted. Do not be concerned at this stage with the access modifiers chosen.

Here are excerpts from Java class definitions which correspond to this design.

```
//Definition of coordinating class
public class HospCoord
{
    //code omitted

    public Collection<Patient> getPatients(Ward aWard)
    {
        return aWard.getPatients(); //send getPatients() to the argument
    }

    //code omitted
}

//Definition of Ward
public class Ward
{
    //code omitted

    private Collection<Patient> patients; //all the linked Patient objects

    Collection<Patient> getPatients()
    {
        return patients;
    }

    //code omitted
}
```

Note the following.

- The code does not specify what kind of collection of `Patient` objects is actually referenced by the `Ward` instance variable `patients`, just that the collection implements the interface `Collection<Patient>`. For example, it might be appropriate to use an instance of `HashSet`, having the following code within the constructor for `Ward`:

```
patients = new HashSet<Patient>();
```

These issues are not however pertinent at this design stage; they are an implementation concern.
- You will, hopefully, have realised that the code is actually very simple. It is quite possible that for this use case you could have bypassed the design activities we have illustrated, and written the same code straightaway. But as we consider more complex use cases, your appreciation of the benefits of a systematic design approach should increase.

3.2 Getting complex information

You have just seen a design for a use case whose coordinating method returns a straightforward collection of objects, all of the same class (`Patient`). In the next example, you will learn how a more complex collection of objects can be returned, and you will gain practice in employing our approach to design.

Here is the description of the *List Team's Patients* use case (labelled F in the requirements document).

The administrator identifies the team. For each patient cared for by the team, the System displays:

- the patient's name;
- the name of the ward that the patient is on.

The part of the class diagram relevant to this point is shown in Figure 21.

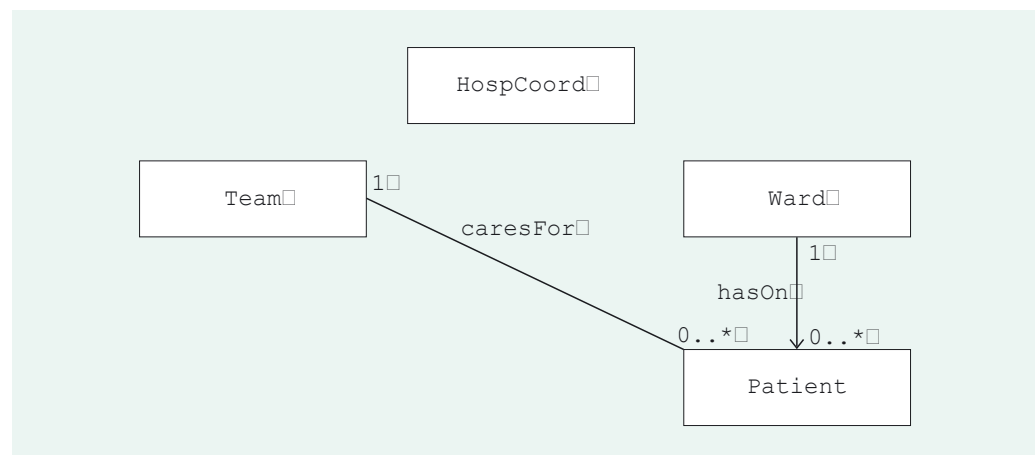


Figure 21 Classes and associations relevant to *List Team's Patients*

In a typical scenario, suppose that the `Team` object corresponding to the team identified by the administrator is `team2`, and that, based on the class diagram, the relevant `Patient` and `Ward` objects are as shown in the object diagram in Figure 22 overleaf.

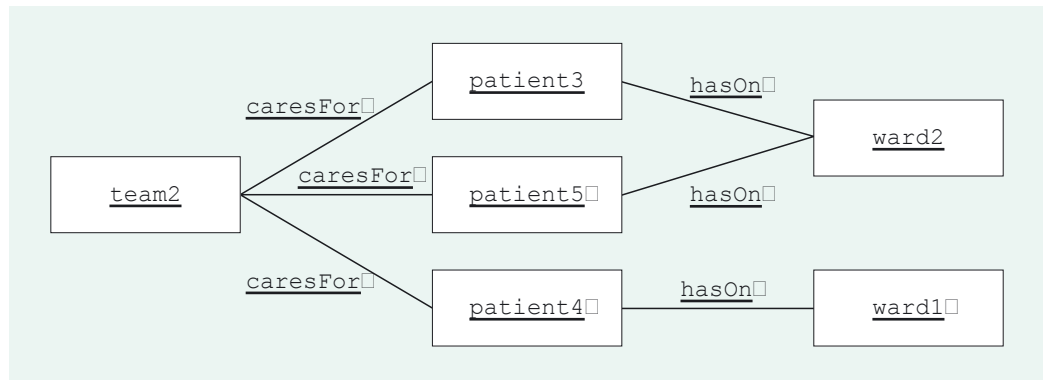


Figure 22 Object diagram for the *List Team's Patients* scenario

For the user interface to be able to display a patient's name along with the name of the ward that the patient is on, we will arrange for the coordinating object to return a collection of pairs, each consisting of a `Patient` object and the corresponding `Ward` object.

Exercise 3

Sketch a sequence diagram illustrating communication between the user interface and the coordinating object, `hospital`, for this scenario. It should show the coordinating message `getPatientsAndWards(team2)` being sent from the user interface to `hospital`, and it should show the results returned from `hospital` to the user interface.

Use notation such as `(patient3, ward2)` for pairs of objects.

Discussion.....

Our sequence diagram is shown in Figure 23 below. You might have listed the pairs in a different order.

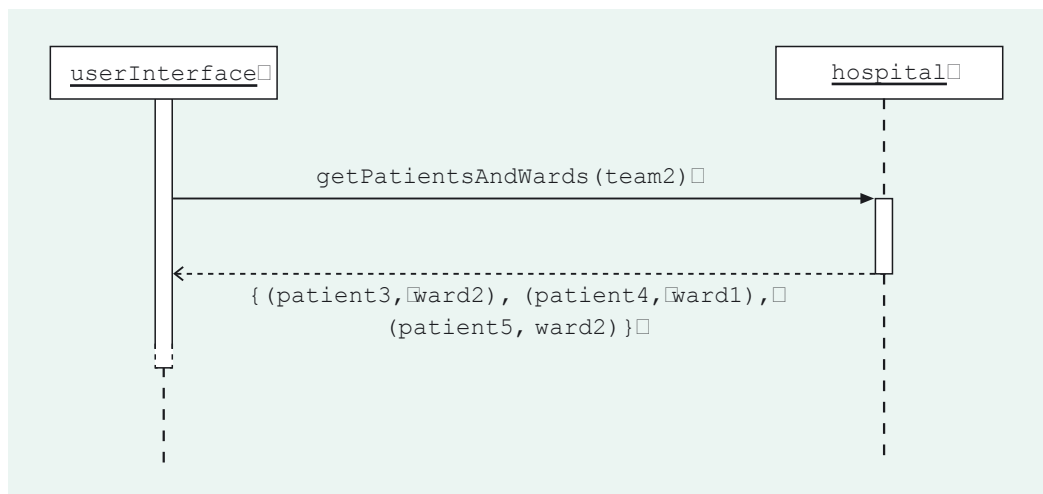


Figure 23 Communication with the user interface for the *List Team's Patients* scenario

Specifying the coordinating method

For each patient cared for by the team, the coordinating method needs to return both the `Patient` object and the `Ward` object corresponding to the ward the patient is on. From the object diagram in Figure 22, you can see that the required `Patient` objects are those linked to the supplied `Team` object via `caresFor`. And for each such `Patient` object, the corresponding `Ward` object is the one linked to it via `hasOn`.

There are several forms which a collection of such pairs of objects could take. For example, we could define a new class whose objects have two attributes, one a `Patient` object and the other a `Ward` object; such a class might be called `PatientWard`. Then some kind of collection could be used to hold `PatientWard` objects. Or something like a Java `Map` object might be employed, with `Patient` objects as keys and, for each `Patient` object key, the corresponding `Ward` object as the value.

For simplicity we will use the `Map` object option, and specify the coordinating method as follows.

```
Map<Patient, Ward> getPatientsAndWards(Team aTeam)
```

Post-condition: returns a map of pairs of `Patient` and `Ward` objects, such that for each `Patient` object `aPatient` linked to `aTeam`, the map contains the key-value pair (`aPatient`, `aWard`), where `aWard` is linked to `aPatient`.

Next, we need to decide how the specification is to be achieved.

Devising a walk-through and a sequence diagram

We start with what is given in our scenario.

Given: the `Team` object `team2`.

The user interface supplies `team2` to `hospital` when it initiates the use case. The coordinating object `hospital` needs to ensure that all the `Patient` objects linked to `team2` are accessed. This can be done if `team2` references a collection of the `Patient` objects linked to it via `caresFor`.

- 1 Access the collection of `Patient` objects linked to `team2`, which in this case is {`patient3`, `patient4`, `patient5`}.

To carry out step 1, `hospital` should send a message to `team2`. We will call this message `getPatientsAndWards()`. This is shown in Figure 24.

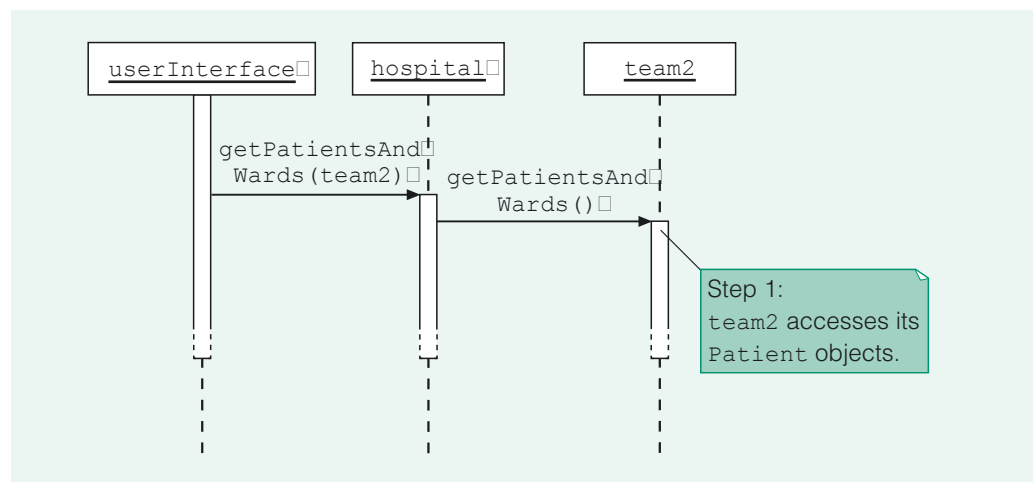


Figure 24 Getting `team2` involved

If each `Patient` object holds a reference to the `Ward` object representing the ward the patient is on, this `Ward` object can be accessed, and the pair added to a collection. First a new empty collection is formed to hold the results.

- 2 Create a new empty collection, `results`.

For each `Patient` object, `aPatient`, accessed in step 1:

- 3 access `aWard`, the `Ward` object linked to `aPatient`;
- 4 add (`aPatient`, `aWard`) to `results`.

Most object-oriented languages have the equivalent of Java maps and collections.

You will see shortly that `team2`'s response to `getPatientsAndWards()` is to return `Ward` objects as well as `Patient` objects, hence the name of the message.

To achieve step 3, `team2` will need to send a message to each of the `Patient` objects accessed in step 1, asking it for its `Ward` object. Steps 3 and 4 are carried out for each `Patient` object linked to `team2`; the indentation of the text of these steps in the layout of the walk-through indicates that they are repeated for each of the relevant `Patient` objects. This is illustrated in Figure 25. Recall that in this particular scenario there are three patients.

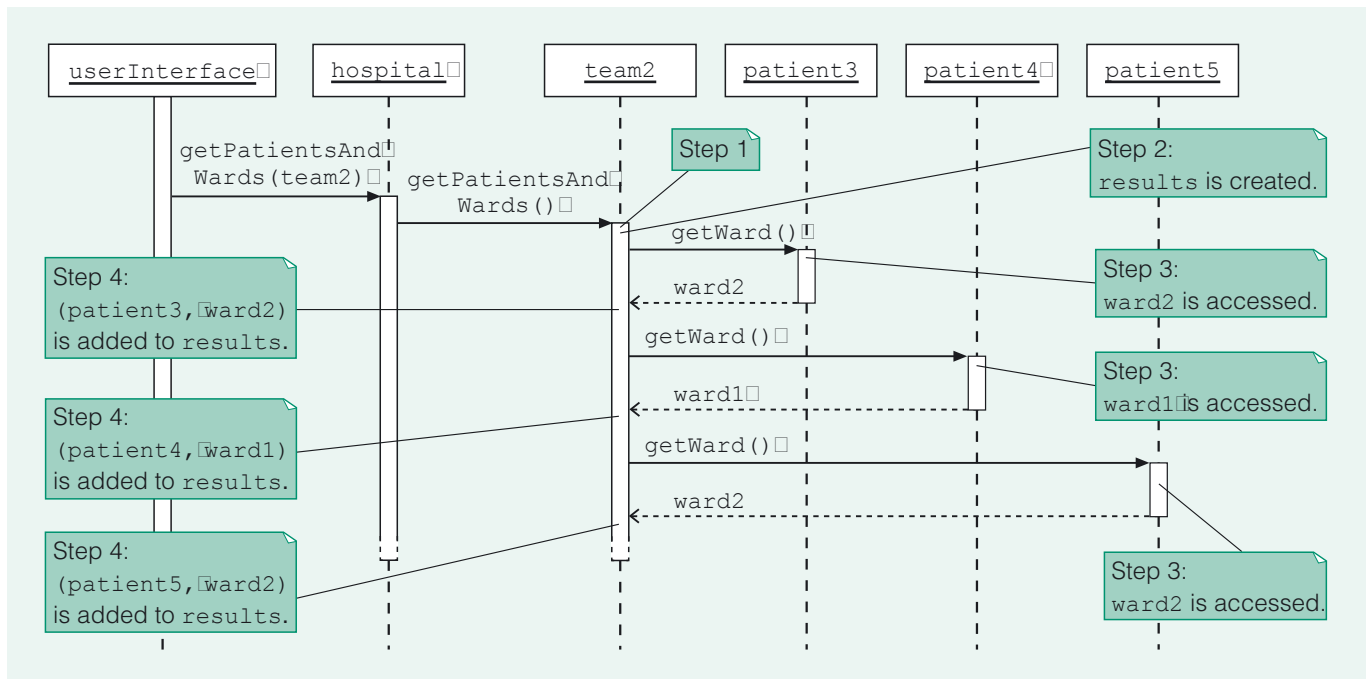


Figure 25 Getting the patients to reveal their wards

Note the following.

- An explicit formation of a collection is involved for this use case, in contrast to the previous one. There is no collection of (`aPatient`, `aWard`) pairs available in the system: instead one needs to be formed.
- The order in which the `Patient` objects are sent the `getWard()` messages is immaterial.
- We have labelled the diagram, showing where some of the walk-through steps are accomplished. Such annotations are not always required, but can be helpful to emphasise the relationship between the walk-through and the sequence diagram.

As `team2` has formed the required collection of pairs of `Patient` and `Ward` objects, it can return this collection to `hospital`, which in turn can return it to the user interface. Putting all the steps together gives the following walk-through and sequence diagram for this scenario (the sequence diagram in Figure 26 extending that presented in Figure 23).

Given: the `Team` object `team2`.

1 Access the collection of `Patient` objects linked to `team2`, which in this case is `{patient3, patient4, patient5}`.

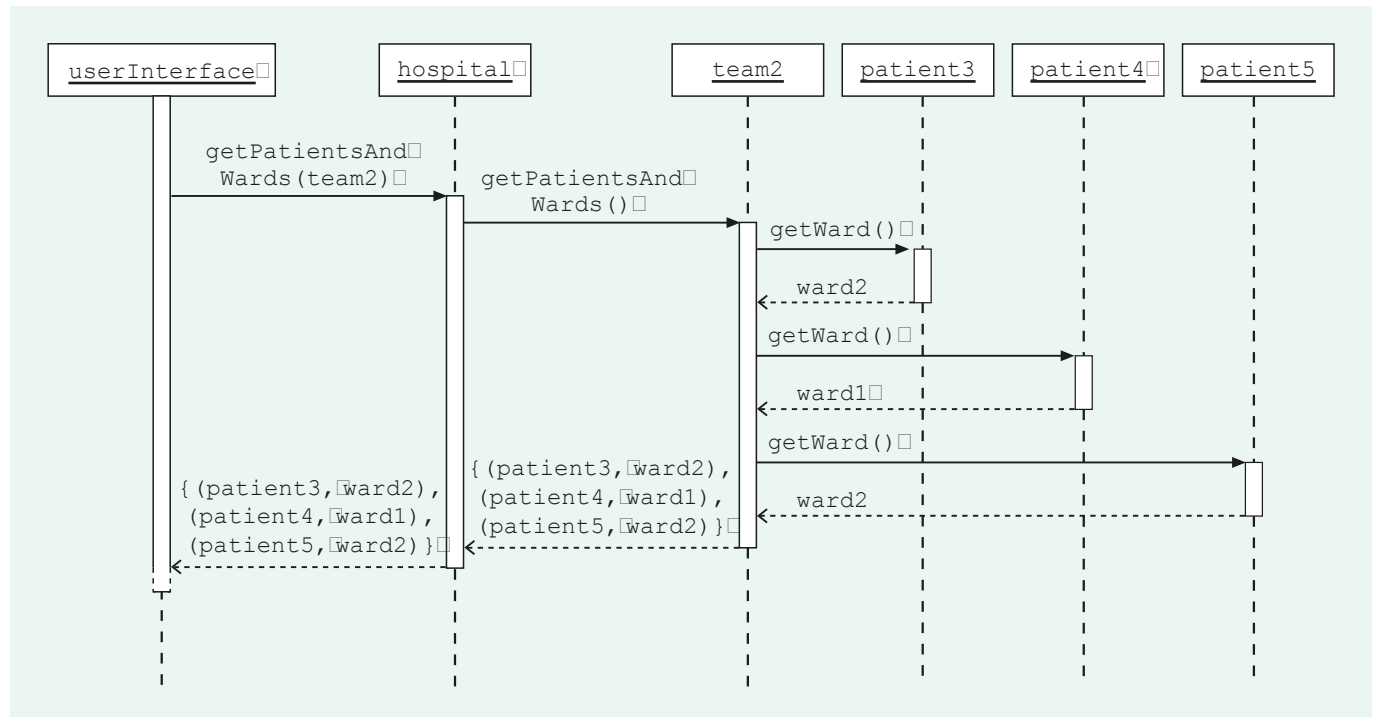
2 Create a new empty collection, `results`.

For each `Patient` object, `aPatient`, accessed in step 1:

3 access `aWard`, the `Ward` object linked to `aPatient`;

4 add (`aPatient`, `aWard`) to `results`.

5 Return `results`.

Figure 26 Sequence diagram for *List Team's Patients*

SAQ 8

Consider step 3 of the walk-through for this scenario.

- What messages are sent to initiate step 3, and to which objects?
- For each such message, which object is the client and which is the server?

ANSWER.....

- A `getWard()` message is sent to each of `patient3`, `patient4` and `patient5`.
- The client is `team2` and the servers are `patient3`, `patient4` and `patient5` respectively.

Updating the structural model

Now we can identify the responsibilities for holding link references, the consequent navigations and the methods needed for this use case.

Exercise 4

Answer each of the following in the context of the above dynamic model.

- Write down the responsibilities for holding link references needed in the classes `Team` and `Patient`.
- Which associations are navigated and in which directions? For each one, indicate the walk-through step that requires it.
- Write down the protocol for the class `Patient` that follows from the design.

Discussion.....

(a) In the class `Team`:

Links

`Collection<Patient> patients`

References a collection of all the linked `Patient` objects.

In the class `Patient`:

Links

`Ward ward`

References the linked `Ward` object.

(b) The association `caresFor` is navigated from `Team` to `Patient` (step 1);

`hasOn` is navigated from `Patient` to `Ward` (step 3).

(c) In the class `Patient`:

Protocol

`Ward getWard()`

Post-condition: returns the linked `Ward` object.

In the class `HospCoord` we have:

Protocol

`Map<Patient, Ward> getPatientsAndWards(Team aTeam)`

Post-condition: returns a map of pairs of `Patient` and `Ward` objects, such that for each `Patient` object `aPatient` linked to `aTeam`, the map contains the key-value pair (`aPatient`, `aWard`), where `aWard` is linked to `aPatient`.

SAQ 9

In addition to the `HospCoord` method noted above, the design also necessitates a method in the class `Team`.

(a) What is the name of this method?

(b) The method should return a collection of pairs of `Patient` and `Ward` objects. How are these `Patient` and `Ward` objects related to the `Team` object that receives the message invoking this method?

ANSWER.....

(a) `getPatientsAndWards()`

(b) For each `Patient` object `aPatient` linked to the receiver `Team` object, the collection contains the pair (`aPatient`, `aWard`), where `aWard` is linked to `aPatient`.

Since the collection returned by `getPatientsAndWards()` will be passed back to the user interface as the response to the coordinating method, it will be sensible for the return type of `getPatientsAndWards()` to match that of the coordinating method. Accordingly, the following method specification is needed for `Team`.

Protocol

`Map<Patient, Ward> getPatientsAndWards()`

Post-condition: returns a map of pairs of `Patient` and `Ward` objects, such that for each `Patient` object `aPatient` linked to the receiver, the map contains the key-value pair (`aPatient`, `aWard`), where `aWard` is linked to `aPatient`.

The class diagram with the additional navigations included is shown in Figure 27.

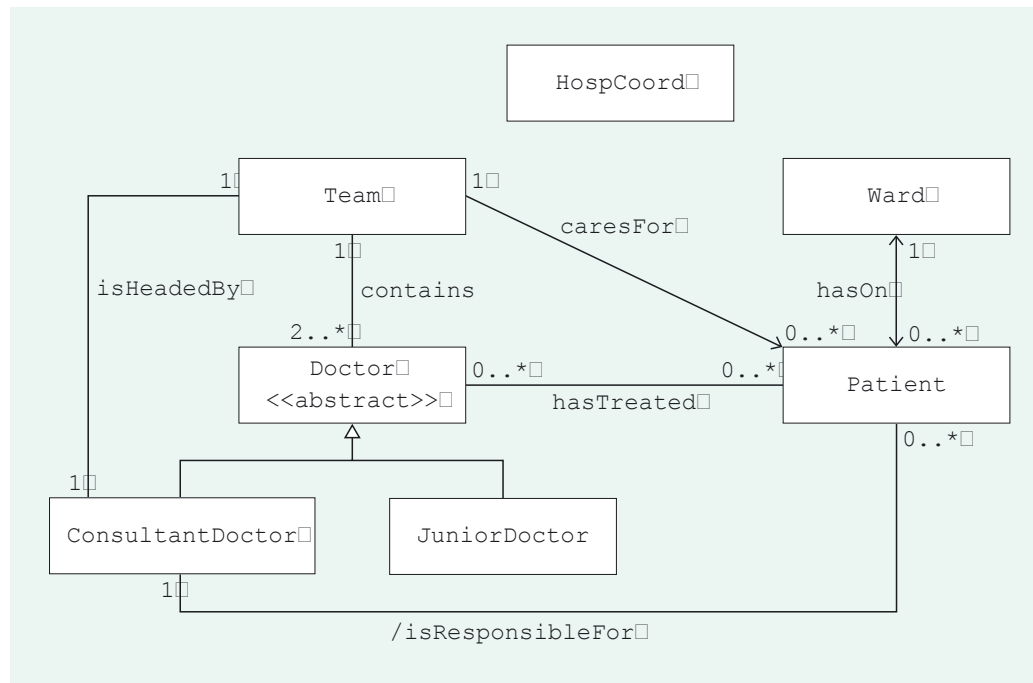


Figure 27 Class diagram showing navigations so far

Notice that Figure 27 shows the association `hasOn` between `Ward` and `Patient` now being two-way.

SAQ 10

- Which use case requires `hasOn` to be navigated from `Ward` to `Patient`?
- Which use case requires `hasOn` to be navigated from `Patient` to `Ward`?

ANSWER.....

- List Ward's Patients
- List Team's Patients

In this subsection you have seen how to form a design which involves navigating two associations. You have also seen that the formation of a collection of objects for return may be a necessary part of a walk-through.

We approached the design in the same way as before.

- Create an object diagram illustrating the example objects of a typical scenario.
- Construct a sequence diagram illustrating the communication between the user interface and the coordinating object.
- Specify the coordinating method.
- Set out a walk-through for the use case – the steps required, for the typical scenario, to achieve the coordinating method specification – and extend the sequence diagram to show how the steps of the walk-through can be achieved by sending messages.
- From the walk-through and sequence diagram:
 - identify method specifications and responsibilities for holding link references, and add them to the relevant class descriptions;
 - add (if not already present) a navigation arrow to the class diagram which corresponds to each class's responsibility for holding a link reference.

This approach is summarised in Section 2 of the *Handbook*.

As before, the design at this stage relates to what happens within the core system. How the patient and ward names can be displayed to the user is dealt with later.

The consequences of the design so far have been incorporated into the structural part of an implementation model in the Appendix.

Envisaging an implementation

Again, it may be helpful for you to visualise a possible implementation of this design. Here are excerpts from Java class definitions which correspond to our design.

```
//Definition of coordinating class
public class HospCoord
{
    //code omitted

    public Map<Patient, Ward> getPatientsAndWards(Team aTeam)
    {
        return aTeam.getPatientsAndWards();
    }

    //code omitted
}

//Definition of Team class
public class Team
{
    //code omitted

    private Collection<Patient> patients; //all the linked Patient objects

    Map<Patient, Ward> getPatientsAndWards()
    {
        Map<Patient, Ward> results = new HashMap<Patient, Ward>();
        for (Patient aPatient : patients)
        {
            results.put(aPatient, aPatient.getWard());
        }
        return results;
    }

    //code omitted
}

//Definition of Patient class
public class Patient
{
    //code omitted

    private Ward ward; //the linked Ward object

    Ward getWard();
    {
        return ward;
    }

    //code omitted
}
```

SAQ 11

Consider the code for the method `getPatientsAndWards()` of `Team`. Which part of the code relates to each of the following walk-through steps?

- (a) Step 1. Access the collection of `Patient` objects linked to `team2`, which in this case is `{patient3, patient4, patient5}`.
- (b) Step 2. Create a new empty collection, `results`.
- (c) Step 3. Access `aWard`, the `Ward` object linked to `aPatient`.

ANSWER.....

- (a) The part of the code that uses the `Team` object's `patients` instance variable, i.e. the use of `patients` within the loop heading:

```
for (Patient aPatient : patients)
```

- (b) The following code:

```
Map<Patient, Ward> results = new HashMap<Patient, Ward>();
```

- (c) The part of the code that sends a message to a `Patient` object to get its `Ward` object, i.e.:

```
aPatient.getWard();
```

Note the following.

- The ordering of walk-through steps need not always be maintained within the code. Here, for example, step 2 (creating a new collection) is done before step 1. For some steps, there is an obvious requirement for an ordering to be maintained. For example, it would not make sense to 'access `aWard`, the `Ward` object linked to `aPatient`' (step 3) before `aPatient` has been accessed (step 1). For other steps, the details of the implementation might require minor variations in ordering.
- There is an important difference in the ways `Patient` and `Ward` objects are accessed from within `Team`'s `getPatientsAndWards()` method. For a `Team` object to access its `Patient` objects requires simply that it use its `patients` instance variable. For a `Team` object to access the `Ward` object linked to one of its `Patient` objects requires the `Team` object to send a message, `getWard()`, to that `Patient` object. Because the `Ward` object is not directly linked to the `Team` object, the `Team` object sends a message to the intermediate `Patient` object to access the `Ward` object.

3.3 Splitting the coordination

In our final example of a use case that returns results to the user interface, you will meet an approach to simplifying the design for a use case. You will also practise creating simple designs.

Here is the description of the *List Patient's Treatment* use case (labelled G in the requirements document).

The administrator identifies the patient. The system displays: the name of the consultant responsible for the patient; the team code of the team that cares for the patient; the name of each doctor who has treated the patient and, for each such junior doctor, their grade (1, 2 or 3).

Three things have to be returned to the user interface:

- 1 a `ConsultantDoctor` object;
- 2 a `Team` object;
- 3 a collection of `Doctor` objects.

Recall that, since `Doctor` is an abstract class, by a `Doctor` object we mean any instance of any concrete subclass of `Doctor`.

There are several ways this could be achieved. For example, we could create an object with these three objects as its attributes, and return this as the answer to the user interface. However, we will take the opportunity to illustrate another approach that is useful in dealing with complex use cases where, as here, what is required can be readily partitioned. Instead of one coordinating method, the behaviour will be split amongst three separate coordinating methods.

To begin, the parts of the class diagram relevant to this use case are shown below.

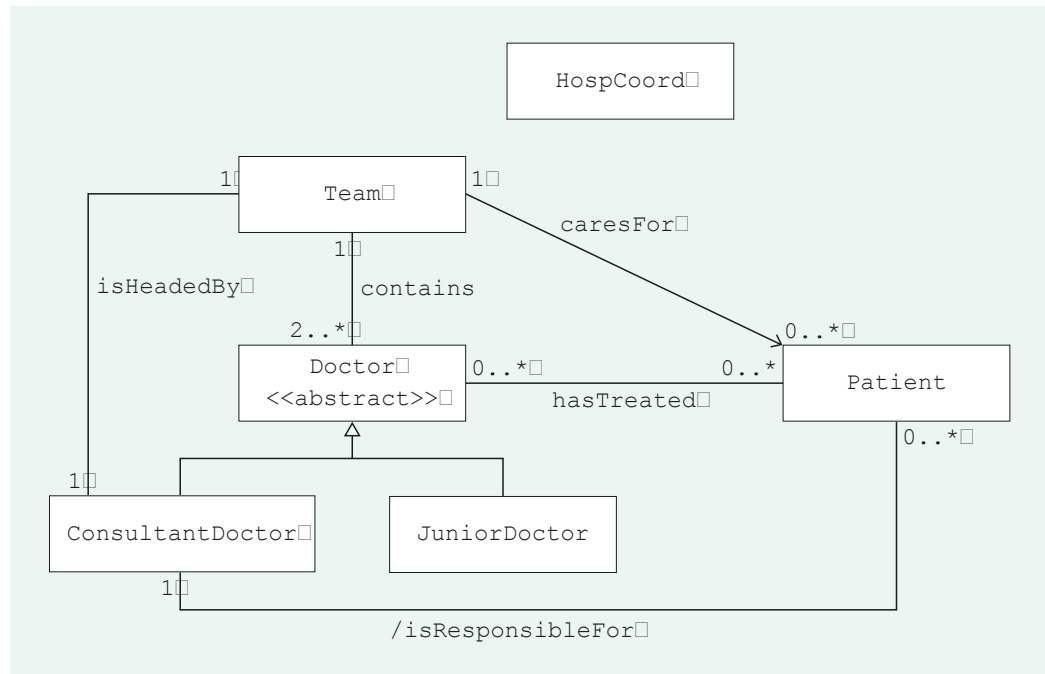


Figure 28 The classes and associations relevant to *List Patient's Treatment*

Exercise 5

Consider a scenario involving a patient represented by `patient2`. Here are some facts about this scenario.

- 1 The patient is cared for by the team represented by `team3`, whose head is the consultant that is represented by `consultantDoctor3`, and who is responsible for the patient.
- 2 The team contains three doctors, represented by `consultantDoctor3`, `juniorDoctor5` and `juniorDoctor7`. Of these doctors, `consultantDoctor3` and `juniorDoctor7` have treated the patient.

Sketch an object diagram showing the objects and links involved in this scenario.

Discussion.....

Our object diagram is shown in Figure 29.

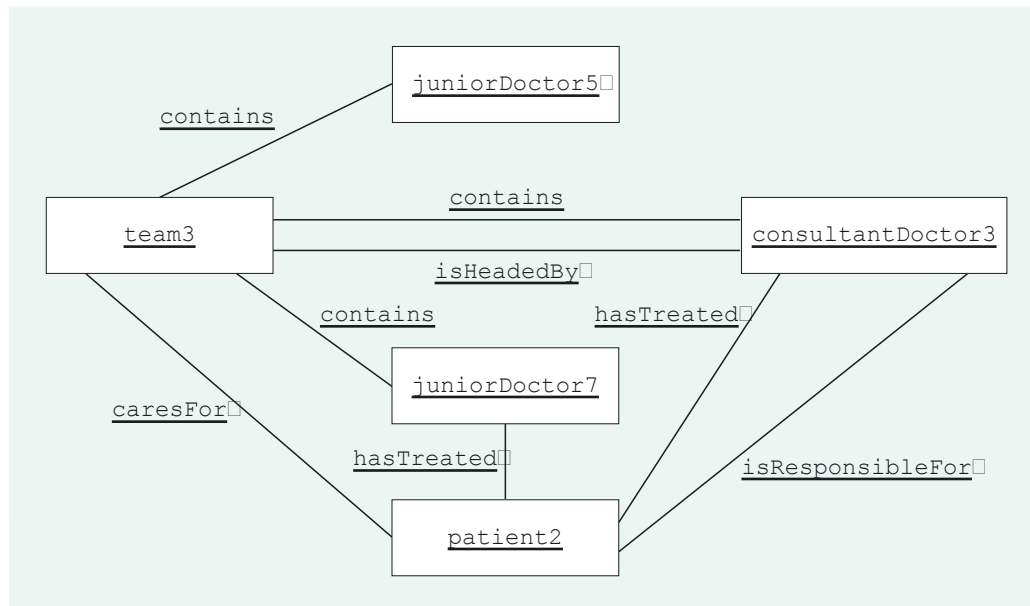


Figure 29 Object diagram for the *List Patient's Treatment* scenario

We will form a design based on the scenario described in Exercise 5.

When the user initiates the *List Patient's Treatment* use case, the user interface will send three coordinating messages to the core system, and from the objects returned the user interface will retrieve the data to be displayed. The interaction is shown in Figure 30.

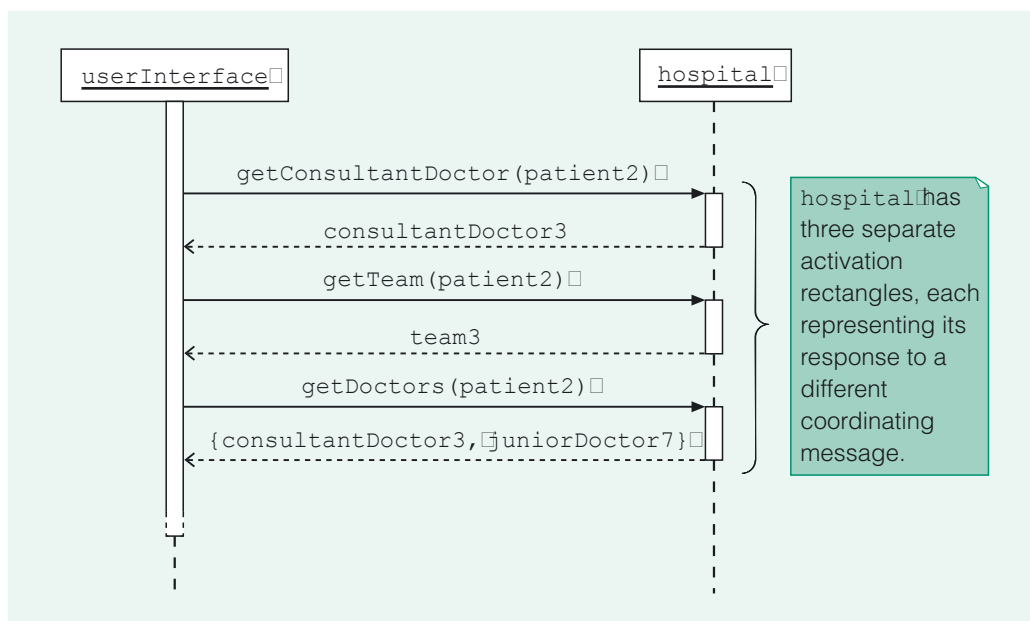


Figure 30 Communication with the user interface for the *List Patient's Treatment* scenario

The coordinating methods

From the object diagram in Figure 29 and the sequence diagram in Figure 30, the specifications of the coordinating methods can be derived, and a corresponding walk-through and complete sequence diagram created for each. They are each straightforward, being similar to the *List Ward's Patients* use case considered in Subsection 3.1.

Getting the consultant

The specification for `getConsultantDoctor(aPatient)` is:

`ConsultantDoctor getConsultantDoctor(Patient aPatient)`

Post-condition: returns the `ConsultantDoctor` object linked to `aPatient`.

Here is a walk-through and sequence diagram (Figure 31) for this method in the context of the scenario.

Given: the `Patient` object `patient2`

- 1 Access the `ConsultantDoctor` object linked to `patient2`, which in this case is `consultantDoctor3`.
- 2 Return `consultantDoctor3`.

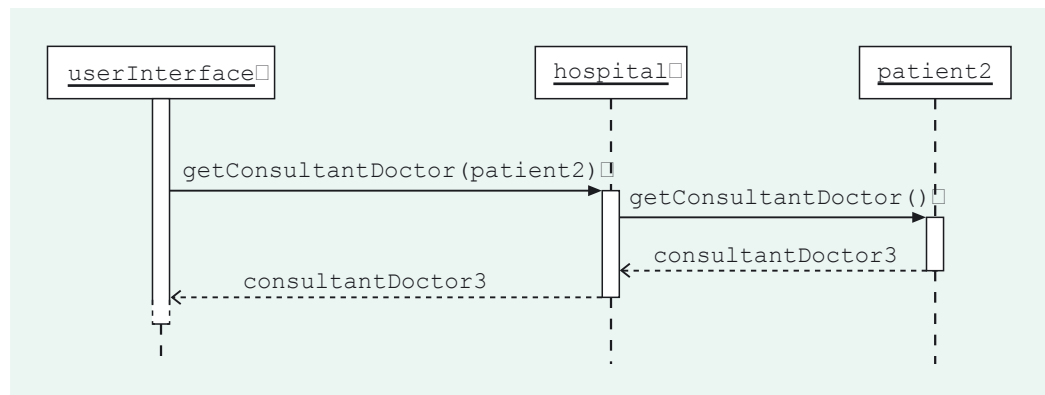


Figure 31 Sequence diagram for `getConsultantDoctor(aPatient)`

From the walk-through and sequence diagram, we can derive the following information about the responsibilities for holding link references, method specifications and navigation.

Class `Patient`

Links

`ConsultantDoctor consultantDoctor`

References the linked `ConsultantDoctor` object.

Protocol

`ConsultantDoctor getConsultantDoctor()`

Post-condition: returns the linked `ConsultantDoctor` object.

Class `HospCoord`

Protocol

`ConsultantDoctor getConsultantDoctor(Patient aPatient)`

Post-condition: returns the `ConsultantDoctor` object linked to `aPatient`.

The association `isResponsibleFor` is navigated from `Patient` to `ConsultantDoctor`.

Getting the team

Exercise 6

- (a) Write a specification for the method `getTeam(aPatient)`.
- (b) Construct a walk-through and a sequence diagram for the method `getTeam(aPatient)`, with `patient2` as the argument and the system in the state shown in Figure 29.
- (c) What method specifications, responsibilities for holding link references, and navigations are needed for this dynamic model?

Discussion.....

- (a) Our specification is:

```
Team getTeam(Patient aPatient)
```

Post-condition: returns the Team object linked to aPatient.

- (b) The walk-through is as follows.

Given: the Patient object `patient2`

- 1 Access the Team object linked to `patient2`, which in this case is `team3`.
- 2 Return `team3`.

The sequence diagram is shown in Figure 32.

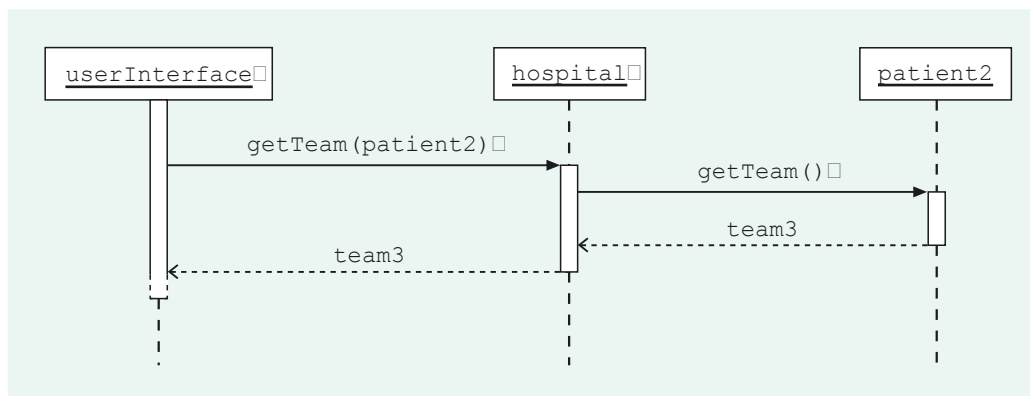


Figure 32 Sequence diagram for `getTeam(aPatient)`

- (c) The method specifications, responsibilities for holding references, and navigations are as follows.

Class Patient

Links

Team team

References the linked Team object.

Protocol

Team `getTeam()`

Post-condition: returns the linked Team object.

Class HospCoord

Protocol

Team `getTeam(Patient aPatient)`

Post-condition: returns the Team object linked to aPatient.

The association `caresFor` is navigated from Patient to Team.

Getting the doctors

Since this is almost identical to *List Ward's Patients*, we give the walk-through and sequence diagram (in Figure 33), and the consequent responsibilities for holding link references, method specifications and navigation, without comment.

Given: the Patient object `patient2`.

- 1 Access the collection of Doctor objects linked to `patient2`, which in this case is `{consultantDoctor3, juniorDoctor7}`.
- 2 Return `{consultantDoctor3, juniorDoctor7}`.

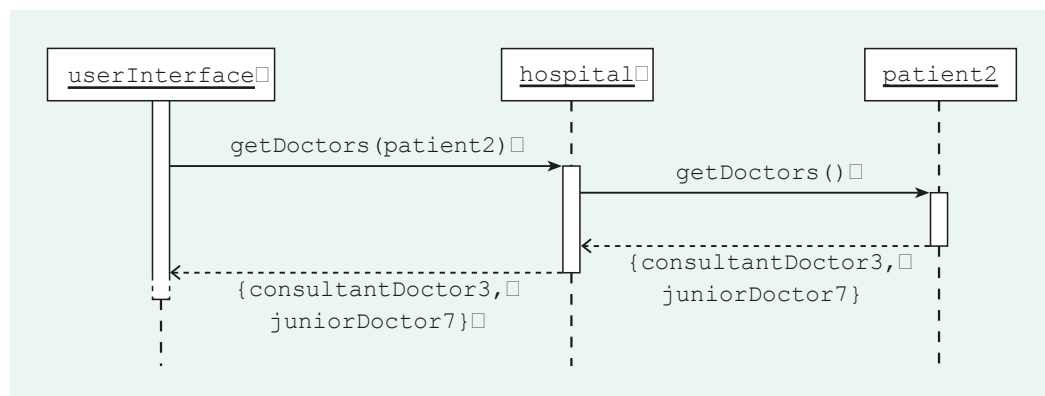


Figure 33 Sequence diagram for `getDoctors(aPatient)`

The required class responsibilities are as follows:

Class Patient

Links

`Collection<Doctor> doctors`

References a collection of all the linked Doctor objects.

Protocol

`Collection<Doctor> getDoctors()`

Post-condition: returns a collection of all the linked Doctor objects.

Class HospCoord

Protocol

`Collection<Doctor> getDoctors(Patient aPatient)`

Post-condition: returns a collection of all the Doctor objects linked to `aPatient`.

The association `hasTreated` is navigated in the direction from Patient to Doctor.

In this subsection, you have seen how the behaviour required of the core system for a use case can be split into separate coordinating methods, requiring the user interface to send more than one coordinating message. This is a way to simplify the design for a complex use case, where the actions required are independent of each other. You have also practised creating simple designs.

Note that, as with all the other designs developed in this unit, the consequences of those of this subsection have been incorporated into the structural part of the implementation model in the Appendix.

In Section 3 as a whole, we have considered creating designs for use cases that involve returning results to the user interface. In Section 5 we turn to what is involved in creating designs for use cases that require a change to the state of the core system. However, before doing that, Section 4 will digress slightly to consider the topic of anthropomorphism.

4

Anthropomorphism

This section forms a short interlude in the midst of design considerations, to reflect on the kinds of language appropriate in software development.

Throughout the course so far we have been talking *anthropomorphically*, that is, attributing human characteristics to what is not human: in our case, software entities. Here are two examples taken from earlier in this unit.

- ▶ How does `hospital` know which `Patient` object to send the `recordTreatmentBy()` message to?
- ▶ `team2` will need to send a message to each of the `Patient` objects accessed in step 1, *asking* it for its `Ward` object.

Software developers commonly talk in this way about software objects 'knowing' things, and of them 'asking' each other things. Indeed, the very notion of a software object as something that has responsibilities is suggestive of **anthropomorphism**. A software object is being treated anthropomorphically, as though it is a person. Is this a sensible thing to do?

Exercise 7

Suggest one advantage, and one possible disadvantage, of thinking about software objects as if they were people.

Discussion.....

An advantage is that the similarities with humans and their interactions can help in forming mental models of objects and their interactions.

A possible disadvantage is that the metaphor might be extended too far, and the assumption made that something is necessarily true of a software object because it is true of the real-world object it represents. For example, you might assume that a `Patient` object should have a reference to a `Ward` object, simply because a patient knows the ward they are on.

You may have thought of other advantages and disadvantages.

It will be helpful to reflect on how ideas about responsibilities and collaborations may be expressed.

Here is an extract from the introductory chapter in *Designing Object-Oriented Software* (Wirfs-Brock et al., 1990).

It might already have struck you that object-oriented design can take a significantly anthropomorphic point of view. And it is true that this field has led apparently sane and sensible engineers to discuss entities in their programs as 'he' and 'she'. We realise that anthropomorphism has traditionally been frowned on as fuzzy thinking, a substitute for rigorous analysis, or simply a sign of lack of self-discipline. In this book, however, we do not consider anthropomorphism to be a sin. In the first place, we are not going to insult your intelligence by imagining that you cannot tell the difference between a real person and objects in an application. And in the second place, when thinking about a software

design in object-oriented terms, anthropomorphism can be an aid to conceptualisation.

Object-oriented design encourages a view of the world as a system of cooperating and collaborating agents. Work is accomplished in an object-oriented system by one object sending a request to another to perform one of its operations, reveal some information, or both. This first request then starts a long complex chain of such requests.

Objects may be modelled on inanimate or even conceptual entities in the real world, but within their systems they act as agents, just as we do within ours. It may sometimes paradoxically seem as if objects know more than their real-world counterparts. After all, in the real world telephones do not dial each other, nor colours paint themselves, without human agency. Whereas the systems of daily life require human agents to make things happen, objects are the agents within their own systems.

This positive attitude to anthropomorphism is largely endorsed nowadays by developers. Two consequences are worth particular emphasis.

First, the extract makes a distinction between the real world and the *model* of that world that can be built using objects. We have made a similar distinction with the notion of a real-world *system domain* being represented in software by objects in the *core system*. Statements about the system domain are different from statements about the software. This difference should be recognised to avoid the ‘fuzzy thinking’ alluded to by Wirfs-Brock et al., where statements are made about the system domain and uncritically treated as statements about the software. In fact, we often emphasise this distinction in the text. For example, here is an extract from this unit:

For each patient cared for by the team, the coordinating method needs to return both the `Patient` object and the `Ward` object corresponding to the ward the patient is on.

What is relevant here is how this extract highlights the distinction between terms referring to the system domain and terms referring to the software. A term that refers to a software entity is signalled by the use of a different font, and we have carefully avoided using this font for a term that refers to a system domain entity. So, in the above extract, we distinguish the software entity ‘`Ward` object’ from the system domain entity ‘ward’.

The software font is used as an aid in the text. Even without the use of a special font for software entities, if the terms are used correctly, the distinction between real-world and software terms should be clear.

SAQ 12

What other system domain entities does the above extract from this unit refer to? Similarly, identify any other software entities.

ANSWER.....
 From the system domain we have ‘patient’ and ‘team’. The only other software entity is ‘`Patient` object’.

Second, the Wirfs-Brock et al. extract makes clear that behaviour in the system domain does not automatically carry over to behaviour in the software. For example, just because a real-world doctor knows what team they are in does not mean that a `Doctor` object has to ‘know’ (that is, have a reference to) the corresponding `Team` object.

Anthropomorphism has its critics

Edsger Dijkstra, one of the most influential early computer scientists, was critical of anthropomorphism.

Is anthropomorphic thinking bad? Well, it is certainly no good in the sense that it does not help. Why did the stone fall in Greek antiquity? Quite simply because it wanted to go to the centre of the earth. And, several centuries later, we had the burning question: why do stones want to go to the centre of the earth? Well, that is simple too: because that's where they belong. Why are heavier stones heavier than lighter stones? Because they are more eager to be at the centre of the earth. But then Galileo made the troubling discovery that the heavier stone does not fall any faster than the lighter one. How come? Simple, dear Watson: the heavier stone has indeed a greater desire to be at the centre of the earth, but it is also more lazy. So much for a – be it somewhat simplified – history of the development of physics. I trust you got the message.

So anthropomorphic thinking is no good in the sense that it does not help. But is it also bad? Yes, it is, because even if we can point to some analogy between Man and Thing, the analogy is always negligible in comparison to the differences, and as soon as we allow ourselves to be seduced by the analogy to describe the Thing in anthropomorphic terminology, we immediately lose our control over which human connotations we drag into the picture. And as most of those are totally inadequate, the anthropomorphism becomes more misleading than helpful.

Dijkstra (1985)

In M256 we will take the view that as long as anthropomorphism is applied carefully, it is a very useful way of discussing software objects.

5 Changing state

In Section 3 we considered designs for some use cases from the Hospital System that return results to the user interface. In this section, we investigate two use cases from the same system, which involve changing the state of the core system. We take the same approach to creating a design as before, but now it will involve consideration of how state changes can be achieved.

5.1 Creating a one-way link

From this first example, you will discover how the creation of a one-way link can be designed, and how a design can require a method to have a pre-condition.

Here is the description of the *Treat Patient* use case (labelled B in the requirements document).

The administrator identifies the patient and the doctor; the doctor must be in the team that cares for the patient. The system records that the doctor has treated the patient.

We considered this use case in Subsection 2.1, and formed a model of the core system's communication with the user interface for a typical scenario, which involved recording the fact that the *Patient* object *patient5*, cared for by the *Team* object *team2*, has been treated by the *Doctor* object *doctor3*, a member of *team2*.

In-depth consideration of this scenario will help in the creation of a general design for achieving the behaviour required of the core system. Here is the part of the class diagram relevant to the scenario.

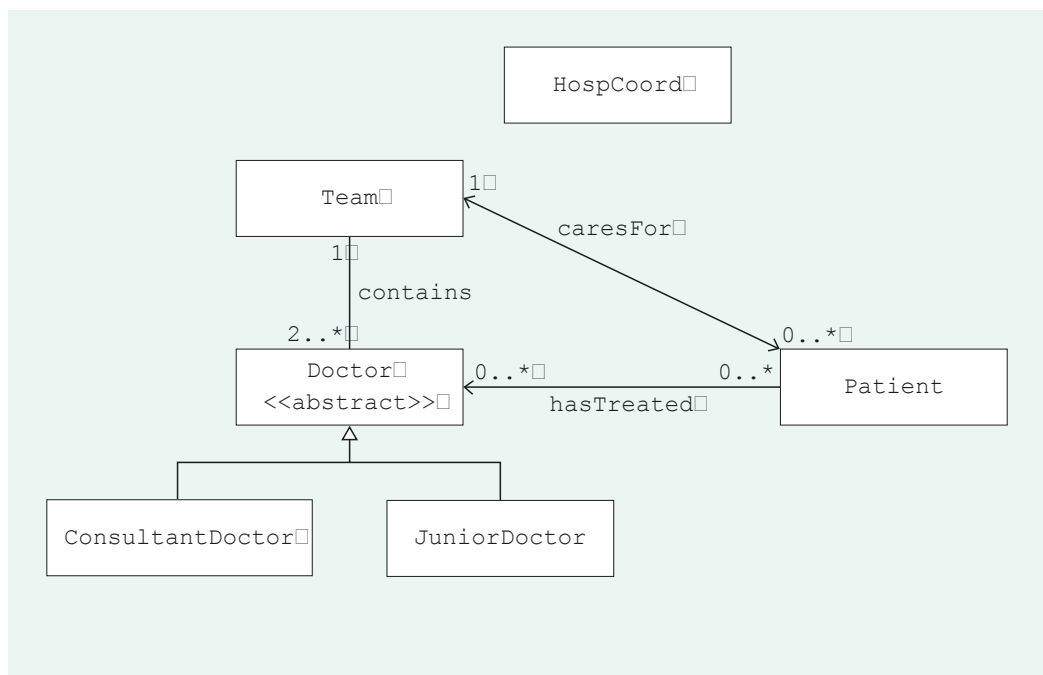


Figure 34 The classes and associations relevant to *Treat Patient*

For a use case that changes the system's state, two object diagrams are needed, one illustrating the objects and links relevant to the scenario before the use case has been initiated, and the other illustrating them after the use case has completed. These are shown in Figure 35 for the above scenario. Any objects and links of the core system that are not shown in the object diagrams are assumed to be unchanged by the use case.

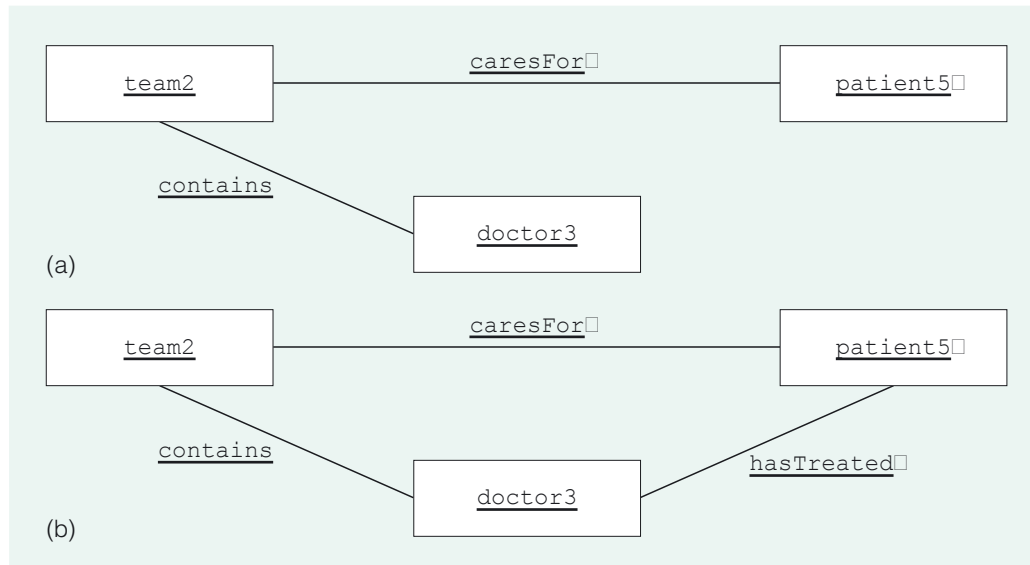


Figure 35 (a) Object diagram for the scenario before *Treat Patient*: patient5 and doctor3 are not linked. (b) Object diagram for the scenario after *Treat Patient*: a hasTreated link exists between patient5 and doctor3

Note that in Figure 35:

- ▶ patient5 and doctor3 are linked to the same Team object, team2 (reflecting a condition expressed in the use case, which we discuss later);
- ▶ before the use case (part (a)), patient5 and doctor3 are *not* linked to each other via hasTreated;
- ▶ after the use case (part (b)), patient5 and doctor3 are linked to each other via hasTreated.

Specifying the coordinating method

In Section 2, we created the sequence diagram shown in Figure 36 below to illustrate the communication between user interface and coordinating object for this particular scenario of the *Treat Patient* use case.

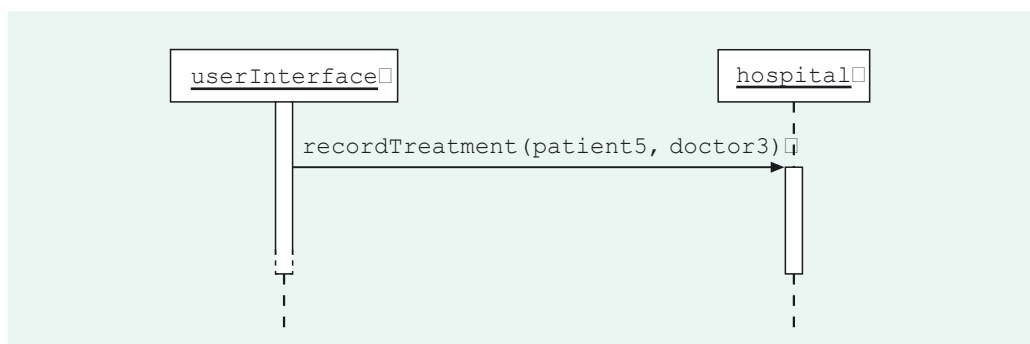


Figure 36 Communication with the user interface for the *Treat Patient* scenario

In *Unit 7* we will consider what should happen if `aPatient` and `aDoctor` are already linked.

Recall that an invariant reflects a constraint on the real world in which the system is to operate. It is a condition on the state of the system that must always be true.

This is invariant 7 in Section 4 of the *Handbook*.

In *Unit 7* we will consider what should happen if the pre-condition is not met.

On receipt of the coordinating message `recordTreatment(patient5, doctor3)`, `hospital`'s coordinating method is invoked. The method must initiate actions within the core system which change the state of the system to reflect the treatment of the patient by the doctor. For this use case there is no requirement for the user interface to display any resulting information to the user, so no answer is shown to the coordinating message.

The aim of the coordinating method `recordTreatment(aPatient, aDoctor)` is straightforward. It should result in a `hasTreated` link between `aPatient` and `aDoctor`, as shown in Figure 35(b) above.

For this use case, there is however a condition attached: there is a *pre-condition* on the method, which, as you saw in *Unit 5*, is a condition that must be true before a method can be validly invoked.

Why is a pre-condition involved here? To understand this, it is important to realise that every coordinating method must ensure that when it completes, it leaves the core system in a valid state; in particular, no invariants may be violated. That is, an implicit part of every post-condition is that all invariants remain true.

This is trivial for the use cases we have investigated previously, but for use cases that involve a change of state, we must ensure that the change of state does not violate any invariant.

Refer to the Hospital System initial structural model, in Section 4 of the *Handbook*.

SAQ 13

Which invariant of the Hospital System might be affected by the coordinating method `recordTreatment(aPatient, aDoctor)`?

ANSWER.....

The only change that this method can make is to create a new link between a `Patient` object and a `Doctor` object. The relevant invariant is the following.

If `aPatient` is any `Patient` object then any `Doctor` object linked to `aPatient` is also linked to the `Team` object which is linked to `aPatient`.

So that the invariant above remains true, the coordinating method should only be invoked if the `Doctor` object and the `Patient` object supplied as arguments are linked to the same `Team` object. This is the pre-condition, and the coordinating method for *Treat Patient* is specified as follows.

```
void recordTreatment(Patient aPatient, Doctor aDoctor)
    Pre-condition: aDoctor and aPatient must be linked to the same Team object.
    Post-condition: aDoctor is linked to aPatient.
```

Next, we must form a design for achieving the coordinating method specification. Assuming that the pre-condition is met, what is required for the post-condition to be satisfied? For our particular scenario, in which the pre-condition is met, the method must create a link between the `Patient` and `Doctor` objects.

How are links created?

Creating a link of an association involves recording the necessary link references. So, when deciding how a link of an association should be created, it is necessary to consider which classes have responsibilities for holding link references. For a one-way association, navigated in the direction from a class `P` to a class `Q`, each `P` object needs to hold references to the `Q` objects to which it is linked.

In creating a one-way link, then, the object at the *from* end (let us call this object $p1$) must record a reference to the object at the *to* end (let us call it $q1$). At this design stage, this recording of a reference simply needs to be noted; there is no need to consider the referencing in greater detail. However, it will be helpful for you to think in advance about what this reference creation might involve. (We will return to this issue at implementation.)

Figure 37 illustrates the situation where each P object may be linked to at most one Q object; that is, the multiplicity of the association between P and Q at the Q end is no more than 1. Before the link creation, $p1$ has a null reference. After the link creation, $p1$ has a reference to $q1$.

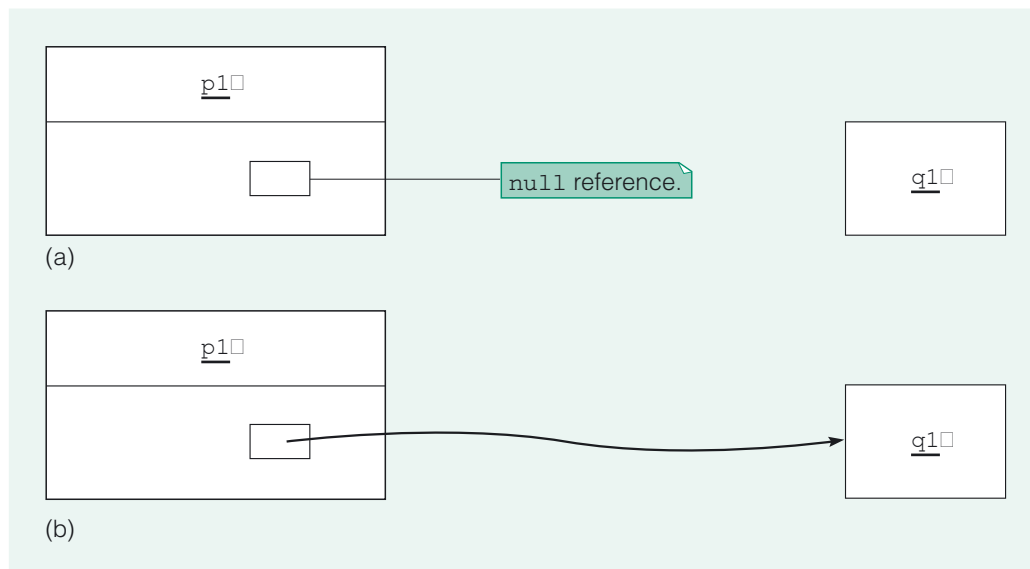


Figure 37 Creating a link: multiplicity no more than 1

The other situation to consider is where a P object can be linked to several Q objects; that is, the multiplicity of the association between P and Q at the Q end is greater than 1. What happens when a link is created in this situation?

Exercise 8

In Section 2 we observed that a collection can be used in implementing link references in the case of a multiplicity greater than 1. Suppose that $p1$ is already linked to some Q objects, $q2$ and $q4$ (using the collection, `collection1`), and that a new link is to be created between $p1$ and the Q object $q1$. Figure 38 illustrates the situation before the link creation.

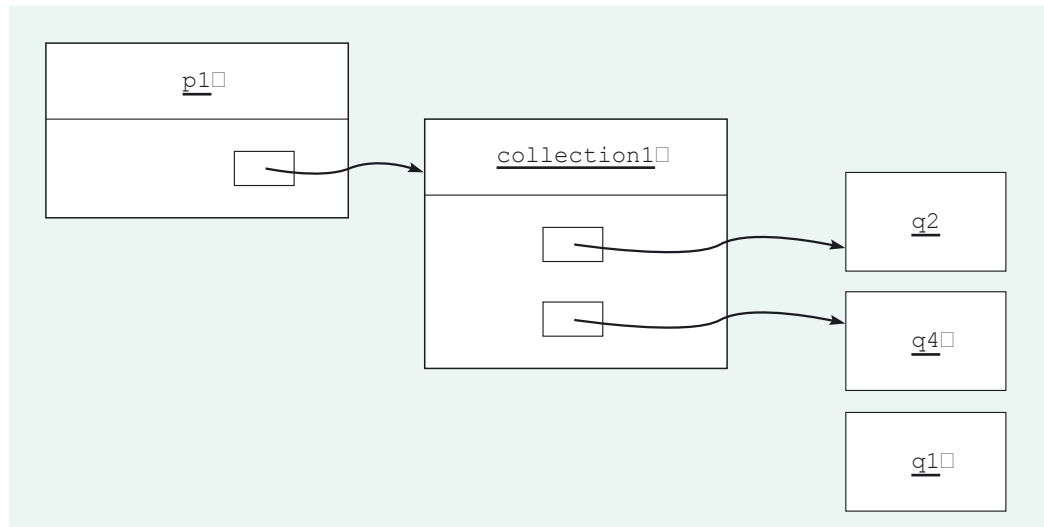


Figure 38 Creating a link when the multiplicity is greater than 1: before the new link is created
 Sketch a similar diagram illustrating the situation after the link between `p1` and `q1` has been created.

Discussion.....
 Our diagram is shown in Figure 39.

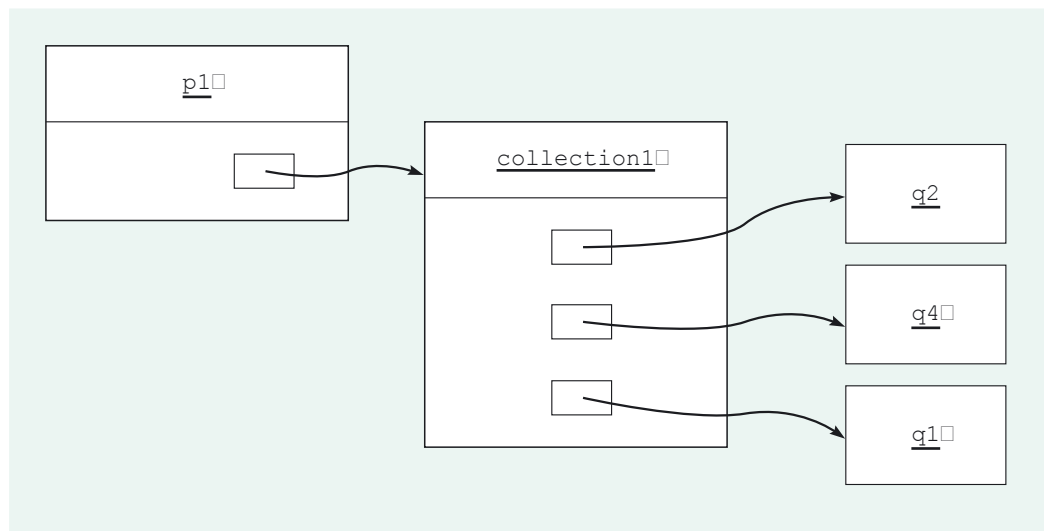


Figure 39 Creating a link when the multiplicity is greater than 1: after the new link is created

In summary: in creating a one-way association, the object at the *from* end must record a reference to the object at the *to* end. The precise details depend on the multiplicity at the *to* end, and will be returned to later in the development. At this design stage, the recording of a reference simply needs to be included as a walk-through step, and we must determine an appropriate sequence of messages to achieve the step.

The class at the *from* end of the association is the obvious one to make responsible for creating the link since it is the one in which the link reference must be recorded. But note that a link cannot be created without access to the objects at both ends. To create a one-way link from a *P* object `p1` to a *Q* object `q1`, both `p1` and `q1` must be accessed – `p1` so that it can be sent a message, and `q1` so that the message can tell `p1` to record a reference to `q1`. Figure 40 contains a sequence diagram illustrating what happens. The object `p1` is sent a message, telling it to record a reference to `q1`, and `p1` does this. Creating links in this way implements one-way navigation of the association from *P* to *Q*.

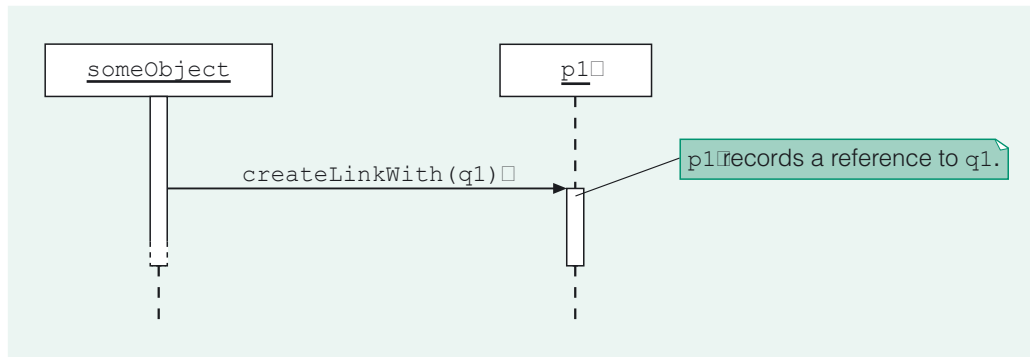


Figure 40 p1 is responsible for creating a link of a one-way association

You will later see how a link of a two-way association might be created.

SAQ 14

- (a) From the designs in Section 3, in what direction(s) does the association `hasTreated` need to be navigated? (Look at Figure 34.)
- (b) Which class should be made responsible for creating `hasTreated` links?

ANSWER.....

- (a) The association `hasTreated` is navigated only in the direction from `Patient` to `Doctor`.
- (b) The class `Patient` should be made responsible for creating `hasTreated` links.

Remember that although allocating responsibility to `Patient` is sensible on the basis of the designs at this stage, it is not necessarily a definitive answer to the question of responsibility for creating `hasTreated` links. There remain other use cases to consider, and it may later be found that `hasTreated` needs to be navigated in both directions. What makes design interesting, and challenging, is that decisions made for one part of the system can affect the design of other parts of the system: use cases are generally not independent.

Creating a walk-through and a sequence diagram

It is straightforward to produce a walk-through and sequence diagram for the scenario in this case. There is only one step.

Given: the `Patient` object `patient5`, and the `Doctor` object `doctor3`.

- 1 `patient5` records a reference to `doctor3`.

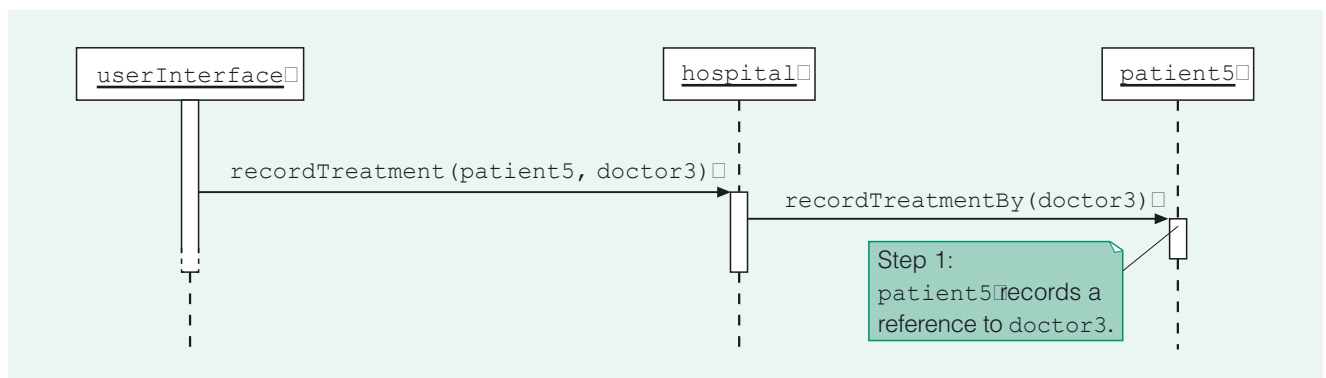


Figure 41 Sequence diagram for *Treat Patient*

This is essentially the same sequence diagram as in Figure 9(a) in Section 2 – however, we now have some justification for making `patient5` rather than `doctor3` responsible for recording the treatment.

Note that no navigation is required for this design. There is no need to do anything to find any of the objects involved since both `patient5` and `doctor3` are provided as arguments by the user interface. Also notice that although the link that is created is between `patient5` and `doctor3`, `doctor3` does not have to do anything. After the link has been created, `patient5` will have a reference to `doctor3`, but `doctor3` will *not* have a reference to `patient5`.

SAQ 15

Why does `doctor3` not need (at this stage) to have a reference to `patient5`?

ANSWER.....

Because, from the designs up to this point, there is no requirement for `Doctor` objects to be able to access the `Patient` objects to which they are linked via `hasTreated`. The association `hasTreated` is not navigated from `Doctor` to `Patient`.

Updating the structural model

Although there is no new navigation required for this use case, and hence there are no additional responsibilities for the classes to hold link references, methods corresponding to the messages shown in Figure 41 need to be specified.

Exercise 9

Specify the additional methods required for the above design.

Discussion.....

In the class `Patient`:

Class `Patient`

Protocol

`void recordTreatmentBy(Doctor aDoctor)`

Post-condition: a reference to `aDoctor` is recorded.

Note that we have given this method no pre-condition. In theory, this means it could be invoked whatever the `Doctor` object and the `Patient` object involved; in practice, in the Hospital System, it will only ever be invoked in the context of *Treat Patient*, in which the coordinating method's pre-condition should ensure that the `Doctor` object and the `Patient` object are appropriate. However, you will see that later in the design process we have reason to return to and amend this particular method.

In the class `HospCoord`:

Class `HospCoord`:

Protocol

`void recordTreatment(Patient aPatient, Doctor aDoctor)`

Pre-condition: `aDoctor` and `aPatient` must be linked to the same `Team` object.

Post-condition: `aDoctor` is linked to `aPatient`.

Levels of detail

The discussion in Exercise 9 shows different, but appropriate, levels of detail in the method specifications for the classes `Patient` and `HospCoord`, highlighting an important general principle, which is evident here for the first time.

A coordinating method specification sets out what has to be done in the core system. But it does not state *how* this is achieved, since a client (a user interface, for example) interacting with the core system by invoking the coordinating method should not know the details of how the core system is implemented.

The methods that are *subsequently* invoked play a part in *how* the coordinating method specification is achieved, and will *not* be accessible to client code. The specification of such a method sets out in detail what the method does, for the benefit of those working on the core system.

In Exercise 9, contrast the coordinating method's post-condition 'aDoctor is linked to aPatient' (not detailing what is involved in the creation of a link between aDoctor and aPatient) with the post-condition 'A reference to aDoctor is recorded' (revealing what is involved in link creation) for the `Patient` method that helps the coordinating method achieve its goal.

In this subsection, you have seen what is involved in creating a design for a coordinating method that changes the state of the core system, in this case by creating a one-way link. The process followed was the same as before, but it involved determining responsibility for the link creation. Additionally, you learnt that design may involve specifying a pre-condition for a method.

5.2 Creating an object and a two-way link

To conclude our introduction to design, we turn to the *Admit Patient* use case (labelled A in the Hospital System requirements document). This is a more intricate use case than you will be expected to tackle alone, and involves a change of core system state *and* information being returned to the user interface. The example is sufficiently complicated to demonstrate that attempting to write the code without first thinking through all the steps carefully would be a very poor idea, yet once a design has been developed, coding will be comparatively straightforward. As we discuss what is required, try to imagine what it would be like to create code from scratch, without a design – we think it would be very difficult to get right. Consequently, this use case should help you appreciate the advantages of design.

Studying this use case should bring a number of particular benefits.

- 1 You will see the value, for a complex use case, of producing an outline walk-through and sequence diagram, and then refining them.
- 2 You will learn what is involved in creating a new object.
- 3 You will learn what is involved in creating a two-way link.
- 4 You will see that a change of core system state may involve changing the values of attributes.
- 5 You will learn that it can be necessary during design to add an association to a class diagram.

For this use case in particular, there are many possible designs. As before, here we simply present one way of achieving what is required.

Here is the description of the *Admit Patient* use case.

The administrator provides the system with the patient's name, sex and date of birth and identifies the team that cares for the patient.

The system identifies a ward of the appropriate type with empty beds; if there is a choice of such wards one of those with the largest number of empty beds is chosen.

If there is no appropriate ward with empty beds the administrator is informed of this fact.

If there is an appropriate ward with empty beds the system does the following two things.

- 1 It records:
 - ▶ the patient's name, sex and date of birth;
 - ▶ that the patient is under the care of the given team;
 - ▶ that the consultant that heads the team is responsible for the patient;
 - ▶ that the patient is on the ward;
- 2 It informs the administrator of the ward to which the patient has been admitted.

Figure 42 shows the part of the class diagram that is relevant to this use case.

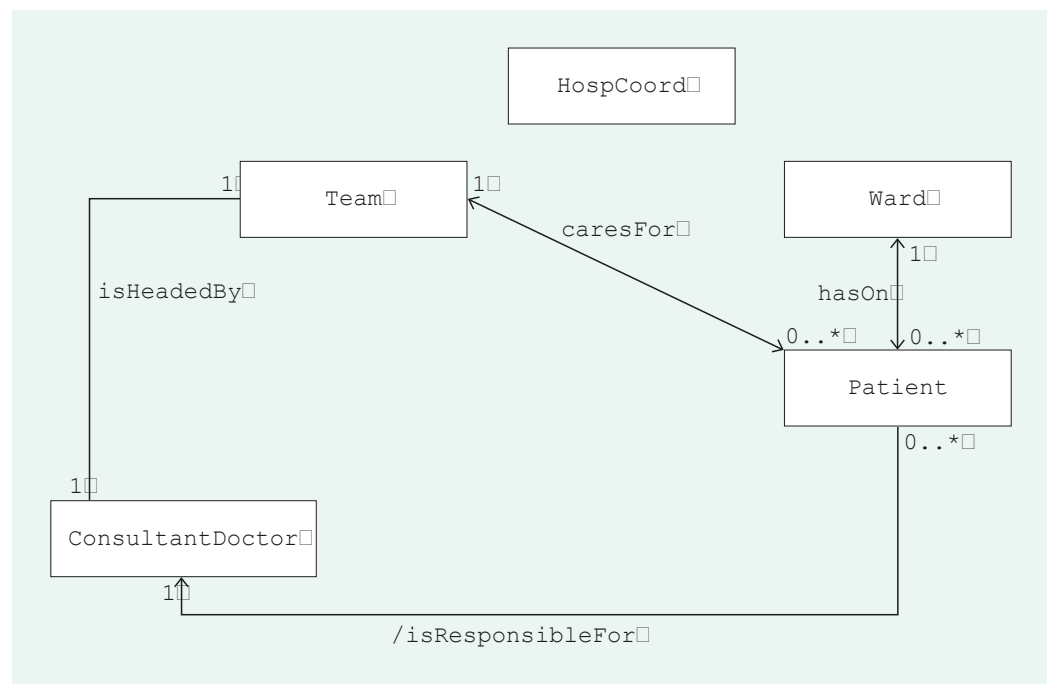
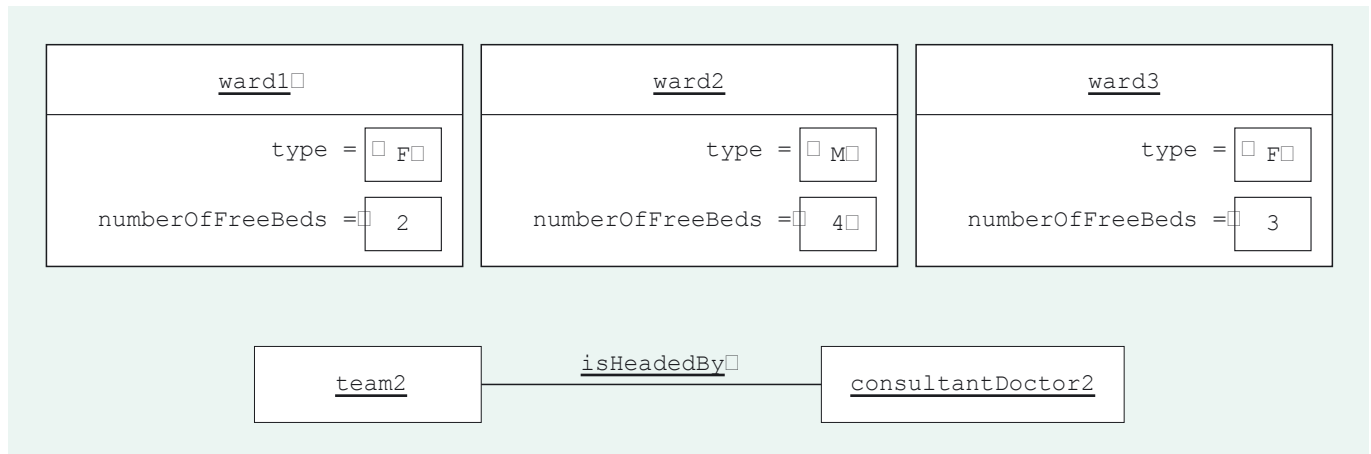
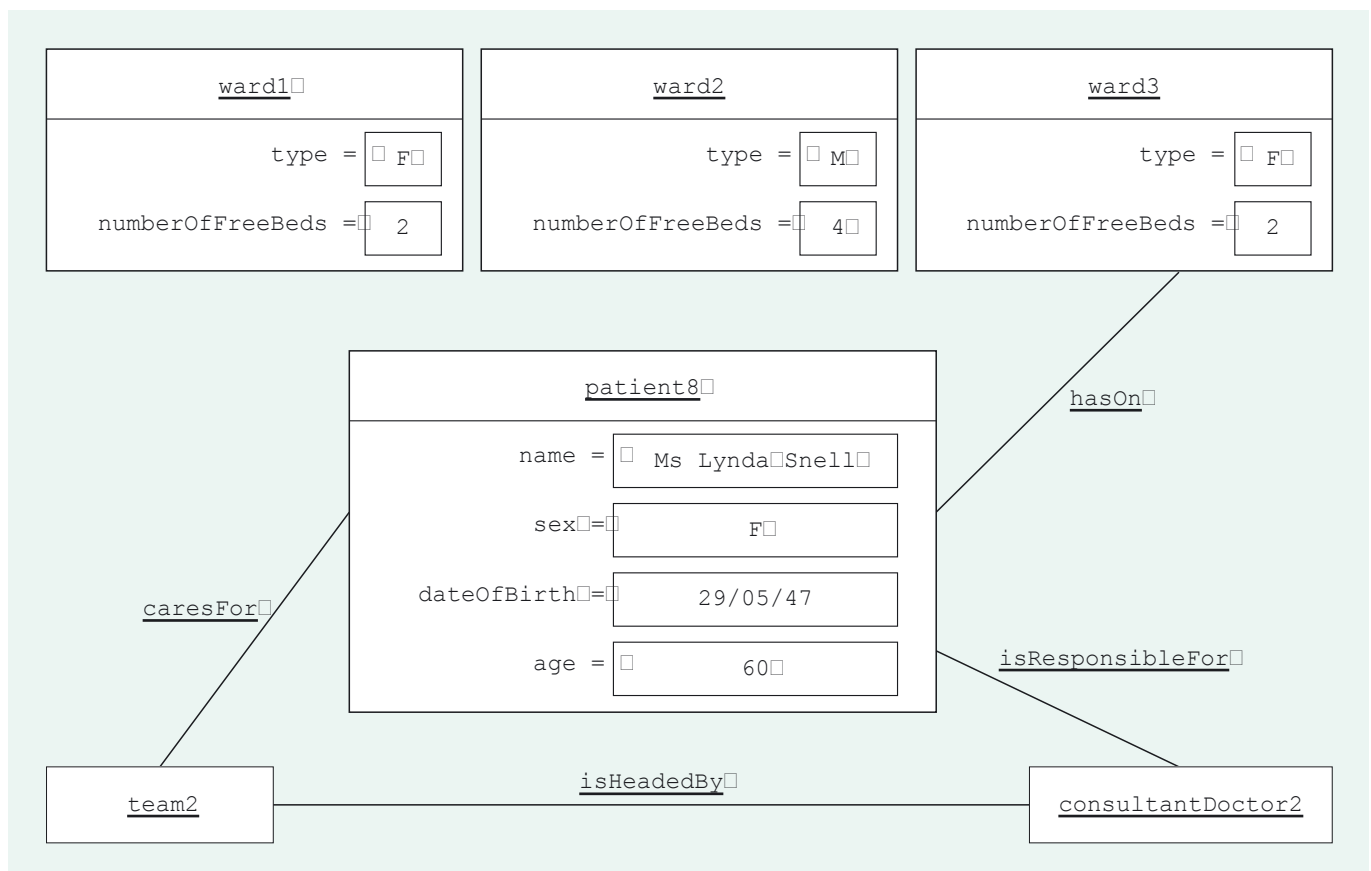


Figure 42 The classes and associations relevant to *Admit Patient*

From the class diagram, we can devise example objects for a typical scenario, to guide the design. The state of the core system for such a scenario before the use case is illustrated in the object diagram in Figure 43 below. Only the relevant objects and links are included, along with indications of certain attribute values that are relevant to this use case. The **Patient** object does not yet exist, of course.

Figure 43 Object diagram for the scenario before *Admit Patient*

After the use case has been employed to record the admission of the new patient, the relevant part of the core system is as shown in Figure 44 below.

Figure 44 Object diagram for the scenario after *Admit Patient*

For this use case, as with the previous one, certain restrictions are relevant to preserving the integrity of the system. The sex of a patient must match the type of their ward, a fact expressed in the following invariant:

The sex attribute of any Patient object has the same value as the type attribute of the linked Ward object.

Also relevant is the fact that the number of free beds on a ward cannot be less than 0.

In contrast to the previous use case, there is no restriction on the requirements for *Admit Patient*. Instead, the use case description specifies an alternative course of action when there is no appropriate ward with empty beds. (We will consider alternative courses of

The age attribute of a Patient object is derived from the date of birth and the current year (the scenario here reflects the situation in 2007).

Such 'common-sense' restrictions on attribute values were mentioned in Unit 3, but are not formally listed as invariants.

action in *Unit 7*.) However, our straightforward scenario assumes that there is a `Ward` object of the required type with at least one free bed in the 'before' state.

Specifying the coordinating method

What is required of this use case cannot be readily partitioned into separate coordinating methods, because the tasks it involves are all interconnected. A single coordinating method will be specified.

First, communication between the user interface and the coordinating object has to be considered.

In M256, the details required for creating a new object – in this case the name, sex and date of birth – will be passed from the user interface to the coordinating object. For our scenario we will take the details of the new patient to be as follows:

name Ms Lynda Snell, sex F, date of birth 29/05/47.

Also passed from the user interface to the coordinating object will be `team2`. Here, for the first time, not all of the arguments to the coordinating message are core objects: `team2` is a core object, but `Ms Lynda Snell`, for example, is a simple piece of data, required to furnish a new object with an attribute value.

At this design stage, it is not appropriate to get too involved in specifying attribute types; that is for more detailed design. In the appendix to this unit, however, you will see that to facilitate the specification of methods, such as the one currently being designed, we have given a type for each attribute of a class (`int`, for `capacity` of `Ward`, for example). At this stage, such choices are provisional, and will be reviewed in detail later. We will not discuss this further here; suffice to note that, when it is not obvious what sort of type will be appropriate, one is 'invented' in anticipation of its being fully specified later, when more is known about how it is used. At this stage, the following classes for input data will be assumed, matching the corresponding attribute types, with the detailed specification of `Name` and `Sex` being left for later:

- ▶ `Name`, for names;
- ▶ `Sex`, for sex (where `Sex` has values corresponding to `M` and `F`);
- ▶ `M256Date`, for dates.

For our scenario, the coordinating message will return `ward3`.

'Obvious' types might include integers, strings, etc.

The introduction of `Name`, for example, does not prevent the developer later on deciding that a simple string will suffice for recording a name.

Exercise 10

Sketch a sequence diagram illustrating communication between the user interface and the coordinating object, for our scenario.

Discussion.....

Our diagram is given in Figure 45.

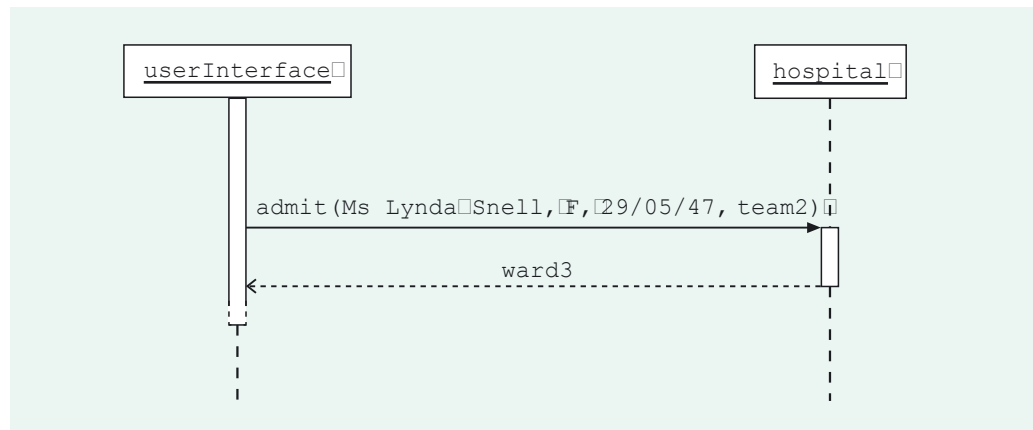


Figure 45 Communication with the user interface for the *Admit Patient* scenario

The next step is to specify the coordinating method that is invoked when the coordinating object receives a coordinating message such as `admit(Ms Lynda Snell, F, 29/05/47, team2)`.

Although for this straightforward scenario there is a suitable `Ward` object for the patient, in general such an object may not exist. The use case description states that the system reports if there is no suitable ward. A simple way to achieve this is for the coordinating method to return `null` in this situation. Upon receipt of this answer, the user interface will then report to the user that there is no bed available for the patient.

Here is our specification for the *Admit Patient* coordinating method.

```
Ward admit(Name aName, Sex aSex, M256Date aDate, Team aTeam)
```

Post-condition: if there is no `Ward` object of the appropriate type with at least one free bed then `null` is returned.

Otherwise a new `Patient` object, `aPatient`, is created with the supplied attribute values, and age according to `aDate`, and:

- (i) `aPatient` is linked to `aWard`, where `aWard` is a `Ward` object of the appropriate type with the greatest number of free beds, and `numberOfFreeBeds` of `aWard` is decremented;
- (ii) `aPatient` is linked to `aTeam`;
- (iii) `aPatient` is linked to the `ConsultantDoctor` object that is linked to `aTeam`;
- (iv) `aWard` is returned.

SAQ 16

Why is there no pre-condition specified for the above method?

ANSWER.....

There is no pre-condition for the above method because it does not require any restrictions. The requirements for the use case dictate what should happen in all cases and give no conditions on the state of the system or the values of the arguments. It is not necessary to ensure that there is an appropriate ward with a free bed before invoking the method; instead, as specified in the post-condition, the method takes an alternative course of action when this is not the case.

Outlining a walk-through and sequence diagram

Having specified the method, we now consider how it will work. Because it involves several tasks, it will be useful to create an outline of a walk-through, and then refine it as the steps are considered in detail, one by one.

Here is an outline walk-through for this scenario.

Given: name Ms Lynda Snell, sex F, date of birth 29/05/47, and team2, a Team object.

- 1 Find the Ward object with type F and the greatest value for numberOfFreeBeds, which in this case is ward3.
- 2 Create patient8, a new Patient object with the supplied attribute values and age according to the supplied date.
- 3 Create a hasOn link between ward3 and patient8, and reduce numberOfFreeBeds in ward3 by 1.
- 4 Create a caresFor link between team2 and patient8.
- 5 Access the ConsultantDoctor object linked to team2, that is, consultantDoctor2.
- 6 Create an isResponsibleFor link between consultantDoctor2 and patient8.
- 7 Return ward3.

SAQ 17

Which invariant (see Section 4 of the *Handbook*) does the following part of step 3 of the above walk-through ensure is preserved?

... and reduce numberOfFreeBeds in ward3 by 1.

ANSWER.....

The attribute numberOfFreeBeds must be decremented to preserve the following invariant (number 3 in the initial structural model).

For each Ward object, the value of its numberOfFreeBeds attribute is equal to the value of its capacity attribute minus the number of Patient objects to which it is linked.

SAQ 18

The association isResponsibleFor is, at this stage in the design process, one-way, navigated only from Patient to ConsultantDoctor.

- (a) Which object should have responsibility for creating the isResponsibleFor link in step 6? What should the object do?
- (b) Why is step 5 in the above walk-through required?

ANSWER.....

- (a) The object at the *from* end should have responsibility, as discussed in Subsection 5.1. Here, this is patient8, and it needs to record a reference to consultantDoctor2.
 - (b) In the subsequent step, step 6, a link is created between consultantDoctor2 and patient8. You have seen that to create a link, it is necessary to have access to the objects at each end of the link in order to tell the object at one end that it is to link itself to the object at the other end. The object consultantDoctor2 was not provided as an argument to the coordinating message, so to access the appropriate ConsultantDoctor object, the isHeadedBy association is navigated from the given Team object, team2, to the required ConsultantDoctor object, consultantDoctor2. This is the purpose of step 5.
-

Step 6 can now be refined to show the detail determined in SAQ 18:

6 `patient8` records a reference to `consultantDoctor2`.

Currently, we have only an outline walk-through, because some more steps need to be refined to identify all the responsibilities of the classes. However, it is possible at this stage to decide which object has overall responsibility for carrying out each step, that is, which object will initiate the actions required of other objects for that particular step, and so produce an outline sequence diagram. This will be tackled in the following subsections.

Creating an object

Step 1 (finding an appropriate ward) will be done by the coordinating object, `hospital`; it will, of course, need to access data held by the `Ward` objects, and we will discuss shortly how this could be done.

The designs in M256 will also make the coordinating object responsible for initiating the creation of new objects in the core system. Thus step 2 (creating a new `Patient` object) will be initiated by `hospital`.

The way the creation of a new object is modelled in a sequence diagram is shown below.

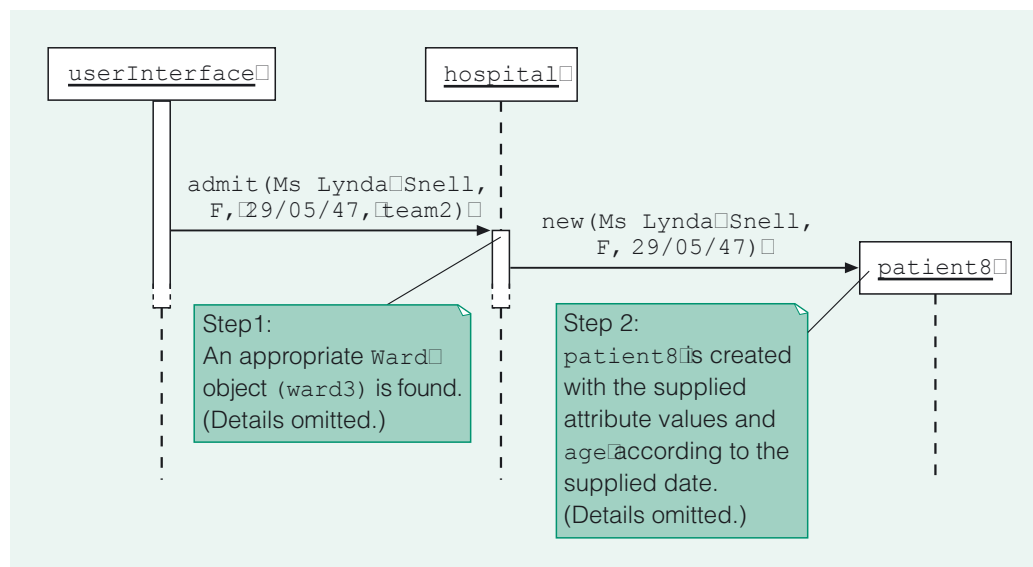


Figure 46 A new `Patient` object is created

The arrow labelled `new(Ms Lynda Snell, F, 29/05/47)` corresponds in Java to the invocation of a constructor using the `new` operator. The fact that `patient8` is newly constructed at this point in time is shown by drawing the `patient8` box at the arrowhead rather than at the top of the diagram.

Next in developing the outline sequence diagram, for each step that creates a link we will determine which object will have overall responsibility for managing what is involved. Later on, we will determine the details of what is involved.

Responsibility for managing the creation of a two-way link

You have seen that the way in which an association is navigated influences how a link of the association is created. If an association is navigated from a class `P` to a class `Q`, then, in creating a link of that association, the object at the *from* end must record a reference to the object at the *to* end.

Until now only one-way links – those navigated in only one direction – have been created, as you just saw in SAQ 18 for `isResponsibleFor`. In this use case, there are some two-way associations involved: `hasOn` is navigated in both directions (Patient to Ward and Ward to Patient), as is `caresFor` (Patient to Team and Team to Patient).

For a two-way association, the object at *each* end of the link will record a reference to the other object. However, as before, an object of the class at one end of an association will be made responsible for managing the process, i.e. for prompting the other object to take action, as well as performing its own actions.

We will choose to give `patient8` overall responsibility both for creating the one-way link (of the `isResponsibleFor` association) and for managing the creation of the two-way links (of the `hasOn` and `caresFor` associations). We will return shortly to the details of what these responsibilities for two-way link creation involve. But first, this decision results in the following outline sequence diagram, extending that shown in Figure 46.

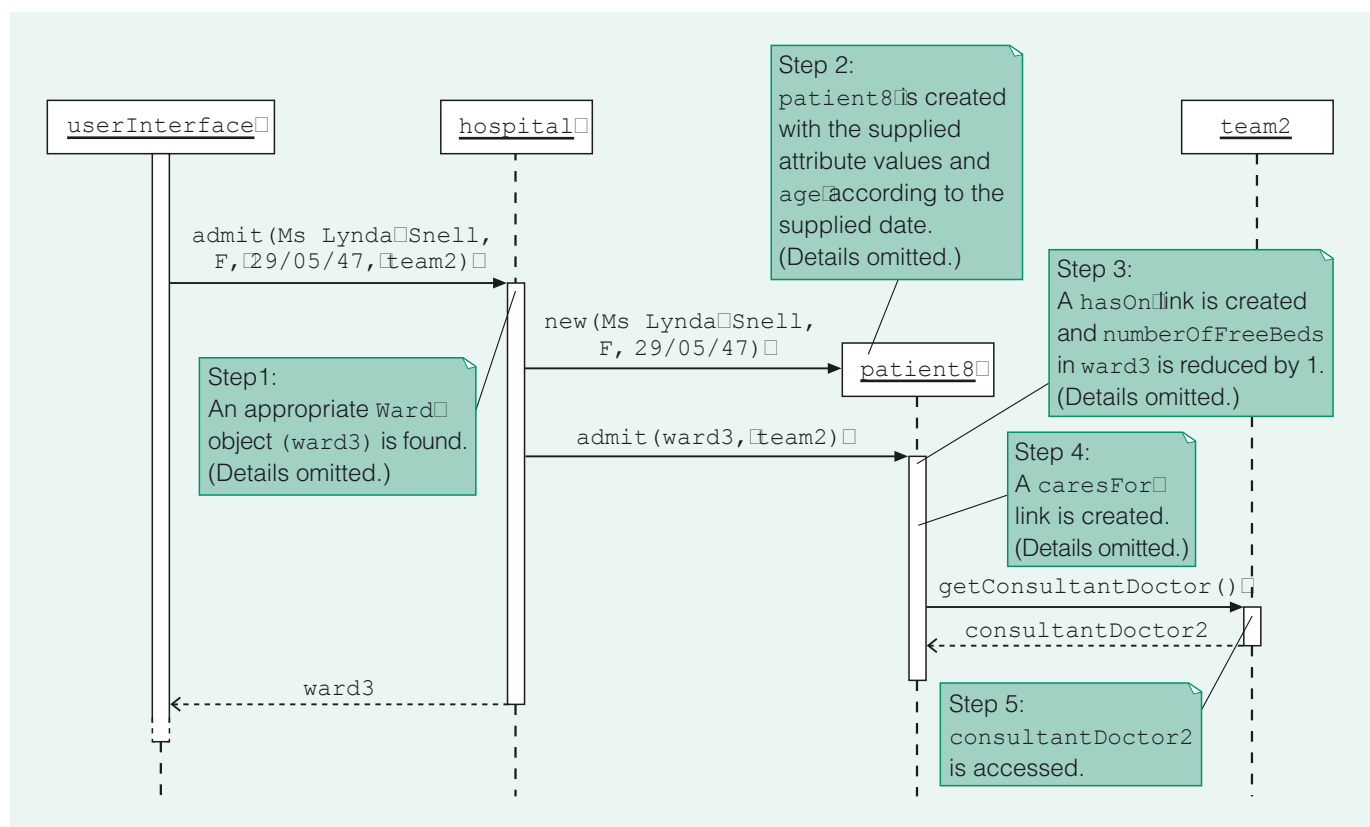


Figure 47 An outline sequence diagram for *Admit Patient*

SAQ 19

If the label in Figure 48 were to be added to the diagram in Figure 47, where should it point?

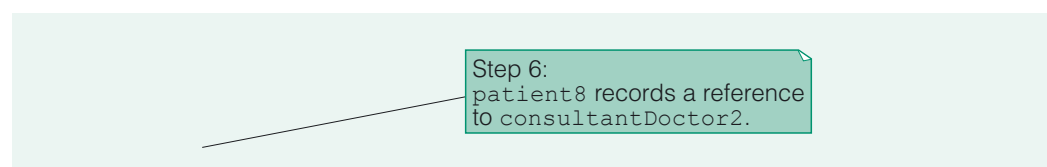


Figure 48 A label for step 6?

ANSWER.....

The object `patient8` can only record a reference to `consultantDoctor2` after it receives `consultantDoctor2` from `team2`. Therefore the label should point to the `patient8` activation rectangle, after `consultantDoctor2` is returned, as shown in Figure 49.

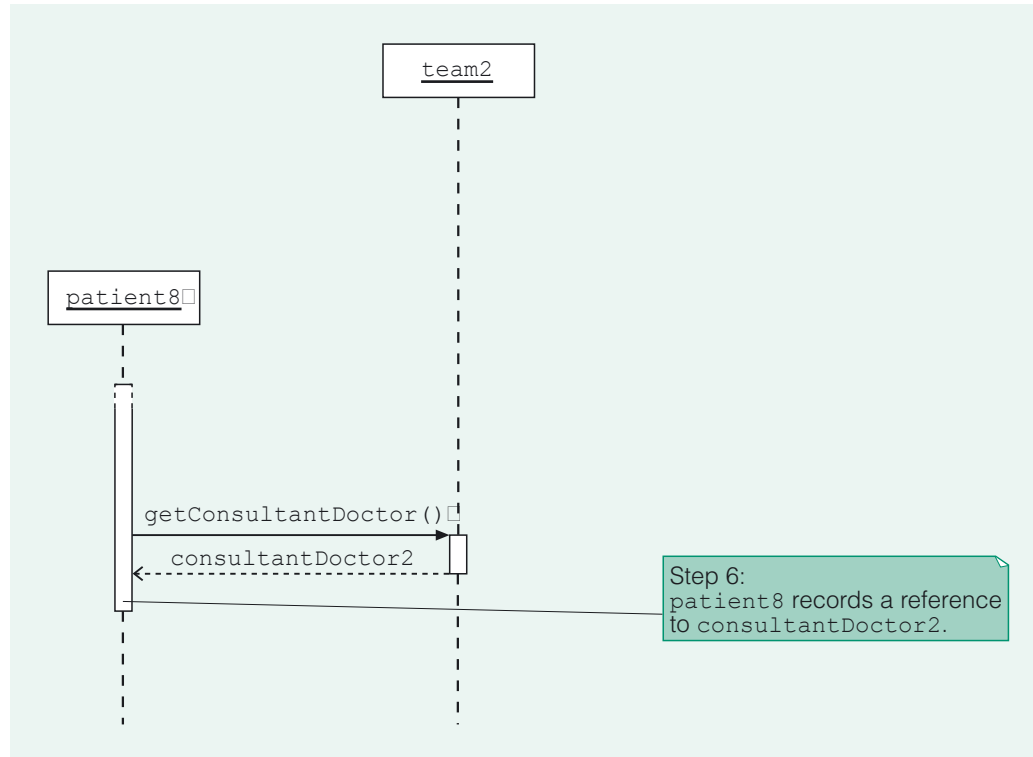


Figure 49 Where the label for step 6 should point

The sequence diagram in Figure 47 does not show all the message passing that is required, because we have not yet considered the details of steps 1, 3 and 4. Each of these may require the involvement of other objects in addition to the one with overall responsibility. We will now look at these in more detail.

Refining the walk-through and sequence diagram

Finding a ward

In step 1, how is `hospital` going to identify the `Ward` object corresponding to the ward to which the patient should be admitted? What it can do is to find from each `Ward` object whether it is female and if it is, how many free beds it has. The coordinating object will then be able to identify the female `Ward` object with the largest number of free beds. In terms of message passing, `hospital` sends a message to each `Ward` object asking it what type it is, then if the type is female, it sends a message asking how many free beds it has.

Since `hospital` has to send a message to each `Ward` object, it will need to be able to access all of them. To facilitate this, an association is required between `HospCoord` and `Ward`, which we will call `hasWards`. There needs to be a link from `hospital` to each `Ward` object. Adding an association was not necessary in previous use cases because the appropriate objects were always supplied to `hospital` in the coordinating messages.

We use `has` plus the plural of the class name for the name of an association between a class and the coordinating class.

Step 1 in the walk-through can now be refined as follows:

- 1a Access the collection of `Ward` objects linked to `hospital`, which in this case is `{ward1, ward2, ward3}`.

For each of these `Ward` objects:

- 1b find out its type;
 - 1c if `F`, get its number of free beds.
- 1d Identify `ward3`, the female `Ward` object with the greatest number of free beds.

Remember that, in our scenario, there is a female ward with a free bed.

Exercise 11

Draw a sequence diagram showing just the messages required for step 1 of the walk-through for the particular `Ward` objects in our scenario (given in Figure 43).

Discussion.....

Figure 50 shows a suitable sequence diagram. The attribute values are as illustrated in Figure 43.

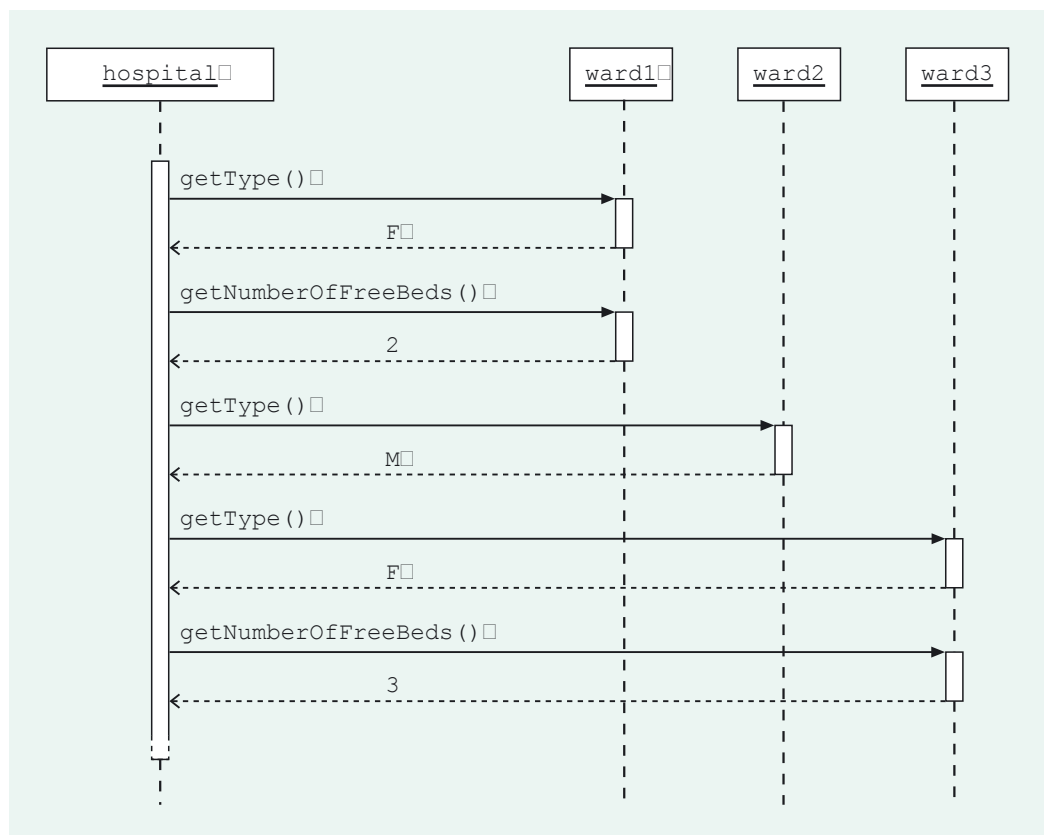


Figure 50 Step 1 in detail

Managing two-way link creation

We have given `patient8` overall responsibility for managing the creation of the links of the two-way associations `hasOn` and `caresFor`. But what does carrying out this responsibility involve?

For a two-way association, a link must be created in both directions at once. Two references must be set: the object at each end of the link must reference the other. We give overall responsibility for managing this link creation to the object at one end of the link. This means that as well as setting its own reference, the responsible object will need to send a message to the object at the other end telling it to set its reference too.

The sequence diagram of Figure 51 illustrates what happens when a two-way link is created between an object `p1` of class `P` and an object `q1` of class `Q`, where overall responsibility for managing this link creation has been assigned to `p1`.

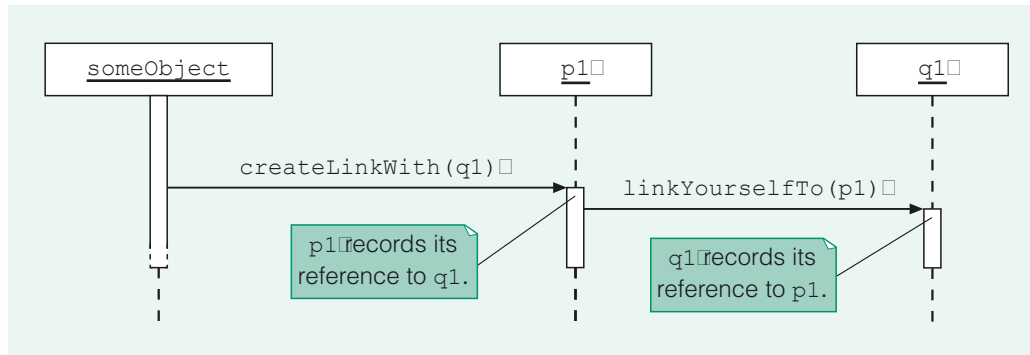


Figure 51 `p1` is responsible for creating a link of a two-way association

What happens in Figure 51 can be broken down as follows:

- 1 `p1` is sent a message, telling it to create the link with `q1`.
- 2 `p1` records its reference to `q1`.
- 3 `p1` then sends a message to `q1`, with itself (`p1`) as argument, telling `q1` to record that it is linked to `p1`.
- 4 `q1` records its reference to `p1`.

Now `p1` can access `q1` and `q1` can access `p1`.

Note that the corresponding Java message from `p1` to `q1` would be `linkYourselfTo(this)`.

SAQ 20

Figure 51 is based on the assumption that class `P` has a two-way association (call it `A`) with class `Q`. Use this example to answer the following:

- (a) In which direction(s) is `A` navigated?
- (b) Which object(s) has responsibility for holding link references for `A`?
- (c) Which object(s) has responsibility for managing the process of creating the links of `A`?

ANSWER.....

- (a) `A` is navigated in both directions: `P` to `Q` and `Q` to `P`.
- (b) Both `P` objects and `Q` objects have responsibility for holding link references for `A`: that is, a `P` object has the responsibility of holding a reference to a `Q` object (or several `Q` objects), and vice versa.
- (c) As shown in Figure 51, `P` objects have the responsibility for managing the process of link creation.

You will now see how the approach illustrated in Figure 51 works in a specific situation. Consider step 3 of the walk-through for *Admit Patient*, which is to create a link of the two-way association `hasOn`, and reduce `numberOfFreeBeds` in `ward3` by 1. This involves `patient8` recording a reference to `ward3`, and `ward3` recording a reference to `patient8`. Step 3 can therefore be refined into two parts:

- 3a `patient8` records a reference to `ward3`.
- 3b `ward3` records a reference to `patient8`, and reduces `numberOfFreeBeds` by 1.

Because `patient8` has overall responsibility for managing the creation of `hasOn` links, step 3b will be initiated by `patient8` sending a message to `ward3`. This is shown on the sequence diagram in Figure 52, where we have called the message `addPatient(aPatient)`.

Note that step 3b also involves changing an attribute: the value of `numberOfFreeBeds` is reduced by 1.

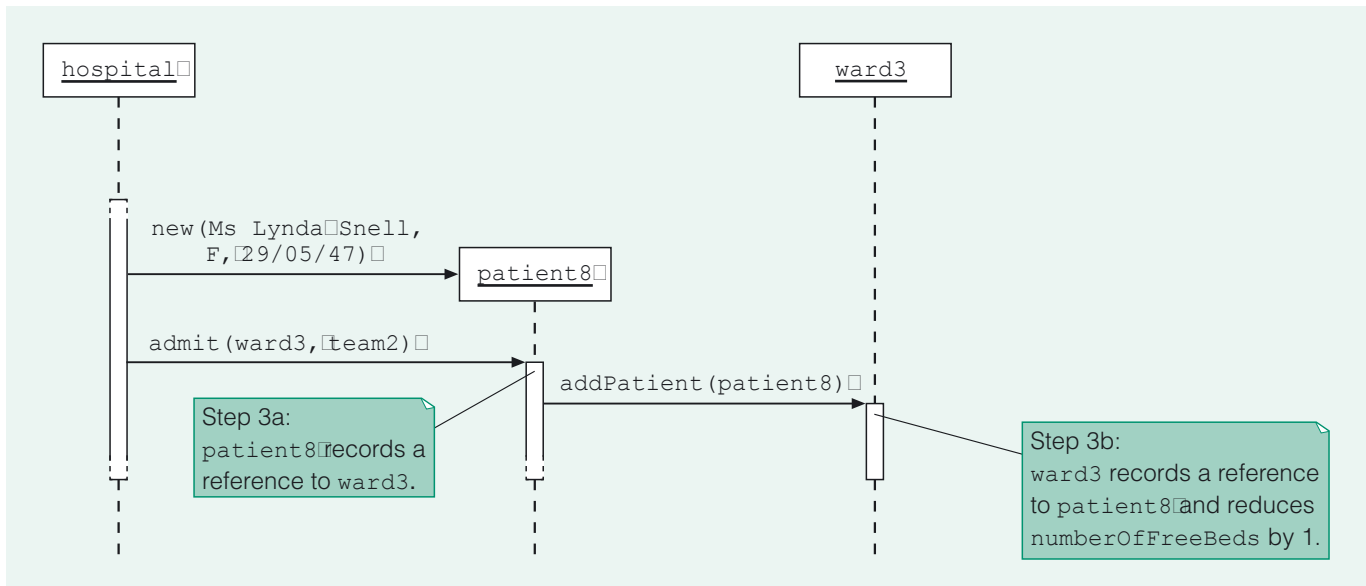


Figure 52 Creating a `hasOn` link

Step 4 can be refined in a similar manner to step 3.

SAQ 21

Write down the two steps into which step 4 should be split.

ANSWER.....

Step 4 should be split into the following.

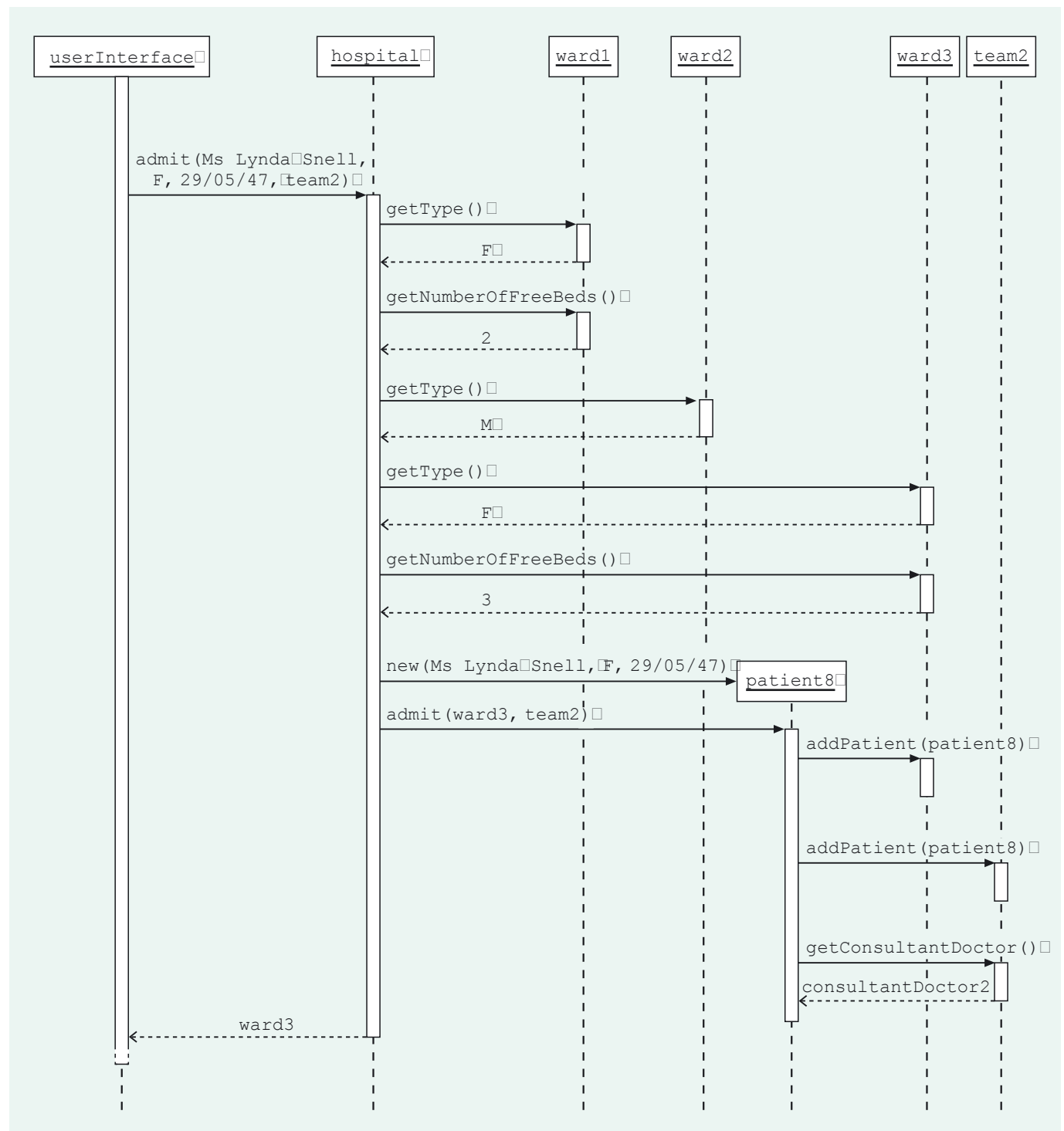
- 4a `patient8` records a reference to `team2`.
- 4b `team2` records a reference to `patient8`.

Now we can set out a complete walk-through and sequence diagram for this scenario.

Given: name Ms Lynda Snell, sex F, date of birth 29/05/47, and `team2`, a `Team` object.

- 1a Access the collection of `Ward` objects linked to `hospital`, which in this case is `{ward1, ward2, ward3}`.
For each of these `Ward` objects:
 - 1b find out its type;
 - 1c if F, get its number of free beds.
- 1d Identify `ward3`, the female `Ward` object with the greatest number of free beds.
- 2 Create `patient8`, a new `Patient` object with the supplied attribute values and age according to the supplied date.
- 3a `patient8` records a reference to `ward3`.
- 3b `ward3` records a reference to `patient8`, and reduces `numberOfFreeBeds` by 1.
- 4a `patient8` records a reference to `team2`.

- 4b team2 records a reference to patient8.
- 5 Access the ConsultantDoctor object linked to team2, that is, consultantDoctor2.
- 6 patient8 records a reference to consultantDoctor2.
- 7 Return ward3.

Figure 53 A refined sequence diagram for *Admit Patient*

Exercise 12

Identify, on the sequence diagram in Figure 53, where each step of the walk-through takes place.

Discussion.....

The annotated diagram in Figure 54 shows where each step takes place.

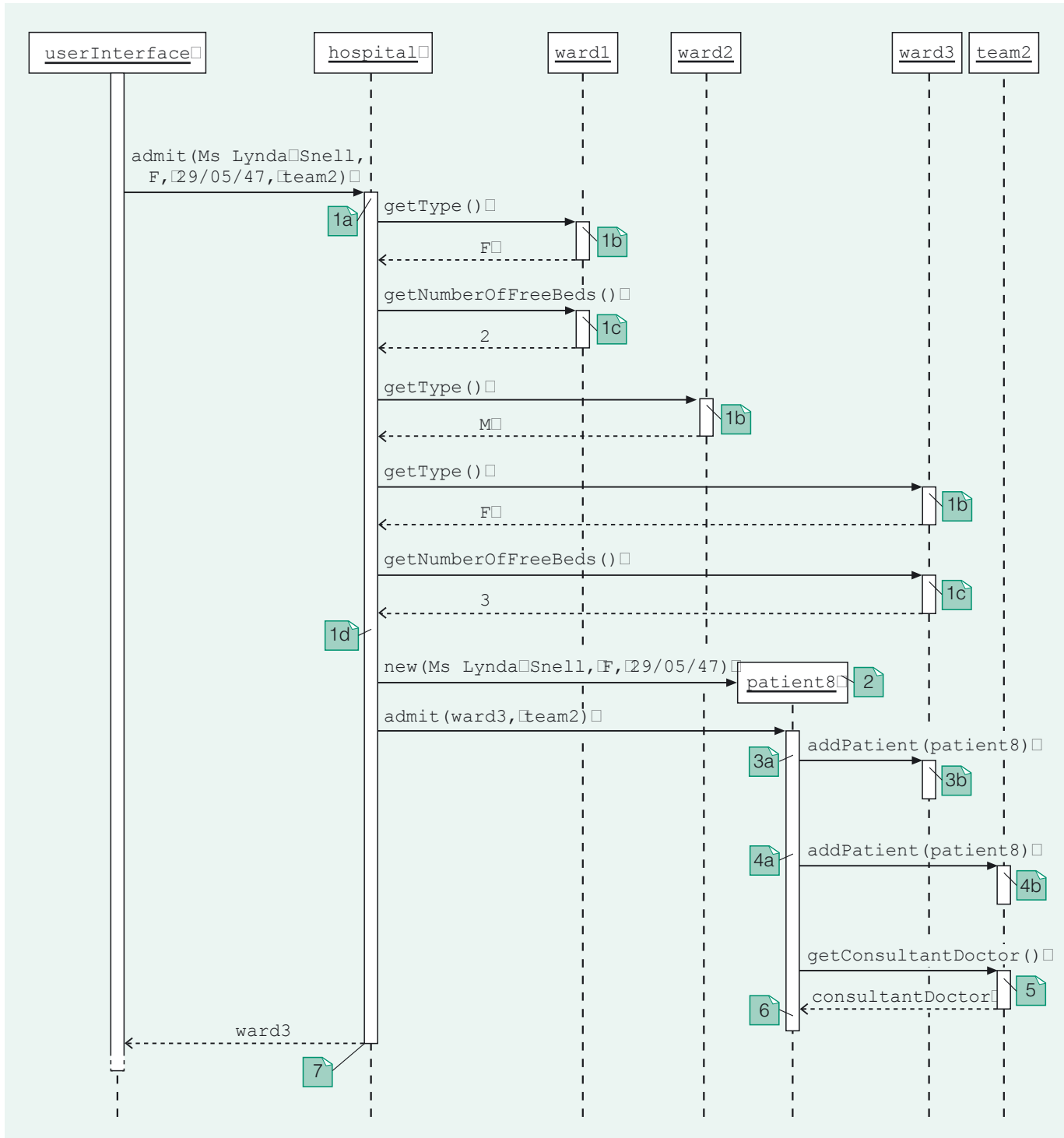


Figure 54 An annotated sequence diagram for *Admit Patient*

The sequence diagram looks complex because it models many different steps and the messages initiating those steps. Be assured that there is no point in memorising it, and indeed there is often no need to collate everything into a single diagram like this. If you find the complexity daunting, the important thing is to start by understanding what is going on in overview, then look at how the detail fits in. Below is a summary.

Summary of interactions for *Admit Patient*

A message is sent from the user interface to `hospital` to admit a new patient, under the care of a given team. The object `hospital` interrogates the `Ward` objects in turn to select an appropriate ward to which to admit the patient. A new `Patient` object is then created. Prompted by `hospital`, this new `Patient` object initiates the creation of two-way links to the `Ward` object that has been selected, and to the `Team` object given at the start. The `Patient` object enlists the help of the `Team` object in identifying the consultant who is responsible, and the `Patient` object records a link to the corresponding `ConsultantDoctor` object. Finally, the `Ward` object corresponding to the ward selected is returned to the user interface.

Updating the structural model

Below are the responsibilities for holding link references, the method specifications needed in each class and the required navigations. As you read, confirm, from the walk-through and sequence diagram, where the need for each arises.

Class HospCoord

Links

`Collection<Ward> wards`

References a collection of all `Ward` objects.

Protocol

`Ward admit(Name aName, Sex aSex, M256Date aDate, Team aTeam)`

Post-condition: if there is no `Ward` object of the appropriate type with at least one free bed then `null` is returned.

Otherwise a new `Patient` object, `aPatient`, is created with the supplied attribute values, and age according to `aDate`, and:

- (i) `aPatient` is linked to `aWard`, where `aWard` is a `Ward` object of the appropriate type with the greatest number of free beds, and `numberOfFreeBeds` of `aWard` is decremented;
- (ii) `aPatient` is linked to `aTeam`;
- (iii) `aPatient` is linked to the `ConsultantDoctor` object that is linked to `aTeam`;
- (iv) `aWard` is returned.

Class Ward

Protocol

`Sex getType()`

Post-condition: returns `type`.

`int getNumberOfFreeBeds()`

Post-condition: returns `numberOfFreeBeds`.

`void addPatient(Patient aPatient)`

Post-condition: a reference to `aPatient` is recorded; `numberOfFreeBeds` is decremented.

Class Team**Links**

ConsultantDoctor consultantDoctor

References the linked ConsultantDoctor object.

Protocol

void addPatient(Patient aPatient)

Post-condition: a reference to aPatient is recorded.

ConsultantDoctor getConsultantDoctor()

Post-condition: returns the linked ConsultantDoctor object.

Class Patient**Protocol**

void admit(Ward aWard, Team aTeam)

Post-condition: a reference to aWard is recorded, aWard records a reference to the receiver, and numberOfFreeBeds of aWard is decremented.

A reference to aTeam is recorded, and aTeam records a reference to the receiver.

A reference to the ConsultantDoctor object linked to aTeam is recorded.

In addition a constructor is required for the class Patient.

Constructor

Patient(Name aName, Sex aSex, M256Date aDate)

Post-condition: initialises a new Patient object with the given attribute values and age according to aDate.

The association hasWards is navigated from HospCoord to Ward.

The association isHeadedBy is navigated from Team to ConsultantDoctor.

In Subsection 5.1 you studied the design of a use case involving a change of state: the creation of a one-way link. In this subsection you have investigated a more complicated use case, which involved:

- ▶ creating an object;
- ▶ creating both one-way and two-way links;
- ▶ changing attribute values.

You also saw that the design necessitated the addition of an association to the class diagram, between HospCoord and Ward.

In creating this more complicated design, you have also learnt how complexity can be managed by first outlining a walk-through and sequence diagram, and then refining them.

Finally, the consequences of all the designs of this section (along with those of the previous section) have been incorporated in the structural part of an implementation model in the Appendix.

6

Summary

In this unit you have been introduced to the design phase of software development, through consideration of some of the Hospital System use cases.

We introduced a coordinating object as a simple way of facilitating communication between the separate user interface and core system components without sacrificing their independence.

The designs for two types of use case were considered: those involving the return of information; and those involving the more complicated process of changing the state. However, no matter how complicated the use case, the approach to its design followed the same process which is set out in Section 2 of the *Handbook*. For each use case, a typical scenario was illustrated by object diagrams, and a dynamic model of the core system in action was created in the form of a walk-through and sequence diagram for the scenario. From these models, we determined the responsibilities of each class for holding link references and providing methods, and the requirements for navigation of associations.

In this introduction, we simply set out one design for each use case. Our work here might be viewed as the first cycle in an iterative development process, where subsequent developments would consider further use cases, and also review and refine the work we have already done. In *Unit 7* we will evaluate some different designs, and consider some alternative, 'atypical' scenarios for those use cases we have already looked at.

All the design information for the Hospital System that was created in this unit has been gathered together in the Appendix. It represents the structural model for the Hospital System corresponding to its current stage of development, and we will build on it in subsequent units.

This unit also included a brief discussion of the use of anthropomorphic language.

LEARNING OUTCOMES

After studying this unit you should be able to:

- ▶ describe and use each of this unit's key terms (summarised in the Glossary);
- ▶ make appropriate use of models in designing a core system;
- ▶ divide the responsibility for a use case appropriately between the user interface and the core system;
- ▶ use a coordinating object to facilitate communication between the user interface and the core system, and to coordinate actions within the core system;
- ▶ construct a typical scenario for a use case, illustrating it using object diagrams;
- ▶ specify the coordinating method (or methods) for a use case;
- ▶ construct a walk-through and a sequence diagram for a typical scenario of a use case, incorporating as appropriate the navigation and creation of links, and the creation of objects;
- ▶ relate a design to a possible implementation;
- ▶ specify the responsibilities for holding link references, the methods and navigation required in the structural model as a consequence of a design expressed in a particular dynamic model;
- ▶ understand what is involved in the preservation of invariants, and where appropriate specify a pre-condition;
- ▶ use anthropomorphic language appropriately when discussing object-oriented software development.

Appendix

Hospital System implementation model (the structure so far)

For reference, this Appendix contains the structural model corresponding to the point we have reached in the design process at the end of *Unit 6*.

Class diagram

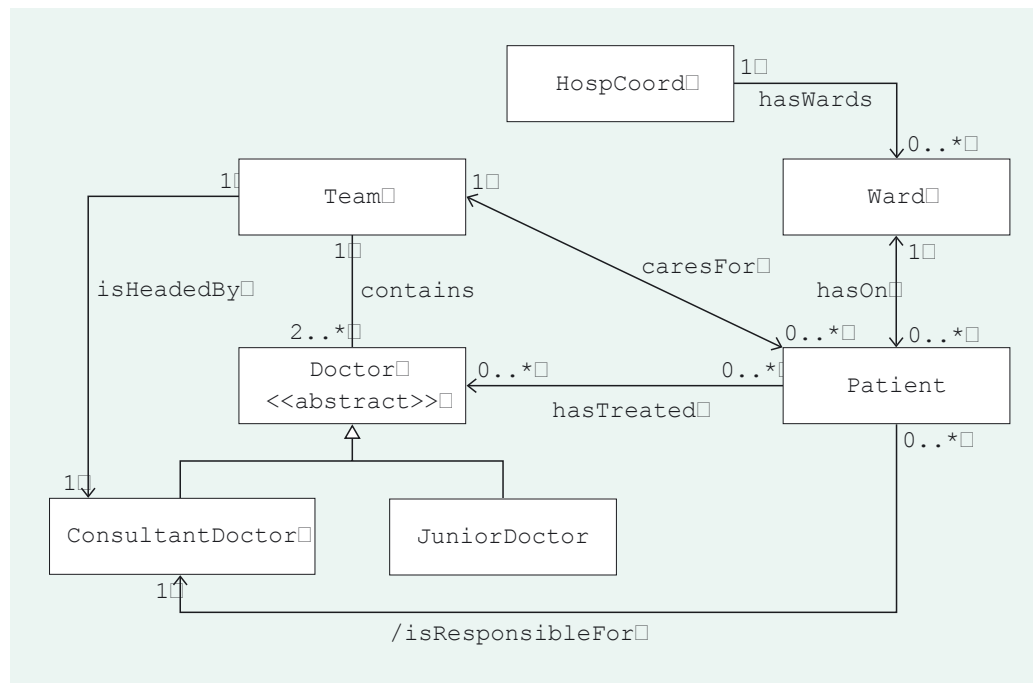


Figure 55 Class diagram showing navigations so far

Class descriptions

Class HospCoord The coordinating class

Attributes

None

Links

Collection<Ward> wards

References a collection of all Ward objects.

Protocol

Map<Patient, Ward> getPatientsAndWards(Team aTeam)

Post-condition: returns a map of pairs of Patient and Ward objects, such that for each Patient object aPatient linked to aTeam, the map contains the key-value pair (aPatient, aWard), where aWard is linked to aPatient.

ConsultantDoctor getConsultantDoctor(Patient aPatient)

Post-condition: returns the ConsultantDoctor object linked to aPatient.

```
Team getTeam(Patient aPatient)
```

Post-condition: returns the Team object linked to aPatient.

```
Collection<Doctor> getDoctors(Patient aPatient)
```

Post-condition: returns a collection of all the Doctor objects linked to aPatient.

```
void recordTreatment(Patient aPatient, Doctor aDoctor)
```

Pre-condition: aDoctor and aPatient must be linked to the same Team object.

Post-condition: aDoctor is linked to aPatient.

```
Ward admit(Name aName, Sex aSex, M256Date aDate, Team aTeam)
```

Post-condition: if there is no Ward object of the appropriate type with at least one free bed then null is returned.

Otherwise a new Patient object, aPatient, is created with the supplied attribute values, and age according to aDate, and

(i) aPatient is linked to aWard, where aWard is a Ward object of the appropriate type with the greatest number of free beds, and numberOfFreeBeds of aWard is decremented;

(ii) aPatient is linked to aTeam;

(iii) aPatient is linked to the ConsultantDoctor object that is linked to aTeam;

(iv) aWard is returned.

Class	Ward	A hospital ward
Attributes		
	String name	The unique name of the ward
	Sex type	Whether the ward is for male or female (M or F) patients
	int capacity	The maximum number of patients that can be on the ward at any one time
	int /numberOfFreeBeds	The number of free beds on the ward

Links

```
Collection<Patient> patients
```

References a collection of all the linked Patient objects.

Protocol

```
Collection<Patient> getPatients()
```

Post-condition: returns a collection of all the linked Patient objects.

```
Sex getType()
```

Post-condition: returns type.

```
int getNumberOfFreeBeds()
```

Post-condition: returns numberOfFreeBeds.

```
void addPatient(Patient aPatient)
```

Post-condition: a reference to aPatient is recorded; numberOfFreeBeds is decremented.

Class Team A team of doctors

Attributes

String code The unique code of the team

Links

Collection<Patient> patients

References a collection of all the linked Patient objects.

ConsultantDoctor consultantDoctor

References the linked ConsultantDoctor object.

Protocol

Map<Patient, Ward> getPatientsAndWards()

Post-condition: returns a map of pairs of Patient and Ward objects, such that for each Patient object, aPatient, linked to the receiver, the map contains the key-value pair (aPatient, aWard) where aWard is linked to aPatient.

void addPatient(Patient aPatient)

Post-condition: a reference to aPatient is recorded.

ConsultantDoctor getConsultantDoctor()

Post-condition: returns the linked ConsultantDoctor object.

Class Patient A patient in the hospital

Attributes

Name name The name of the patient

Sex /sex The sex (M or F) of the patient

M256Date dateOfBirth The date of birth of the patient

int /age The age of the patient in years

Links

Ward ward

References the linked Ward object.

ConsultantDoctor consultantDoctor

References the linked ConsultantDoctor object.

Team team

References the linked Team object.

Collection<Doctors> doctors

References a collection of all the linked Doctor objects.

Constructor

Patient(Name aName, Sex aSex, M256Date aDate)

Post-condition: initialises a new Patient object with the given attribute values and age according to aDate.

Protocol

Ward getWard()

Post-condition: returns the linked Ward object.

ConsultantDoctor getConsultantDoctor()

Post-condition: returns the linked ConsultantDoctor object.

Team getTeam()

Post-condition: returns the linked Team object.

Collection<Doctor> getDoctors()

Post-condition: returns a collection of all the linked Doctor objects.

void recordTreatmentBy(Doctor aDoctor)

Post-condition: a reference to aDoctor is recorded.

void admit(Ward aWard, Team aTeam)

Post-condition: a reference to aWard is recorded, aWard records a reference to the receiver, and numberOfFreeBeds of aWard is decremented. A reference to aTeam is recorded, and aTeam records a reference to the receiver. A reference to the ConsultantDoctor object linked to aTeam is recorded.

Class	Doctor	A doctor at the hospital Abstract: generalises JuniorDoctor and ConsultantDoctor
--------------	--------	--

Attributes

Name name	The name of the doctor
-----------	------------------------

Class	JuniorDoctor	A junior doctor at the hospital Specialises Doctor
--------------	--------------	---

Attributes

Grade grade	The grade (1, 2 or 3) of the junior doctor
-------------	--

Class	ConsultantDoctor	A consultant doctor at the hospital Specialises Doctor
--------------	------------------	---

Attributes

None

Glossary

anthropomorphism The attribution of human personality to what is not human (in the case of this course, to software entities).

coordinating class The class of the coordinating object.

coordinating message A message from the user interface to the coordinating object to initiate the core system behaviour required for a use case, or part of a use case.

coordinating method The method invoked on receipt of a coordinating message.

core object An object contained within the core system.

dynamic model A model illustrating actions and interactions within the system when the system is in operation. In this course, a dynamic model consists of a walk-through and a sequence diagram, in the context of a particular scenario.

implementation model A model specifying the actual classes that will be used to implement the system. It contains a structural model and dynamic models. Its structural model is the final result of the evolution of the initial structural model.

navigate To access an object or objects from another object by using a link or links of an association.

one-way association An association that is navigated in only one direction.

one-way link A link of a one-way association.

scenario An example (instance) of a use case involving particular example objects.

two-way association An association that is navigated in both directions.

two-way link A link of a two-way association.

walk-through A list of individual steps within the core system which carry out a use case in the context of particular scenario.

References

Dijkstra, E. (1985) *On anthropomorphism in science*, Talk delivered at The Philosophers' Lunch, 25 September 1985. Transcription available at <http://www.cs.utexas.edu/users/EWD/transcriptions/EWD09xx/EWD936.html> (Accessed 17 November 2006).

Wirfs-Brock, R., Wilkerson, B. and Lauren Wiener, L (1990) *Designing Object-Oriented Software*, Englewood Cliffs, NJ, Prentice Hall.

Index

A

anthropomorphic language 45

anthropomorphism 45

association

one-way 28

two-way 28

C

client 25

collaboration 25

component 9

conceptual model 5

coordinating

class 12

message 13

method 13, 20

method specification 22

object 10, 12

coordination of behaviour 14

core

object 9–10

system 9, 46

D

design 19

dynamic model 7, 19, 25

G

getter message 15

I

implementation model 6, 25

initial structural model 5

L

link 23

one-way 28

two-way 28, 61, 64

N

navigating an association 27

navigation 28

O

one-way

association 28

link 28

P

pre-condition 50

S

scenario 19, 28

sequence diagram 7, 18

server 25

structural model 6

system domain 46

T

two-way

association 28

link 28, 61, 64

U

user interface 9

W

walk-through 24

